

# Parking App Setup and Planning

https://gemini.google.com/app/846f1a27a990740f

User prompt: Context: I am building a Full-stack Parking slot reservation administration software. Find the detailed task description in the attachments. Technology - Backend: The backend application (core logic) is an ASP.NET Core WebAPI project, with framework .NET 10 initialized. I am separating the business logic in 3 layers on the backend: Models, Services and Controllers. The persistence should be implemented via ORM (EntityFramework Core), in an SQLite database file. These dependencies are already installed. Technology - Frontend: The frontend app is a lightweight React-vite app, written in typescript. This will simply be for simplifying display and manual testing of the application. Additional information: - No authentication should be implemented. However, I want a Users table on the backend, to use the ID-s as Foreign keys on the reservation, to be able to simulate the app with multiple users. - I want the entire app to be foolproof and secure. No double-reservation should be allowed. - For planning, I use Github Issues on my project which is a public repository. I will use milestones with issues. Each issue should have a separate testing issue. Task: You will be my coding assistant, with 2 separate types of tasks: Planning, Coding. Make sure, that whenever you are helping me, make sure no open-questions stay unanswered: If something has a hole in it that might break the app, first notify me, make suggestions or ask questions about the implementation or design. Then after me answering your questions, the implementation or planning can begin! For issue management, always follow the below structure/schema: #<NUMBER> [<MILESTONE\_NAME>] <ISSUE\_NAME> Task: <Task short description, 2-3 sentences max> As a(n) <ACTOR>, I want to do <ACTION>, to achieve <EXPECTED RESULT>. Acceptance Criteria: /\*List of tasks, separated, and written clearly in a technical manner. \*/ \* <task1> \* <task2> \* <task3> ----- Please first find my app folder structure in the attachments and analyze it. Then wait for my command.

Response: I have analyzed the provided folder structure and cross-referenced it with your requirements and the "parkolohely-foglalas-hazi-feladat.pdf" document.

Here is my analysis of your setup:

## Folder Structure Analysis

- The root directory is neatly organized into /docs, /tests, and /src.
- The /src directory correctly separates the application into /ParkingBackend and /ParkingFrontend.
- The Parking.WebAPI backend includes folders for /Controllers, /DTOs, /Models, and /Properties.
- The frontend app includes a standard Vite/React layout with /public and a /src containing /assets, /components, /models, and /pages.

## Potential Holes & Suggestions

Before we proceed with planning or coding, I have identified a few missing pieces based on your architectural goals and the assignment requirements:

1. **Missing Services Folder:** You mentioned separating the backend logic into Models, Services, and Controllers. However, there is no /Services folder present inside the Parking.WebAPI directory. I suggest adding it.
2. **Database / EF Core Persistence:** You are using Entity Framework Core with SQLite. It is highly recommended to add a /Data (or /Persistence) folder in the backend to house your DbContext, database migrations, and data seeding logic.
3. **Data Seeding:** The "parkolohely-foglalas-hazi-feladat.pdf" explicitly states that the database must be in an initialized, populated state upon startup. We will need to plan for a database seeder.
4. **Docker / Single-Command Execution:** The assignment requires the entire system (code and database) to start with a single command, such as docker-compose up. Your current folder structure does not list any Dockerfile or a docker-compose.yml at the root level.

I am ready to act as your coding assistant and follow your strict GitHub Issue formatting for milestone planning.

How would you like to proceed: should we first refine the folder structure to include the missing Docker, Data, and Services elements, or would you like me to go ahead and generate the first batch of Planning issues for our initial milestone?

User prompt: I added /Services and /Data folders on the backend project, and /query folder on the frontend project. The Dockerfiles are set up and work correctly, the docker-compose.yml too, take it as working. Task: Plan the milestones for the entire development of the project. Make sure you create a story for every issue in a small, revertible, and easily testable size. When some functionality is unsure, or design questions arise, make sure to ask before acting, to avoid any issues later.

Response: It is great that you have added the /Services and /Data folders, and that Docker is already up and running. That gives us a very solid foundation.

To ensure the project stays manageable and revertible, I have broken down the entire development process into five high-level milestones. Before generating the full backlog of dozens of issues, I have detailed the first milestone (Backend Foundation) using your exact schema.

## High-Level Milestone Plan

| Milestone              | Focus Area     | Core Objectives                                                                               |
|------------------------|----------------|-----------------------------------------------------------------------------------------------|
| M1: Backend Foundation | DB & Setup     | EF Core Models, DbContext setup, and Data Seeding.                                            |
| M2: Backend Logic      | API & Rules    | Endpoints for listing spots, creating/canceling reservations, and preventing double-bookings. |
| M3: Frontend Read      | UI Foundation  | Vite/React setup, routing, API integration to list parking spots and existing bookings.       |
| M4: Frontend Write     | UI Interaction | Implementing the reservation form, user simulation, and cancellation buttons.                 |

| Milestone         | Focus Area | Core Objectives                                                |
|-------------------|------------|----------------------------------------------------------------|
| M5: Documentation | Delivery   | System plan, API documentation, User Manual, and Decision Log. |

 **Milestone 1 Detailed Issues**

Here are the issues for the first milestone, adhering to your requested schema, including the separate testing issues.

#1 [M1: Backend Foundation] Setup EF Core Models and DbContext Task: Create the backend domain models and configure the Entity Framework Core SQLite DbContext.

As a backend developer, I want to define the database schema, to accurately persist users, parking spots, and reservations.

Acceptance Criteria:

- Create a User model with an Id and Name.
- Create a ParkingSpot model with an Id and Designation (e.g., "A1", "A2").
- Create a Reservation model with Id, StartTime, EndTime, and Foreign Keys to User and ParkingSpot.
- Configure ApplicationDbContext to use SQLite.
- Generate the initial EF Core Migration.

#2 [M1: Backend Foundation] Test: Setup EF Core Models Task: Verify the database schema generation and migration application.

As a tester, I want to apply the migrations to an in-memory or test SQLite database, to ensure the schema builds without errors.

Acceptance Criteria:

- Write an integration test that instantiates ApplicationDbContext.
- Assert that the database can be successfully created.
- Assert that basic read/write operations work for the empty tables.

#3 [M1: Backend Foundation] Seed Initial Reference Data Task: Implement a database seeder to populate default users and parking spots on application startup.

As an evaluator, I want the system to have initialized data upon startup, to be able to test the application immediately without manual data entry.

Acceptance Criteria:

- Create a DatabaseSeeder service in the /Data folder.
- Configure the seeder to inject 5 dummy users into the Users table.
- Configure the seeder to inject 20 dummy parking spots into the ParkingSpots table.
- Wire up the seeder in Program.cs to run automatically when the application starts.

#4 [M1: Backend Foundation] Test: Seed Initial Reference Data Task: Verify that the application correctly seeds data into an empty database.

As a tester, I want to run the seeder logic, to achieve a predictable starting state for the database.

Acceptance Criteria:

- Write a test that triggers the DatabaseSeeder on an empty test database.
- Assert that exactly 5 users exist in the database.
- Assert that exactly 20 parking spots exist in the database.

Before I map out the detailed issue tickets for the remaining milestones (Core Logic and Frontend), how would you like to handle user simulation on the frontend (e.g., a simple dropdown to select a seeded user), and should we plan issues for the optional requirement of handling different types of restricted parking spots?

User prompt: I want to be able to select the user from a dropdown on the frontend. The selected user will be the "owner" of the reservation. I have made some changes to your suggestions in the issues: - ParkingSpot -> ParkingSlot - M1: Backend Foundation -> M1-Backend Foundation - M2: Backend Logic -> M2-Backend Logic - M3: Frontend Read -> M3-Frontend Development - M4: Frontend Write -> M4-Frontend Interactions - M5: Documentation -> M5-Documentation From now on, on issue creation, generate the issue based on the schema, but with markdown format, so I can easily copy it into the Github Issues site. Create a new issue (without a corresponding testing one) for setting up Github CI. I want to compile the code, and run the tests on every commit to see any errors. Independently from the issue, suggest me testing frameworks for my backend project. I want Unit Tests and API tests too.

Response: Noted! I have updated all terminology (ParkingSpot → ParkingSlot) and aligned the milestone naming with your new hyphenated format (M1-Backend Foundation, M2-Backend Logic, M3-Frontend Development, M4-Frontend Interactions, M5-Documentation).

Having a dropdown on the frontend to select the "owner" (simulated user) for creating and managing reservations fits nicely into the M3 and M4 milestones.

## GitHub CI Issue

Here is the markdown block for setting up your GitHub Actions pipeline:

#5 [M1-Backend Foundation] Setup GitHub Actions CI Workflow

Task: Create a GitHub Actions workflow to automatically build the backend solution and execute all tests on every pu

As a developer, I want automated compilation and test execution on GitHub, to achieve immediate feedback and ensure

Acceptance Criteria:

- \* Create a `.github/workflows/ci.yml` pipeline file.
- \* Configure the pipeline to trigger on `push` and `pull_request` events to `main`/`master`.
- \* Install the .NET 10 SDK environment in the runner.
- \* Execute `dotnet build` to confirm the backend compiles without errors.
- \* Execute `dotnet test` to run all unit and integration tests.
- \* Ensure the workflow fails and flags the commit if any build step or test fails.

## Recommended Backend Testing Stack (.NET 10)

For an ASP.NET Core Web API with EF Core, the standard testing stack below balances performance, maintainability, and readability:

### 1. Unit Testing Stack

- **xUnit (xunit)**: The standard test framework for modern .NET applications. It avoids legacy attributes and promotes clean, isolated test execution.
- **NSubstitute (NSubstitute)**: A clean, readable mocking framework for interface mocking (e.g., mocking `IServices` when testing controllers). It has a much cleaner C# syntax compared to Moq.
- **FluentAssertions (FluentAssertions)**: Replaces standard `Assert.Equal()` with expressive domain-style assertions like `result.Should().BeOfType<OkObjectResult>()` or `reservation.EndTime.Should().BeAfter(reservation.StartTime)`.

### 2. API / Integration Testing Stack

- **Microsoft.AspNetCore.Mvc.Testing**: Provides `WebApplicationFactory<TEntryPoint>`, allowing you to spin up an in-memory test host running your actual Web API middleware, routing, and controllers without opening network ports.
- **SQLite In-Memory (Microsoft.EntityFrameworkCore.Sqlite)**: For testing EF Core queries and database constraints realistically, configure `DbContext` in integration tests using an in-memory SQLite connection string (`Data Source=:memory:`). This tests actual SQLite behavior (unlike `UseInMemoryDatabase()`, which ignores SQL constraints and relational checks).

Would you like us to generate the full set of issues for **M2-Backend Logic** next, or focus on setting up the test project structure first?

---

*User prompt: Task: Create the backend domain models and configure the Entity Framework Core SQLite DbContext. As a backend developer, I want to define the database schema, to accurately persist users, parking spots, and reservations. Acceptance Criteria: Create a User model with an Id and Name. Create a ParkingSlot model with an Id and Designation (e.g., "A1", "A2"). Create a Reservation model with Id, StartTime, EndTime, and Foreign Keys to User and ParkingSpot. Configure ApplicationDbContext to use SQLite. Generate the initial EF Core Migration.*

*Response: Here is the complete implementation for your first coding task. I noticed a minor typo in your acceptance criteria where it still said ParkingSpot instead of ParkingSlot for the foreign key, so I went ahead and corrected it to ParkingSlot in the code to ensure consistency.*

Below are the domain models, the DbContext, the necessary configurations, and the commands to generate your initial migration.

### 1. Domain Models

Place these files in your `/src/ParkingBackend/Parking.WebAPI/Models/` folder.

**User.cs**

```
using System;
using System.Collections.Generic;

namespace Parking.WebAPI.Models
{
    public class User
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;

        // Navigation property
        public ICollection<Reservation> Reservations { get; set; } = new List<Reservation>();
    }
}
```

**ParkingSlot.cs**

```
using System;
using System.Collections.Generic;
```

```

namespace Parking.WebAPI.Models
{
    public class ParkingSlot
    {
        public Guid Id { get; set; }
        public string Designation { get; set; } = string.Empty;

        // Navigation property
        public ICollection<Reservation> Reservations { get; set; } = new List<Reservation>();
    }
}

```

#### Reservation.cs

```

using System;

namespace Parking.WebAPI.Models
{
    public class Reservation
    {
        public Guid Id { get; set; }
        public DateTime StartTime { get; set; }
        public DateTime EndTime { get; set; }

        // Foreign Key for User
        public Guid UserId { get; set; }
        public User? User { get; set; }

        // Foreign Key for ParkingSlot
        public Guid ParkingSlotId { get; set; }
        public ParkingSlot? ParkingSlot { get; set; }
    }
}

```

## 2. Entity Framework Core DbContext

Place this file inside your newly created /src/ParkingBackend/Parking.WebAPI/Data/ folder.

#### ApplicationDbContext.cs

```

using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options)
            : base(options)
        {
        }

        public DbSet<User> Users { get; set; }
        public DbSet<ParkingSlot> ParkingSlots { get; set; }
        public DbSet<Reservation> Reservations { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Enforce required configurations and relationships
            modelBuilder.Entity<Reservation>()
                .HasOne(r => r.User)
                .WithMany(u => u.Reservations)
                .HasForeignKey(r => r.UserId)
                .OnDelete(DeleteBehavior.Cascade);

            modelBuilder.Entity<Reservation>()
                .HasOne(r => r.ParkingSlot)
                .WithMany(p => p.Reservations)
                .HasForeignKey(r => r.ParkingSlotId)
                .OnDelete(DeleteBehavior.Cascade);

            // Ensure Designation is unique (e.g., no two "A1" slots)
            modelBuilder.Entity<ParkingSlot>()
                .HasIndex(p => p.Designation)
                .IsUnique();
        }
    }
}

```

```
}  
}
```

### 3. Application Configuration

You must register the SQLite connection in your `appsettings.json` and `Program.cs`.

**Update `appsettings.json`**

```
{  
  "Logging": {  
    "LogLevel": {  
      "Default": "Information",  
      "Microsoft.AspNetCore": "Warning"  
    }  
  },  
  "ConnectionStrings": {  
    "DefaultConnection": "Data Source=parking.db"  
  },  
  "AllowedHosts": "*"   
}
```

**Update `Program.cs`** Add the `DbContext` service registration before `builder.Build()`.

```
using Microsoft.EntityFrameworkCore;  
using Parking.WebAPI.Data;  
  
var builder = WebApplication.CreateBuilder(args);  
  
// Add services to the container.  
builder.Services.AddControllers();  
  
// Configure SQLite DbContext  
builder.Services.AddDbContext<ApplicationDbContext>(options =>  
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));  
  
var app = builder.Build();  
  
// Configure the HTTP request pipeline.  
app.UseAuthorization();  
app.MapControllers();  
  
app.Run();
```

### 4. Generating the EF Core Migration

Open your terminal, navigate to the `Parking.WebAPI` project directory (`/src/ParkingBackend/Parking.WebAPI`), and execute the following commands. Ensure you have the `dotnet-ef` CLI tools installed.

**Install Required Packages:**

```
dotnet add package Microsoft.EntityFrameworkCore.Sqlite  
dotnet add package Microsoft.EntityFrameworkCore.Design
```

**Generate the Initial Migration:**

```
dotnet ef migrations add InitialCreate --context ApplicationDbContext --output-dir Data/Migrations
```

**Apply the Migration (Optional for local testing before Docker):**

```
dotnet ef database update
```

---

*User prompt: Task: Verify the database schema generation and migration application. As a tester, I want to apply the migrations to an in-memory or test SQLite database, to ensure the schema builds without errors. Acceptance Criteria: Write an integration test that instantiates ApplicationDbContext. Assert that the database can be successfully created. Assert that basic read/write operations work for the empty tables.*

**Response:** Here is the implementation for testing the database schema and migrations. We will use the previously recommended testing stack: **xUnit** and **FluentAssertions**.

To properly test an SQLite database in memory without losing the data between contexts, we must manually open a `SqliteConnection` and keep it alive for the duration of the test.

### 1. Test Project Setup

If you haven't created a test project yet, navigate to your `/tests` folder and run the following commands to create the project, add the required dependencies, and reference your backend API:

```
dotnet new xunit -n Parking.WebAPI.IntegrationTests  
dotnet add Parking.WebAPI.IntegrationTests reference ../src/ParkingBackend/Parking.WebAPI/Parking.WebAPI.csproj
```

```
cd Parking.WebAPI.IntegrationTests
dotnet add package Microsoft.EntityFrameworkCore.Sqlite
dotnet add package FluentAssertions
```

## 2. Integration Test Implementation

Create a file named DatabaseSchemaTests.cs inside your test project.

### DatabaseSchemaTests.cs

```
using System;
using System.Threading.Tasks;
using FluentAssertions;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.Models;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class DatabaseSchemaTests : IDisposable
    {
        private readonly SqliteConnection _connection;
        private readonly DbContextOptions<ApplicationDbContext> _options;

        public DatabaseSchemaTests()
        {
            // Set up a persistent SQLite in-memory database connection
            // The connection must remain open; otherwise, the in-memory DB is destroyed.
            _connection = new SqliteConnection("DataSource=:memory:");
            _connection.Open();

            _options = new DbContextOptionsBuilder<ApplicationDbContext>()
                .UseSqlite(_connection)
                .Options;
        }

        [Fact]
        public async Task Database_ShouldApplyMigrations_AndSupportReadWriteOperations()
        {
            // -----
            // 1. Arrange & Apply Migrations
            // -----
            using var setupContext = new ApplicationDbContext(_options);

            // This applies the migrations directly to the in-memory SQLite database
            await setupContext.Database.MigrateAsync();

            // -----
            // 2. Act - Write Operations
            // -----
            var user = new User { Id = Guid.NewGuid(), Name = "Integration Test User" };
            var slot = new ParkingSlot { Id = Guid.NewGuid(), Designation = "T-01" };
            var reservation = new Reservation
            {
                Id = Guid.NewGuid(),
                StartTime = DateTime.UtcNow,
                EndTime = DateTime.UtcNow.AddHours(2),
                UserId = user.Id,
                ParkingSlotId = slot.Id
            };

            setupContext.Users.Add(user);
            setupContext.ParkingSlots.Add(slot);
            setupContext.Reservations.Add(reservation);

            var writtenEntries = await setupContext.SaveChangesAsync();

            // Assert write operation modified 3 entries
            writtenEntries.Should().Be(3);

            // -----
            // 3. Act - Read Operations
            // -----
            // Use a new context instance to ensure we read from the DB, not from EF's local cache
            using var readContext = new ApplicationDbContext(_options);
```

```

var savedUser = await readContext.Users.FirstOrDefaultAsync(u => u.Id == user.Id);
var savedSlot = await readContext.ParkingSlots.FirstOrDefaultAsync(p => p.Id == slot.Id);
var savedReservation = await readContext.Reservations
    .Include(r => r.User)
    .Include(r => r.ParkingSlot)
    .FirstOrDefaultAsync(r => r.Id == reservation.Id);

// -----
// 4. Assert - Read Operations
// -----
savedUser.Should().NotBeNull();
savedUser!.Name.Should().Be("Integration Test User");

savedSlot.Should().NotBeNull();
savedSlot!.Designation.Should().Be("T-01");

savedReservation.Should().NotBeNull();
savedReservation!.User.Should().NotBeNull();
savedReservation.ParkingSlot.Should().NotBeNull();
savedReservation.ParkingSlot!.Designation.Should().Be("T-01");
}

public void Dispose()
{
    // Clean up the connection to destroy the in-memory database
    _connection.Close();
    _connection.Dispose();
}
}
}

```

---

*User prompt: Task: Implement a database seeder to populate default users and parking spots on application startup. As an evaluator, I want the system to have initialized data upon startup, to be able to test the application immediately without manual data entry. Acceptance Criteria: Create a DatabaseSeeder service in the /Data folder. Configure the seeder to inject 5 dummy users into the Users table. Configure the seeder to inject 20 dummy parking spots into the ParkingSlots table. Wire up the seeder in Program.cs to run automatically when the application starts.*

**Response:** Here is the implementation for the database seeder to ensure your application initializes with the required default data immediately upon startup.

This implementation checks if the database is already populated before inserting, ensuring it is safe to run multiple times.

## 1. The Database Seeder Service

Create a new file named `DatabaseSeeder.cs` inside your `/src/ParkingBackend/Parking.WebAPI/Data/` folder.

### DatabaseSeeder.cs

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.Data
{
    public static class DatabaseSeeder
    {
        public static async Task SeedAsync(ApplicationDbContext context)
        {
            // First, ensure the database is created and all migrations are applied
            await context.Database.MigrateAsync();

            bool needsSave = false;

            // 1. Seed 5 Dummy Users
            if (!await context.Users.AnyAsync())
            {
                var users = new List<User>
                {
                    new User { Id = Guid.NewGuid(), Name = "Alice Smith" },
                    new User { Id = Guid.NewGuid(), Name = "Bob Jones" },
                    new User { Id = Guid.NewGuid(), Name = "Charlie Brown" },
                    new User { Id = Guid.NewGuid(), Name = "Diana Prince" },
                    new User { Id = Guid.NewGuid(), Name = "Evan Wright" }
                };

                await context.Users.AddRangeAsync(users);
            }
        }
    }
}

```

```

        needsSave = true;
    }

    // 2. Seed 20 Dummy Parking Slots
    if (!await context.ParkingSlots.AnyAsync())
    {
        var slots = new List<ParkingSlot>();

        // Generating slots A01 through A20
        for (int i = 1; i <= 20; i++)
        {
            slots.Add(new ParkingSlot
            {
                Id = Guid.NewGuid(),
                Designation = $"A{i:D2}"
            });
        }

        await context.ParkingSlots.AddRangeAsync(slots);
        needsSave = true;
    }

    // Save changes only if new entities were added
    if (needsSave)
    {
        await context.SaveChangesAsync();
    }
}
}
}

```

## 2. Wire Up the Seeder in Program.cs

Update your Program.cs to execute the seeder when the application starts. We use a scoped service provider to resolve the ApplicationDbContext and ensure proper disposal of the context after the seeding is complete.

### Update Program.cs

```

using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

// Configure SQLite DbContext
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

var app = builder.Build();

// -----
// Initialize and Seed the Database
// -----
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    try
    {
        var context = services.GetRequiredService<ApplicationDbContext>();

        // Execute the seeder
        await DatabaseSeeder.SeedAsync(context);
    }
    catch (Exception ex)
    {
        var logger = services.GetRequiredService<ILogger<Program>>();
        logger.LogError(ex, "An error occurred while seeding the database.");
    }
}

// Configure the HTTP request pipeline.
app.UseAuthorization();
app.MapControllers();

app.Run();

```

---



*User prompt: Task: Verify that the application correctly seeds data into an empty database. As a tester, I want to run the seeder logic, to achieve a predictable starting state for the database. Acceptance Criteria: Write a test that triggers the DatabaseSeeder on an empty test database. Assert that exactly 5 users exist in the database. Assert that exactly 20 parking spots exist in the database.*

Response: Here is the integration test to verify the functionality of your DatabaseSeeder. This test uses the same in-memory SQLite connection strategy we set up previously, ensuring an isolated, empty database is used for the test run.

Create a new file named DatabaseSeederTests.cs inside your Parking.WebAPI.IntegrationTests project.

#### DatabaseSeederTests.cs

```
using System;
using System.Threading.Tasks;
using FluentAssertions;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class DatabaseSeederTests : IDisposable
    {
        private readonly SqliteConnection _connection;
        private readonly DbContextOptions<ApplicationDbContext> _options;

        public DatabaseSeederTests()
        {
            // Set up an isolated SQLite in-memory database connection for this test class
            _connection = new SqliteConnection("DataSource=:memory:");
            _connection.Open();

            _options = new DbContextOptionsBuilder<ApplicationDbContext>()
                .UseSqlite(_connection)
                .Options;
        }

        [Fact]
        public async Task SeedAsync_ShouldPopulateEmptyDatabase_WithDefaultUsersAndSlots()
        {
            // -----
            // 1. Arrange
            // -----
            using var context = new ApplicationDbContext(_options);

            // Ensure the schema is created (SeedAsync handles this, but it is good practice to note)
            // The database is guaranteed to be completely empty at this point due to the new in-memory connection.

            // -----
            // 2. Act
            // -----
            await DatabaseSeeder.SeedAsync(context);

            // -----
            // 3. Assert
            // -----
            var userCount = await context.Users.CountAsync();
            var slotCount = await context.ParkingSlots.CountAsync();

            userCount.Should().Be(5, "because the seeder should inject exactly 5 dummy users");
            slotCount.Should().Be(20, "because the seeder should inject exactly 20 dummy parking slots");
        }

        [Fact]
        public async Task SeedAsync_WhenCalledMultipleTimes_ShouldNotDuplicateData()
        {
            // -----
            // 1. Arrange
            // -----
            using var context = new ApplicationDbContext(_options);

            // -----
            // 2. Act (Call seeder twice)
            // -----
            await DatabaseSeeder.SeedAsync(context);
            await DatabaseSeeder.SeedAsync(context);

            // -----
            // 3. Assert
            // -----
        }
    }
}
```

```

        var userCount = await context.Users.CountAsync();
        var slotCount = await context.ParkingSlots.CountAsync();

        userCount.Should().Be(5, "because the seeder should be idempotent and not insert duplicates");
        slotCount.Should().Be(20, "because the seeder should be idempotent and not insert duplicates");
    }

    public void Dispose()
    {
        // Clean up the connection to destroy the in-memory database
        _connection.Close();
        _connection.Dispose();
    }
}
}
}

```

This completes the backend foundation tasks for Milestone 1. The tests explicitly cover your acceptance criteria, validating both the exact counts and adding a safety check to ensure calling the seeder multiple times does not duplicate the reference data.

---

*User prompt: There is still one task for M1. Task: Create a GitHub Actions workflow to automatically build the backend solution and execute all tests on every push and pull request. As a developer, I want automated compilation and test execution on GitHub, to achieve immediate feedback and ensure no breaking changes enter the repository. Acceptance Criteria: Create a .github/workflows/ci.yml pipeline file. Configure the pipeline to trigger on push and pull\_request events to main/master. Install the .NET 10 SDK environment in the runner. Execute dotnet build to confirm the backend compiles without errors. Execute dotnet test to run all unit and integration tests. Ensure the workflow fails and flags the commit if any build step or test fails.*

**Response:** Here is the implementation for your GitHub Actions CI workflow. This pipeline will automatically trigger on the specified branches, set up the .NET 10 environment, and execute your build and test commands, acting as a strict quality gate for your repository.

## GitHub Actions CI Pipeline

Create the necessary folder structure in the root of your repository (.github/workflows/) and add the following YAML file.

**.github/workflows/ci.yml**

```

name: Backend CI

on:
  push:
    branches: [ "main", "master" ]
  pull_request:
    branches: [ "main", "master" ]

jobs:
  build-and-test:
    name: Build and Test .NET 10
    runs-on: ubuntu-latest

    steps:
      - name: Checkout Repository
        uses: actions/checkout@v4

      - name: Setup .NET 10 SDK
        uses: actions/setup-dotnet@v4
        with:
          dotnet-version: '10.0.x'

      - name: Restore dependencies
        run: dotnet restore

      - name: Build the backend
        run: dotnet build --no-restore --configuration Release

      - name: Execute tests
        run: dotnet test --no-build --configuration Release --verbosity normal

```

## Workflow Details

- **Triggers:** The on block ensures the pipeline runs automatically whenever code is pushed to main or master, or when a pull request targeting those branches is opened or updated.
  - **Environment:** actions/setup-dotnet@v4 fetches and configures the exact .NET 10 SDK required for your backend.
  - **Failure Handling:** By default, GitHub Actions will fail the entire workflow run and flag the commit/PR with a red cross (✖) if any of the shell commands (dotnet build or dotnet test) return a non-zero exit code.
  - **Project Discovery:** The dotnet CLI commands are executed at the root of the repository. It will recursively find your .csproj files (both the WebAPI and the IntegrationTests projects) to restore, build, and test them.
-

*User prompt: On Restore dependencies step, I got the following error: Run dotnet restore dotnet restore shell: /usr/bin/bash -e {0} env: DOTNET\_ROOT: /usr/share/dotnet MSBUILD : error MSB1003: Specify a project or solution file. The current working directory does not contain a project or solution file. Error: Process completed with exit code 1. Please either pinpoint the issue if it is possible, or suggest at least 3 paths to solution.*

Response: I can pinpoint exactly what happened here. This is a very common snag when setting up CI pipelines for the first time!

The error MSBUILD : error MSB1003 means that the dotnet CLI is looking for a project (.csproj) or solution (.sln) file in the root of your repository, but it cannot find one. Because your backend code is nested under /src/ParkingBackend/Parking.WebAPI/ and your tests are under /tests/Parking.WebAPI.IntegrationTests/ (and there is no .sln file at the root), the command fails.

Here are 3 paths to solve this, depending on how you prefer to structure your project:

### Path 1: Create a Solution File (Recommended)

This is the standard .NET approach. By creating a .sln file at the root of your repository and linking your projects to it, the dotnet commands in your YAML will work automatically without modifications.

Run these commands in your local terminal at the root of your repository, commit, and push:

```
# Create an empty solution file at the root
dotnet new sln -n ParkingAdmin

# Add your WebAPI project to the solution
dotnet sln add src/ParkingBackend/Parking.WebAPI/Parking.WebAPI.csproj

# Add your Integration Tests project to the solution
dotnet sln add tests/Parking.WebAPI.IntegrationTests/Parking.WebAPI.IntegrationTests.csproj
```

### Path 2: Specify the Paths in the CI Workflow

If you prefer not to use a .sln file, you can explicitly tell the dotnet CLI where to find your projects by appending the paths directly to the commands in your .github/workflows/ci.yml.

Modify the steps in your YAML like this:

```
- name: Restore dependencies
  run: |
    dotnet restore src/ParkingBackend/Parking.WebAPI/Parking.WebAPI.csproj
    dotnet restore tests/Parking.WebAPI.IntegrationTests/Parking.WebAPI.IntegrationTests.csproj

- name: Build the backend
  run: dotnet build src/ParkingBackend/Parking.WebAPI/Parking.WebAPI.csproj --no-restore --configuration Release

- name: Execute tests
  run: dotnet test tests/Parking.WebAPI.IntegrationTests/Parking.WebAPI.IntegrationTests.csproj --no-build --con
```

### Path 3: Use the working-directory Attribute in GitHub Actions

You can instruct GitHub Actions to change the directory before executing the commands. This is cleaner than Path 2 if you have many commands, though you still have to handle the test project separately.

Modify your YAML to set the working directory for each step:

```
- name: Restore dependencies
  working-directory: ./src/ParkingBackend/Parking.WebAPI
  run: dotnet restore

- name: Build the backend
  working-directory: ./src/ParkingBackend/Parking.WebAPI
  run: dotnet build --no-restore --configuration Release

# Tests need to run in their own directory, and they will implicitly build/restore themselves if needed
- name: Execute tests
  working-directory: ./tests/Parking.WebAPI.IntegrationTests
  run: dotnet test --configuration Release --verbosity normal
```

**My recommendation:** Go with **Path 1**. Having a .sln file at the root makes the repository much easier to open in IDEs (like Visual Studio or Rider) and keeps your CI workflow file perfectly clean.

---

*User prompt: Next, let's head into planning the Milestone 2 of the project. Make sure the plans, issues don't have any holes in it, and every functionality, security, and logic measurements are working and well-designed!*

Response: Before we generate the specific GitHub issues for **M2-Backend Logic**, I need to pause and address a few potential "holes" in the business rules.

To ensure the system is completely foolproof, secure, and prevents double-bookings, we must define the exact boundaries of the reservation logic. If we don't lock these down now, we could introduce edge-case bugs.

Here are the design questions and my proposed solutions for each.

## 🔴 Potential Logic Holes & Proposals

### 1. Time Overlaps & Boundaries

- **The Hole:** What happens at the exact minute a reservation ends? If User A books from 10:00 to 11:00, can User B book from 11:00 to 12:00?
- **My Proposal:** Yes, boundaries should be inclusive-exclusive. 10:00 to 11:00 and 11:00 to 12:00 is **not** a double booking.

### 2. Time Rules & Granularity

- **The Hole:** Can a user book a slot for 1 minute? Can they book it in the past?
- **My Proposal:**
  - Require all timestamps to be strictly in UTC to avoid timezone bugs.
  - Enforce a rule that `StartTime` must be in the future.
  - Enforce a minimum reservation duration (e.g., 30 minutes or 1 hour).
  - (Optional) Force reservations to start on the hour or half-hour (e.g., 10:00, 10:30).

### 3. Cancellation Limitations

- **The Hole:** Can a user cancel a reservation that has already happened or is currently happening?
- **My Proposal:** Cancellations should only be allowed if the `StartTime` is still in the future.

### 4. The "Extra" Requirement (Restricted Slots)

- **The Hole:** The assignment mentions an optional extra requirement: handling different types of parking spots (e.g., restricted usage).
- **My Proposal:** We skipped this in M1, but if you want to implement it, we need to add a `Type` (e.g., Standard, VIP, EV) to the `ParkingSlot` model and enforce logic that prevents certain users from booking restricted slots.

How would you like to handle the time granularity/duration limits, and do you want to implement the optional restricted slots requirement before I draft the M2 issues?

---

*User prompt: 1. I accept your proposal. Boundaries should be inclusive-exclusive. 2. Timestamps should be UTC. StartTime must be in the future. Minimum reservation time should be 30 minutes. Do not force when the reservations can start. 3. Let's add the functionality for cancelling, only if the reservation has not been started. 4. Let's leave it for a later development.*

**Response:** Here are the detailed issues for **M2-Backend Logic**, incorporating all the agreed-upon business rules (UTC time, minimum duration, inclusive-exclusive overlap checking, and strict cancellation rules).

These are broken down into small, highly testable units, separating the core business logic (Services) from the HTTP routing (Controllers).

#6 [M2-Backend Logic] Define Data Transfer Objects (DTOs)

Task: Create the necessary DTOs in the ``/DTOs`` folder for receiving requests and returning data without exposing raw

As a backend developer, I want to use DTOs for incoming requests and outgoing responses, to decouple the API contrac

Acceptance Criteria:

- \* Create ``ReservationRequestDto`` containing ``ParkingSlotId``, ``UserId``, ``StartTime``, and ``EndTime``.
- \* Create ``ReservationResponseDto`` containing ``Id``, ``ParkingSlotId``, ``UserId``, ``StartTime``, and ``EndTime``.
- \* Create ``ParkingSlotResponseDto`` containing ``Id``, ``Designation``, and a list of ``ReservationResponseDto``.

#7 [M2-Backend Logic] Implement Reservation Creation Logic

Task: Create an ``IReservationService`` and its implementation ``ReservationService`` to handle the business rules for c

As a system, I want to validate and process reservation requests securely, to prevent double-bookings and enforce ti

Acceptance Criteria:

- \* Register the service in ``Program.cs`` with `Scoped` lifetime.
- \* Ensure all timestamps provided are treated as UTC.
- \* Reject the request if ``StartTime`` is in the past (compared to ``DateTime.UtcNow``).
- \* Reject the request if the duration between ``StartTime`` and ``EndTime`` is less than 30 minutes.
- \* Reject the request if an overlapping reservation exists for the same ``ParkingSlotId``. (Overlap formula: ``Existing``).
- \* If all rules pass, map the DTO to the ``Reservation`` model, save it to the database, and return a ``ReservationRespo``.

#8 [M2-Backend Logic] Test: Implement Reservation Creation Logic

Task: Write unit tests using `xUnit` and `NSubstitute` to verify the time and overlap validation rules in ``ReservationSe``

As a tester, I want to thoroughly test the business logic in isolation, to ensure no invalid or overlapping reservat

Acceptance Criteria:

- \* Test that a reservation starting in the past throws a specific validation exception.
- \* Test that a reservation shorter than 30 minutes throws a specific validation exception.
- \* Test that adjacent reservations (e.g., 10:00-11:00 and 11:00-12:00) do NOT trigger an overlap error.

- \* Test that partially overlapping reservations (e.g., 10:00-11:00 and 10:30-11:30) DO trigger an overlap error.
- \* Test that a fully valid reservation successfully calls the database context to save the entity.

#9 [M2-Backend Logic] Implement Reservation Cancellation Logic

Task: Add a cancellation method to the `IReservationService`` that enforces the future-only cancellation rule.

As a user, I want to cancel my reservation if my plans change, to free up the parking slot for others.

Acceptance Criteria:

- \* Add a `CancelReservationAsync(Guid reservationId)`` method.
- \* Retrieve the reservation from the database; if not found, return an appropriate error/exception.
- \* Check the `StartTime`` of the reservation against `DateTime.UtcNow``.
- \* If the `StartTime`` is in the past (meaning the reservation has started or finished), reject the cancellation.
- \* If valid, remove the reservation from the database and save changes.

#10 [M2-Backend Logic] Test: Implement Reservation Cancellation Logic

Task: Write unit tests to verify the cancellation rules in `ReservationService``.

As a tester, I want to verify that in-progress or completed reservations cannot be canceled, to maintain correct his

Acceptance Criteria:

- \* Test that cancelling a non-existent reservation throws a Not Found exception.
- \* Test that cancelling a reservation with a `StartTime`` in the past throws a validation exception.
- \* Test that cancelling a reservation with a `StartTime`` in the future successfully deletes the entity from the datab

#11 [M2-Backend Logic] Implement ParkingSlot Controller

Task: Create the API endpoints to list parking slots and their associated reservations[cite: 1].

As a frontend application, I want to fetch the list of parking slots and their current bookings, to display availabi

Acceptance Criteria:

- \* Create `ParkingSlotController`` with an `[ApiController]`` and route attribute.
- \* Add an endpoint `GET /api/parkingslots`` that returns all parking slots mapped to `ParkingSlotResponseDto``.
- \* Add an endpoint `GET /api/parkingslots/{id}`` that returns a specific parking slot and its nested reservations.
- \* Ensure EF Core uses `.AsNoTracking()``` for these read-only queries to optimize performance.

#12 [M2-Backend Logic] Test: Implement ParkingSlot Controller

Task: Write integration tests using `WebApplicationFactory`` for the `ParkingSlotController`` endpoints.

As a tester, I want to ensure the API correctly serializes and returns database data over HTTP, to guarantee fronten

Acceptance Criteria:

- \* Test `GET /api/parkingslots`` returns a 200 OK status code and a JSON array of slots.
- \* Test `GET /api/parkingslots/{id}`` for a valid ID returns a 200 OK and the correct slot data.
- \* Test `GET /api/parkingslots/{id}`` for an invalid ID returns a 404 Not Found.

#13 [M2-Backend Logic] Implement Reservation Controller

Task: Create the API endpoints to create and cancel reservations using the `ReservationService``.

As a frontend application, I want to submit booking and cancellation requests via HTTP POST and DELETE methods, to i

Acceptance Criteria:

- \* Create `ReservationController`` with an `[ApiController]`` and route attribute.
- \* Add an endpoint `POST /api/reservations`` that accepts a `ReservationRequestDto`` and passes it to the service. Retu
- \* Add an endpoint `DELETE /api/reservations/{id}`` that passes the ID to the cancellation service method and returns
- \* Catch domain exceptions thrown by the service and return 400 Bad Request with the error message.

#14 [M2-Backend Logic] Test: Implement Reservation Controller

Task: Write integration tests using `WebApplicationFactory`` to verify HTTP behavior for reservations.

As a tester, I want to simulate HTTP requests for bookings, to ensure the API handles success and validation failure

Acceptance Criteria:

- \* Test that a valid `POST /api/reservations`` request returns 201 Created.
- \* Test that an invalid `POST /api/reservations`` (e.g., overlapping time) returns 400 Bad Request.
- \* Test that a valid `DELETE /api/reservations/{id}`` request returns 204 No Content.

---

User prompt: Task: Create the necessary DTOs in the /DTOs folder for receiving requests and returning data without exposing raw database entities. As a backend developer, I want to use DTOs for incoming requests and outgoing responses, to decouple the API contract from the internal database schema. Acceptance Criteria: Create `ReservationRequestDto` containing `ParkingSlotId`, `UserId`, `StartTime`, and `EndTime`. Create `ReservationResponseDto` containing `Id`, `ParkingSlotId`, `UserId`, `StartTime`, and `EndTime`. Create `ParkingSlotResponseDto` containing `Id`, `Designation`, and a list of `ReservationResponseDto`.

Response: Here is the implementation for your Data Transfer Objects (DTOs). These classes ensure that we decouple our database models from the API contract, providing a secure and clean way to exchange data.

Please create these files inside the `/src/ParkingBackend/Parking.WebAPI/DTOs/` folder.

## 1. Reservation Request DTO

This DTO is used to receive the incoming payload when a user wants to book a parking slot.

#### ReservationRequestDto.cs

```
using System;
using System.ComponentModel.DataAnnotations;

namespace Parking.WebAPI.DTOS
{
    public class ReservationRequestDto
    {
        [Required]
        public Guid ParkingSlotId { get; set; }

        [Required]
        public Guid UserId { get; set; }

        [Required]
        public DateTime StartTime { get; set; }

        [Required]
        public DateTime EndTime { get; set; }
    }
}
```

## 2. Reservation Response DTO

This DTO is used to return reservation details to the frontend, ensuring we don't expose internal database fields or navigation properties unnecessarily.

#### ReservationResponseDto.cs

```
using System;

namespace Parking.WebAPI.DTOS
{
    public class ReservationResponseDto
    {
        public Guid Id { get; set; }
        public Guid ParkingSlotId { get; set; }
        public Guid UserId { get; set; }
        public DateTime StartTime { get; set; }
        public DateTime EndTime { get; set; }
    }
}
```

## 3. Parking Slot Response DTO

This DTO is used to list parking slots along with their current bookings. It includes a nested list of ReservationResponseDto to provide the frontend with all the necessary data to render an availability calendar or list.

#### ParkingSlotResponseDto.cs

```
using System;
using System.Collections.Generic;

namespace Parking.WebAPI.DTOS
{
    public class ParkingSlotResponseDto
    {
        public Guid Id { get; set; }
        public string Designation { get; set; } = string.Empty;

        public IEnumerable<ReservationResponseDto> Reservations { get; set; } = new List<ReservationResponseDto>();
    }
}
```

---

*User prompt: Task: Create an IReservationService and its implementation ReservationService to handle the business rules for creating a reservation. As a system, I want to validate and process reservation requests securely, to prevent double-bookings and enforce time rules. Acceptance Criteria: Register the service in Program.cs with Scoped lifetime. Ensure all timestamps provided are treated as UTC. Reject the request if StartTime is in the past (compared to DateTime.UtcNow). Reject the request if the duration between StartTime and EndTime is less than 30 minutes. Reject the request if an overlapping reservation exists for the same ParkingSlotId. (Overlap formula: Existing.StartTime < New.EndTime AND Existing.EndTime > New.StartTime). If all rules pass, map the DTO to the Reservation model, save it to the database, and return a ReservationResponseDto.*

Response: Here is the implementation of the reservation business logic. I have created an interface and its concrete implementation, ensuring all validation rules—including the overlap check—are executed against the database efficiently before saving.

## 1. The Service Interface

Create a new file named `IReservationService.cs` inside your `/src/ParkingBackend/Parking.WebAPI/Services/` folder.

#### **IReservationService.cs**

```
using System.Threading.Tasks;
using Parking.WebAPI.DTOS;

namespace Parking.WebAPI.Services
{
    public interface IReservationService
    {
        Task<ReservationResponseDto> CreateReservationAsync(ReservationRequestDto request);
    }
}
```

## **2. The Service Implementation**

Create another file named `ReservationService.cs` in the same `/Services/` folder.

#### **ReservationService.cs**

```
using System;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOS;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.Services
{
    public class ReservationService : IReservationService
    {
        private readonly ApplicationDbContext _context;

        public ReservationService(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<ReservationResponseDto> CreateReservationAsync(ReservationRequestDto request)
        {
            // 1. Ensure timestamps are treated as UTC
            var startTimeUtc = request.StartTime.ToUniversalTime();
            var endTimeUtc = request.EndTime.ToUniversalTime();

            // 2. Reject if StartTime is in the past
            if (startTimeUtc < DateTime.UtcNow)
            {
                throw new ArgumentException("Reservation start time cannot be in the past.");
            }

            // 3. Reject if duration is less than 30 minutes
            var duration = endTimeUtc - startTimeUtc;
            if (duration.TotalMinutes < 30)
            {
                throw new ArgumentException("Reservation duration must be at least 30 minutes.");
            }

            // 4. Reject if an overlapping reservation exists for the same ParkingSlotId
            bool hasOverlap = await _context.Reservations
                .AnyAsync(r => r.ParkingSlotId == request.ParkingSlotId &&
                    r.StartTime < endTimeUtc &&
                    r.EndTime > startTimeUtc);

            if (hasOverlap)
            {
                throw new InvalidOperationException("The parking slot is already booked for the requested time period.");
            }

            // Ensure the User and ParkingSlot actually exist
            bool slotExists = await _context.ParkingSlots.AnyAsync(p => p.Id == request.ParkingSlotId);
            if (!slotExists) throw new ArgumentException("Invalid ParkingSlotId.");

            bool userExists = await _context.Users.AnyAsync(u => u.Id == request.UserId);
            if (!userExists) throw new ArgumentException("Invalid UserId.");

            // 5. Map DTO to Model
            var reservation = new Reservation
            {
                Id = Guid.NewGuid(),
            }
        }
    }
}
```

```

        ParkingSlotId = request.ParkingSlotId,
        UserId = request.UserId,
        StartTime = startTimeUtc,
        EndTime = endTimeUtc
    };

    // 6. Save to Database
    _context.Reservations.Add(reservation);
    await _context.SaveChangesAsync();

    // 7. Return Response DTO
    return new ReservationResponseDto
    {
        Id = reservation.Id,
        ParkingSlotId = reservation.ParkingSlotId,
        UserId = reservation.UserId,
        StartTime = reservation.StartTime,
        EndTime = reservation.EndTime
    };
}
}
}

```

### 3. Register the Service in Program.cs

Update your Program.cs to register the IReservationService with a Scoped lifetime so it can properly interact with the scoped ApplicationDbContext.

**Add this line to Program.cs right after your DbContext registration:**

```

// Configure SQLite DbContext
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));

// Register Business Services
builder.Services.AddScoped<IReservationService, ReservationService>();

```

*User prompt: Task: Write unit tests using xUnit and NSubstitute to verify the time and overlap validation rules in ReservationService. As a tester, I want to thoroughly test the business logic in isolation, to ensure no invalid or overlapping reservations can ever be created. Acceptance Criteria: Test that a reservation starting in the past throws a specific validation exception. Test that a reservation shorter than 30 minutes throws a specific validation exception. Test that adjacent reservations (e.g., 10:00-11:00 and 11:00-12:00) do NOT trigger an overlap error. Test that partially overlapping reservations (e.g., 10:00-11:00 and 10:30-11:30) DO trigger an overlap error. Test that a fully valid reservation successfully calls the database context to save the entity.*

Response: Here is the implementation of your unit tests for the ReservationService.

Because mocking Entity Framework Core's asynchronous extension methods (like AnyAsync) using NSubstitute is notoriously brittle and often considered an anti-pattern, the standard and most robust approach is to use the SQLite in-memory provider. This ensures your overlap queries are executed by a real query parser, validating your exact business logic.

Create a file named ReservationServiceTests.cs inside your Parking.WebAPI.IntegrationTests project.

#### ReservationServiceTests.cs

```

using System;
using System.Threading.Tasks;
using FluentAssertions;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOS;
using Parking.WebAPI.Models;
using Parking.WebAPI.Services;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class ReservationServiceTests : IDisposable
    {
        private readonly SqliteConnection _connection;
        private readonly DbContextOptions<ApplicationDbContext> _options;
        private readonly ApplicationDbContext _context;
        private readonly ReservationService _service;

        public ReservationServiceTests()
        {
            _connection = new SqliteConnection("DataSource=:memory:");
            _connection.Open();

```



```

        _options = new DbContextOptionsBuilder<ApplicationDbContext>()
            .UseSqlite(_connection)
            .Options;

        _context = new ApplicationDbContext(_options);
        _context.Database.Migrate(); // Ensure schema is created

        _service = new ReservationService(_context);
    }

    [Fact]
    public async Task CreateReservation_WhenStartTimeIsInThePast_ShouldThrowArgumentException()
    {
        // Arrange
        var request = new ReservationRequestDto
        {
            ParkingSlotId = Guid.NewGuid(),
            UserId = Guid.NewGuid(),
            StartTime = DateTime.UtcNow.AddHours(-1), // Past
            EndTime = DateTime.UtcNow.AddHours(1)
        };

        // Act
        Func<Task> act = async () => await _service.CreateReservationAsync(request);

        // Assert
        await act.Should().ThrowAsync<ArgumentException>()
            .WithMessage("Reservation start time cannot be in the past.");
    }

    [Fact]
    public async Task CreateReservation_WhenDurationIsLessThan30Minutes_ShouldThrowArgumentException()
    {
        // Arrange
        var startTime = DateTime.UtcNow.AddHours(1);
        var request = new ReservationRequestDto
        {
            ParkingSlotId = Guid.NewGuid(),
            UserId = Guid.NewGuid(),
            StartTime = startTime,
            EndTime = startTime.AddMinutes(29) // Less than 30 mins
        };

        // Act
        Func<Task> act = async () => await _service.CreateReservationAsync(request);

        // Assert
        await act.Should().ThrowAsync<ArgumentException>()
            .WithMessage("Reservation duration must be at least 30 minutes.");
    }

    [Fact]
    public async Task CreateReservation_WithAdjacentReservations_ShouldNotTriggerOverlap()
    {
        // Arrange
        var userId = Guid.NewGuid();
        var slotId = Guid.NewGuid();
        await SeedUserAndSlotAsync(userId, slotId);

        var baseTime = DateTime.UtcNow.AddDays(1);

        // Existing reservation: 10:00 to 11:00
        _context.Reservations.Add(new Reservation
        {
            Id = Guid.NewGuid(),
            UserId = userId,
            ParkingSlotId = slotId,
            StartTime = baseTime,
            EndTime = baseTime.AddHours(1)
        });
        await _context.SaveChangesAsync();

        var request = new ReservationRequestDto
        {
            ParkingSlotId = slotId,
            UserId = userId,
            StartTime = baseTime.AddHours(1), // Starts exactly when the other ends (11:00)
            EndTime = baseTime.AddHours(2) // Ends at 12:00
        };
    }

```

```

// Act
var result = await _service.CreateReservationAsync(request);

// Assert
result.Should().NotBeNull();
result.StartTime.Should().Be(request.StartTime);

var count = await _context.Reservations.CountAsync();
count.Should().Be(2, "because the adjacent reservation should be saved alongside the original");
}

[Fact]
public async Task CreateReservation_WithPartiallyOverlappingReservation_ShouldThrowInvalidOperationException
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    await SeedUserAndSlotAsync(userId, slotId);

    var baseTime = DateTime.UtcNow.AddDays(1);

    // Existing reservation: 10:00 to 11:00
    _context.Reservations.Add(new Reservation
    {
        Id = Guid.NewGuid(),
        UserId = userId,
        ParkingSlotId = slotId,
        StartTime = baseTime,
        EndTime = baseTime.AddHours(1)
    });
    await _context.SaveChangesAsync();

    var request = new ReservationRequestDto
    {
        ParkingSlotId = slotId,
        UserId = userId,
        StartTime = baseTime.AddMinutes(30), // Starts at 10:30 (Overlap)
        EndTime = baseTime.AddMinutes(90)    // Ends at 11:30
    };

    // Act
    Func<Task> act = async () => await _service.CreateReservationAsync(request);

    // Assert
    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage("The parking slot is already booked for the requested time period.");
}

[Fact]
public async Task CreateReservation_WithValidData_ShouldSaveToDatabaseAndReturnDto()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    await SeedUserAndSlotAsync(userId, slotId);

    var request = new ReservationRequestDto
    {
        ParkingSlotId = slotId,
        UserId = userId,
        StartTime = DateTime.UtcNow.AddHours(2),
        EndTime = DateTime.UtcNow.AddHours(4)
    };

    // Act
    var response = await _service.CreateReservationAsync(request);

    // Assert
    response.Should().NotBeNull();
    response.ParkingSlotId.Should().Be(slotId);
    response.UserId.Should().Be(userId);

    var dbEntity = await _context.Reservations.FirstOrDefaultAsync(r => r.Id == response.Id);
    dbEntity.Should().NotBeNull();
    dbEntity!.StartTime.Should().Be(request.StartTime);
    dbEntity.EndTime.Should().Be(request.EndTime);
}

```

```

private async Task SeedUserAndSlotAsync(Guid userId, Guid slotId)
{
    _context.Users.Add(new User { Id = userId, Name = "Test User" });
    _context.ParkingSlots.Add(new ParkingSlot { Id = slotId, Designation = "T01" });
    await _context.SaveChangesAsync();
}

public void Dispose()
{
    _context.Dispose();
    _connection.Close();
    _connection.Dispose();
}
}
}

```

---

*User prompt: Task: Add a cancellation method to the IReservationService that enforces the future-only cancellation rule. As a user, I want to cancel my reservation if my plans change, to free up the parking slot for others. Acceptance Criteria: Add a CancelReservationAsync(Guid reservationId) method. Retrieve the reservation from the database; if not found, return an appropriate error/exception. Check the StartTime of the reservation against DateTime.UtcNow. If the StartTime is in the past (meaning the reservation has started or finished), reject the cancellation. If valid, remove the reservation from the database and save changes.*

Response: Here is the implementation to add the cancellation logic to your service. This ensures that users can only cancel their bookings if the reservation hasn't already started, keeping your historical data intact.

## 1. Update the Service Interface

Add the CancelReservationAsync method signature to your existing IReservationService.

### IReservationService.cs

```

using System;
using System.Threading.Tasks;
using Parking.WebAPI.DTOs;

namespace Parking.WebAPI.Services
{
    public interface IReservationService
    {
        Task<ReservationResponseDto> CreateReservationAsync(ReservationRequestDto request);
        Task CancelReservationAsync(Guid reservationId);
    }
}

```

## 2. Update the Service Implementation

Add the implementation of the method to ReservationService. It checks for the existence of the reservation and enforces the time-based cancellation rule.

### ReservationService.cs (Additions)

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOs;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.Services
{
    public class ReservationService : IReservationService
    {
        private readonly ApplicationDbContext _context;

        public ReservationService(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<ReservationResponseDto> CreateReservationAsync(ReservationRequestDto request)
        {
            // ... (Existing implementation remains unchanged)
            // 1. Ensure timestamps are treated as UTC
            var startTimeUtc = request.StartTime.ToUniversalTime();
            var endTimeUtc = request.EndTime.ToUniversalTime();

            if (startTimeUtc < DateTime.UtcNow)

```

```

        throw new ArgumentException("Reservation start time cannot be in the past.");

var duration = endTimeUtc - startTimeUtc;
if (duration.TotalMinutes < 30)
    throw new ArgumentException("Reservation duration must be at least 30 minutes.");

bool hasOverlap = await _context.Reservations
    .AnyAsync(r => r.ParkingSlotId == request.ParkingSlotId &&
        r.StartTime < endTimeUtc &&
        r.EndTime > startTimeUtc);

if (hasOverlap)
    throw new InvalidOperationException("The parking slot is already booked for the requested time perio

bool slotExists = await _context.ParkingSlots.AnyAsync(p => p.Id == request.ParkingSlotId);
if (!slotExists) throw new ArgumentException("Invalid ParkingSlotId.");

bool userExists = await _context.Users.AnyAsync(u => u.Id == request.UserId);
if (!userExists) throw new ArgumentException("Invalid UserId.");

var reservation = new Reservation
{
    Id = Guid.NewGuid(),
    ParkingSlotId = request.ParkingSlotId,
    UserId = request.UserId,
    StartTime = startTimeUtc,
    EndTime = endTimeUtc
};

_context.Reservations.Add(reservation);
await _context.SaveChangesAsync();

return new ReservationResponseDto
{
    Id = reservation.Id,
    ParkingSlotId = reservation.ParkingSlotId,
    UserId = reservation.UserId,
    StartTime = reservation.StartTime,
    EndTime = reservation.EndTime
};
}

// Add the new cancellation method here:
public async Task CancelReservationAsync(Guid reservationId)
{
    // 1. Retrieve the reservation from the database
    var reservation = await _context.Reservations.FindAsync(reservationId);

    if (reservation == null)
    {
        throw new KeyNotFoundException($"Reservation with ID {reservationId} was not found.");
    }

    // 2. Check the StartTime against DateTime.UtcNow
    if (reservation.StartTime <= DateTime.UtcNow)
    {
        throw new InvalidOperationException("Cannot cancel a reservation that has already started or finishe
    }

    // 3. Remove from database and save
    _context.Reservations.Remove(reservation);
    await _context.SaveChangesAsync();
}
}
}

```

*User prompt: Task: Write unit tests to verify the cancellation rules in ReservationService. As a tester, I want to verify that in-progress or completed reservations cannot be canceled, to maintain correct historical records. Acceptance Criteria: Test that cancelling a non-existent reservation throws a Not Found exception. Test that cancelling a reservation with a StartTime in the past throws a validation exception. Test that cancelling a reservation with a StartTime in the future successfully deletes the entity from the database.*

**Response:** Here are the unit tests to verify your cancellation logic. You can append these directly to your existing ReservationServiceTests.cs file inside the Parking.WebAPI.IntegrationTests project.

#### ReservationServiceTests.cs (Additions)

```

[Fact]
public async Task CancelReservation_WhenReservationDoesNotExist_ShouldThrowKeyNotFoundException()

```

```

{
    // Arrange
    var nonExistentReservationId = Guid.NewGuid();

    // Act
    Func<Task> act = async () => await _service.CancelReservationAsync(nonExistentReservationId);

    // Assert
    await act.Should().ThrowAsync<KeyNotFoundException>()
        .WithMessage($"Reservation with ID {nonExistentReservationId} was not found.");
}

[Fact]
public async Task CancelReservation_WhenStartTimeIsInThePast_ShouldThrowInvalidOperationException()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    var reservationId = Guid.NewGuid();

    await SeedUserAndSlotAsync(userId, slotId);

    // Add a reservation that has already happened
    _context.Reservations.Add(new Reservation
    {
        Id = reservationId,
        UserId = userId,
        ParkingSlotId = slotId,
        StartTime = DateTime.UtcNow.AddHours(-2), // Started 2 hours ago
        EndTime = DateTime.UtcNow.AddHours(-1)
    });
    await _context.SaveChangesAsync();

    // Act
    Func<Task> act = async () => await _service.CancelReservationAsync(reservationId);

    // Assert
    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage("Cannot cancel a reservation that has already started or finished.");
}

[Fact]
public async Task CancelReservation_WhenStartTimeIsInTheFuture_ShouldDeleteFromDatabase()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    var reservationId = Guid.NewGuid();

    await SeedUserAndSlotAsync(userId, slotId);

    // Add a reservation in the future
    _context.Reservations.Add(new Reservation
    {
        Id = reservationId,
        UserId = userId,
        ParkingSlotId = slotId,
        StartTime = DateTime.UtcNow.AddHours(2), // Starts in 2 hours
        EndTime = DateTime.UtcNow.AddHours(3)
    });
    await _context.SaveChangesAsync();

    // Act
    await _service.CancelReservationAsync(reservationId);

    // Assert
    var deletedReservation = await _context.Reservations.FindAsync(reservationId);
    deletedReservation.Should().BeNull("because the valid future reservation should have been deleted from t
}

```

---

*User prompt: Task: Create the API endpoints to list parking slots and their associated reservations[cite: 1]. As a frontend application, I want to fetch the list of parking slots and their current bookings, to display availability to the user[cite: 1]. Acceptance Criteria: Create ParkingSlotController with an [ApiController] and route attribute. Add an endpoint GET /api/parkingslots that returns all parking slots mapped to ParkingSlotResponseDto. Add an endpoint GET /api/parkingslots/{id} that returns a specific parking slot and its nested reservations. Ensure EF Core uses .AsNoTracking() for these read-only queries to optimize performance.*

**Response:** Here is the implementation for the ParkingSlotController. This controller interacts directly with the ApplicationDbContext for read-only operations, utilizing .AsNoTracking() to ensure Entity Framework Core does not waste resources tracking these entities in memory,

which significantly optimizes read performance.

Create a new file named `ParkingSlotController.cs` inside your `/src/ParkingBackend/Parking.WebAPI/Controllers/` folder.

#### **ParkingSlotController.cs**

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOs;

namespace Parking.WebAPI.Controllers
{
    [ApiController]
    [Route("api/parkingslots")]
    public class ParkingSlotController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public ParkingSlotController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/parkingslots
        [HttpGet]
        public async Task<ActionResult<IEnumerable<ParkingSlotResponseDto>>> GetAllParkingSlots()
        {
            var slots = await _context.ParkingSlots
                .AsNoTracking()
                .Include(p => p.Reservations)
                .Select(p => new ParkingSlotResponseDto
                {
                    Id = p.Id,
                    Designation = p.Designation,
                    Reservations = p.Reservations.Select(r => new ReservationResponseDto
                    {
                        Id = r.Id,
                        ParkingSlotId = r.ParkingSlotId,
                        UserId = r.UserId,
                        StartTime = r.StartTime,
                        EndTime = r.EndTime
                    }).ToList()
                })
                .ToListAsync();

            return Ok(slots);
        }

        // GET: api/parkingslots/{id}
        [HttpGet("{id}")]
        public async Task<ActionResult<ParkingSlotResponseDto>> GetParkingSlotById(Guid id)
        {
            var slot = await _context.ParkingSlots
                .AsNoTracking()
                .Include(p => p.Reservations)
                .Where(p => p.Id == id)
                .Select(p => new ParkingSlotResponseDto
                {
                    Id = p.Id,
                    Designation = p.Designation,
                    Reservations = p.Reservations.Select(r => new ReservationResponseDto
                    {
                        Id = r.Id,
                        ParkingSlotId = r.ParkingSlotId,
                        UserId = r.UserId,
                        StartTime = r.StartTime,
                        EndTime = r.EndTime
                    }).ToList()
                })
                .FirstOrDefaultAsync();

            if (slot == null)
            {
                return NotFound(new { message = $"Parking slot with ID {id} was not found." });
            }
        }
    }
}
```

```

        return Ok(slot);
    }
}
}

```

## Key Implementation Details

- **Routing:** The `[Route("api/parkingslots")]` attribute explicitly sets the base path to match your acceptance criteria.
- **Performance:** `.AsNoTracking()` is chained before `.Include()` to bypass the EF Core change tracker for these purely informational, read-only GET requests.
- **Manual Mapping:** The `.Select()` LINQ projections map the database entities directly to your `ParkingSlotResponseDto` and `ReservationResponseDto` formats so no sensitive internal data leaks out to the frontend.
- **Error Handling:** The single-item fetch endpoint safely returns a 404 Not Found if the requested ID does not exist in the database.

*User prompt: Task: Write integration tests using WebApplicationFactory for the ParkingSlotController endpoints. As a tester, I want to ensure the API correctly serializes and returns database data over HTTP, to guarantee frontend compatibility. Acceptance Criteria: Test GET /api/parkingslots returns a 200 OK status code and a JSON array of slots. Test GET /api/parkingslots/{id} for a valid ID returns a 200 OK and the correct slot data. Test GET /api/parkingslots/{id} for an invalid ID returns a 404 Not Found.*

Response: Here is the implementation of the integration tests for your `ParkingSlotController` using `WebApplicationFactory`. This approach spins up an in-memory `TestServer` running your actual application pipeline (including the database seeder), allowing you to test real HTTP requests and responses.

## 1. Test Project Setup

To use `WebApplicationFactory`, you need to add the testing package to your integration tests project. Run this in your terminal:

```
dotnet add tests/Parking.WebAPI.IntegrationTests/Parking.WebAPI.IntegrationTests.csproj package Microsoft.AspNetCore
```

Next, to allow the test project to access your backend's entry point, append this single line to the very bottom of your `/src/ParkingBackend/Parking.WebAPI/Program.cs` file:

```
public partial class Program { }
```

## 2. Integration Test Implementation

Create a file named `ParkingSlotControllerTests.cs` inside your `Parking.WebAPI.IntegrationTests` project.

### ParkingSlotControllerTests.cs

```

using System;
using System.Collections.Generic;
using System.Net;
using System.Net.Http.Json;
using System.Threading.Tasks;
using FluentAssertions;
using Microsoft.AspNetCore.Mvc.Testing;
using Parking.WebAPI.DTOS;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class ParkingSlotControllerTests : IClassFixture<WebApplicationFactory<Program>>
    {
        private readonly WebApplicationFactory<Program> _factory;

        public ParkingSlotControllerTests(WebApplicationFactory<Program> factory)
        {
            _factory = factory;
        }

        [Fact]
        public async Task GetAllParkingSlots_ReturnsOkAndJsonArray()
        {
            // Arrange
            var client = _factory.CreateClient();

            // Act
            var response = await client.GetAsync("/api/parkingslots");

            // Assert
            response.StatusCode.Should().Be(HttpStatusCode.OK);

            var slots = await response.Content.ReadFromJsonAsync<List<ParkingSlotResponseDto>>();
            slots.Should().NotNull();
        }
    }
}

```

```

        // Because our DatabaseSeeder runs automatically on startup,
        // we expect the 20 seeded dummy slots to be returned.
        slots.Should().HaveCount(20);
    }

    [Fact]
    public async Task GetParkingSlotById_ValidId_ReturnsOkAndSlotData()
    {
        // Arrange
        var client = _factory.CreateClient();

        // Fetch all slots first to dynamically grab a valid, seeded ID
        var allSlots = await client.GetFromJsonAsync<List<ParkingSlotResponseDto>>("/api/parkingslots");
        var validSlot = allSlots![0];

        // Act
        var response = await client.GetAsync($"/api/parkingslots/{validSlot.Id}");

        // Assert
        response.StatusCode.Should().Be(HttpStatusCode.OK);

        var slot = await response.Content.ReadFromJsonAsync<ParkingSlotResponseDto>();
        slot.Should().NotNull();
        slot!.Id.Should().Be(validSlot.Id);
        slot.Designation.Should().Be(validSlot.Designation);
    }

    [Fact]
    public async Task GetParkingSlotById_InvalidId_ReturnsNotFound()
    {
        // Arrange
        var client = _factory.CreateClient();
        var invalidId = Guid.NewGuid();

        // Act
        var response = await client.GetAsync($"/api/parkingslots/{invalidId}");

        // Assert
        response.StatusCode.Should().Be(HttpStatusCode.NotFound);
    }
}
}

```

---

*User prompt: Task: Create the API endpoints to create and cancel reservations using the ReservationService. As a frontend application, I want to submit booking and cancellation requests via HTTP POST and DELETE methods, to interact with the backend logic. Acceptance Criteria: Create ReservationController with an [ApiController] and route attribute. Add an endpoint POST /api/reservations that accepts a ReservationRequestDto and passes it to the service. Return a 201 Created status with the result. Add an endpoint DELETE /api/reservations/{id} that passes the ID to the cancellation service method and returns a 204 No Content upon success. Catch domain exceptions thrown by the service and return 400 Bad Request with the error message.*

Response: Here is the implementation for the ReservationController. This controller acts as the HTTP interface for your application, delegating the heavy lifting and business rule validation to the IReservationService.

Create a new file named ReservationController.cs inside your /src/ParkingBackend/Parking.WebAPI/Controllers/ folder.

#### ReservationController.cs

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Parking.WebAPI.DTOS;
using Parking.WebAPI.Services;

namespace Parking.WebAPI.Controllers
{
    [ApiController]
    [Route("api/reservations")]
    public class ReservationController : ControllerBase
    {
        private readonly IReservationService _reservationService;

        public ReservationController(IReservationService reservationService)
        {
            _reservationService = reservationService;
        }

        // POST: api/reservations

```



```

[HttpPost]
public async Task<ActionResult<ReservationResponseDto>> CreateReservation([FromBody] ReservationRequestDto r
{
    try
    {
        var response = await _reservationService.CreateReservationAsync(request);

        // Return 201 Created.
        // We use nameof(CreateReservation) just as a placeholder for the Location header URI,
        // but typically this points to a GET endpoint for the specific resource.
        return Created(string.Empty, response);
    }
    catch (ArgumentException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        return BadRequest(new { message = ex.Message });
    }
}

// DELETE: api/reservations/{id}
[HttpDelete("{id}")]
public async Task<IAActionResult> CancelReservation(Guid id)
{
    try
    {
        await _reservationService.CancelReservationAsync(id);
        return NoContent(); // Returns 204
    }
    catch (KeyNotFoundException ex)
    {
        return NotFound(new { message = ex.Message });
    }
    catch (InvalidOperationException ex)
    {
        // Returns 400 Bad Request if the reservation cannot be canceled due to business rules (e.g., already
        return BadRequest(new { message = ex.Message });
    }
}
}
}
}
}

```

## Key Implementation Details

- **Routing:** The `[Route("api/reservations")]` attribute aligns with your RESTful design.
- **POST Endpoint:** The `CreateReservation` method passes the DTO to the service and returns a 201 Created status with the newly created reservation data.
- **DELETE Endpoint:** The `CancelReservation` method calls the cancellation service and returns a 204 No Content upon success, indicating the action was successful but there is no data to return.
- **Error Handling:** We are catching specific domain exceptions (`ArgumentException`, `InvalidOperationException`) and returning a 400 Bad Request with the specific error message generated by the service. I also added a 404 NotFound specifically for `KeyNotFoundException` to keep the API REST-compliant if someone tries to delete a non-existent ID.

---

*User prompt: Task: Write integration tests using WebApplicationFactory to verify HTTP behavior for reservations. As a tester, I want to simulate HTTP requests for bookings, to ensure the API handles success and validation failure scenarios correctly. Acceptance Criteria: Test that a valid POST /api/reservations request returns 201 Created. Test that an invalid POST /api/reservations (e.g., overlapping time) returns 400 Bad Request. Test that a valid DELETE /api/reservations/{id} request returns 204 No Content.*

Response: Here is the implementation of the integration tests for your ReservationController.

Because we do not currently have an endpoint to list users, these tests will access the `ApplicationDbContext` directly through the `WebApplicationFactory`'s service provider to fetch a valid, seeded `UserId` and `ParkingSlotId` for the API calls. I also used offset future dates (e.g., adding days) to ensure tests don't cause accidental overlaps if executed in parallel.

Create a new file named `ReservationControllerTests.cs` inside your `Parking.WebAPI.IntegrationTests` project.

### ReservationControllerTests.cs

```

using System;
using System.Linq;
using System.Net;
using System.Net.Http.Json;
using System.Threading.Tasks;
using FluentAssertions;

```

```

using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.Extensions.DependencyInjection;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOs;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class ReservationControllerTests : IClassFixture<WebApplicationFactory<Program>>
    {
        private readonly WebApplicationFactory<Program> _factory;

        public ReservationControllerTests(WebApplicationFactory<Program> factory)
        {
            _factory = factory;
        }

        // Helper method to retrieve valid seeded IDs directly from the database context
        private (Guid UserId, Guid SlotId) GetValidIds()
        {
            using var scope = _factory.Services.CreateScope();
            var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();

            var userId = dbContext.Users.First().Id;
            var slotId = dbContext.ParkingSlots.First().Id;

            return (userId, slotId);
        }

        [Fact]
        public async Task PostReservation_WithValidData_ReturnsCreated()
        {
            // Arrange
            var client = _factory.CreateClient();
            var (userId, slotId) = GetValidIds();

            var request = new ReservationRequestDto
            {
                ParkingSlotId = slotId,
                UserId = userId,
                // Use a far future date to avoid overlapping with other parallel tests
                StartTime = DateTime.UtcNow.AddDays(10),
                EndTime = DateTime.UtcNow.AddDays(10).AddHours(2)
            };

            // Act
            var response = await client.PostAsJsonAsync("/api/reservations", request);

            // Assert
            response.StatusCode.Should().Be(HttpStatusCode.Created);

            var createdReservation = await response.Content.ReadFromJsonAsync<ReservationResponseDto>();
            createdReservation.Should().NotBeNull();
            createdReservation!.ParkingSlotId.Should().Be(slotId);
            createdReservation.UserId.Should().Be(userId);
        }

        [Fact]
        public async Task PostReservation_WithOverlappingTime_ReturnsBadRequest()
        {
            // Arrange
            var client = _factory.CreateClient();
            var (userId, slotId) = GetValidIds();

            var request = new ReservationRequestDto
            {
                ParkingSlotId = slotId,
                UserId = userId,
                StartTime = DateTime.UtcNow.AddDays(11),
                EndTime = DateTime.UtcNow.AddDays(11).AddHours(2)
            };

            // Act
            // 1. Create the initial valid reservation
            await client.PostAsJsonAsync("/api/reservations", request);

            // 2. Try to create a second reservation for the exact same time and slot
            var overlappingResponse = await client.PostAsJsonAsync("/api/reservations", request);

```

```

        // Assert
        overlappingResponse.StatusCode.Should().Be(HttpStatusCode.BadRequest);

        var errorContent = await overlappingResponse.Content.ReadAsStringAsync();
        errorContent.Should().Contain("booked");
    }

    [Fact]
    public async Task DeleteReservation_WithValidFutureId_ReturnsNoContent()
    {
        // Arrange
        var client = _factory.CreateClient();
        var (userId, slotId) = GetValidIds();

        var request = new ReservationRequestDto
        {
            ParkingSlotId = slotId,
            UserId = userId,
            StartTime = DateTime.UtcNow.AddDays(12),
            EndTime = DateTime.UtcNow.AddDays(12).AddHours(2)
        };

        // First, create a reservation to ensure we have a valid ID to delete
        var createResponse = await client.PostAsJsonAsync("/api/reservations", request);
        var createdReservation = await createResponse.Content.ReadFromJsonAsync<ReservationResponseDto>();

        // Act
        var deleteResponse = await client.DeleteAsync($"/api/reservations/{createdReservation!.Id}");

        // Assert
        deleteResponse.StatusCode.Should().Be(HttpStatusCode.NoContent);
    }
}
}
}

```

---

*User prompt: Next, let's head into planning the Milestone 3 of the project. Make sure the plans, issues don't have any holes in it, and every functionality, security, and logic measurements are working and well-designed!*

Response: Before we jump into generating the specific GitHub issues for **M3-Frontend Development**, I have analyzed our current backend setup against what the frontend actually needs to function.

I found a few architectural holes and missing pieces that will immediately break the app or block development if we don't address them first. As requested, here are my suggestions and questions before we plan the implementation.

## 🔴 Potential Frontend Holes & Proposals

### 1. The Missing Users Endpoint

- **The Hole:** You mentioned wanting a dropdown on the frontend to select the "owner" of the reservation. However, in Milestone 2, we never created a `UserController` to fetch the seeded users. The frontend currently has no way to populate that dropdown.
- **My Proposal:** We need to backtrack slightly and add a quick backend issue at the start of M3 to create a `GET /api/users` endpoint.

### 2. CORS (Cross-Origin Resource Sharing)

- **The Hole:** Your Vite/React frontend will run on a different port (e.g., `localhost:5173`) than your ASP.NET WebAPI (e.g., `localhost:5000`). By default, browsers will block all API calls due to strict CORS policies.
- **My Proposal:** We must add an issue to configure a CORS policy in the backend's `Program.cs` that allows local frontend development origins.

### 3. Data Fetching Strategy

- **The Hole:** You mentioned adding a `/query` folder to the frontend in a previous step. Managing API calls, caching, and loading states manually in React with `useEffect` can lead to race conditions.
- **My Proposal:** Since you created a `/query` folder, I highly recommend using **TanStack Query (React Query)** combined with **Axios**. It handles caching, loading states, and automatic refetching elegantly.

### 4. Timezone Handling

- **The Hole:** We strictly enforce UTC on the backend. If the frontend sends local time strings without proper timezone indicators, reservations will shift unpredictably.
- **My Proposal:** The frontend should display times in the user's local browser timezone for readability, but we must strictly serialize all dates using JavaScript's `.toISOString()` before sending them to the API.

Do you agree with using React Query and Axios for the frontend data fetching, and shall I include the missing backend tasks (CORS and the Users endpoint) as the first issues in our M3 plan?

---

User prompt: I approve of all your proposals.

Response: Here are the detailed issues for **M3-Frontend Development**. I have included the missing backend tasks (Users endpoint and CORS) at the beginning of the milestone to ensure the frontend has a solid foundation to build upon.

Since this milestone is focused on the frontend foundation and reading data (displaying slots), the interaction logic (booking/canceling) will be left for M4.

#15 [M3-Frontend Development] Implement Users API Endpoint

Task: Create a `UserController` to fetch all users from the database, allowing the frontend to populate the user select

As a frontend application, I want to fetch the list of available users, to allow simulating different reservation options

Acceptance Criteria:

- \* Create `UserResponseDto` containing `Id` and `Name`.
- \* Create `UserController` with an `[ApiController]` and `[Route("api/users")]` attribute.
- \* Add an endpoint `GET /api/users` that returns all users mapped to `UserResponseDto`.
- \* Ensure EF Core uses `.AsNoTracking()` for performance.

#16 [M3-Frontend Development] Test: Implement Users API Endpoint

Task: Write integration tests using `WebApplicationFactory` for the `UserController`.

As a tester, I want to ensure the users API correctly returns the seeded data, to guarantee frontend compatibility.

Acceptance Criteria:

- \* Test `GET /api/users` returns a 200 OK status code.
- \* Test that the response contains exactly 5 users (based on the `DatabaseSeeder`).

#17 [M3-Frontend Development] Configure Backend CORS Policy

Task: Add a Cross-Origin Resource Sharing (CORS) policy to the backend to allow local frontend development servers to

As a developer, I want to allow cross-origin requests from my local Vite app, to avoid browser security blocks during

Acceptance Criteria:

- \* Open `Program.cs` in the backend.
- \* Add a CORS policy that allows requests from `http://localhost:5173` (Vite's default port) and `http://127.0.0.1:5173`.
- \* Allow any HTTP method (GET, POST, DELETE, etc.) and any header.
- \* Apply the CORS middleware via `app.UseCors()` before routing/authorization.

#18 [M3-Frontend Development] Setup Frontend Architecture and Dependencies

Task: Install and configure React Router, Axios, and TanStack React Query in the Vite frontend.

As a frontend developer, I want to set up the core routing and data fetching libraries, to establish a scalable architecture

Acceptance Criteria:

- \* Install `react-router-dom`, `axios`, and `@tanstack/react-query`.
- \* Wrap the root `<App />` component inside `main.tsx` with `<BrowserRouter>`.
- \* Wrap the root `<App />` component with `<QueryClientProvider>` and initialize a new `QueryClient`.

#19 [M3-Frontend Development] Implement API Client and Queries

Task: Configure the Axios instance and create React Query hooks for fetching backend data.

As a frontend application, I want centralized API fetching logic, to cleanly manage server state, caching, and loading

Acceptance Criteria:

- \* Create an `axiosInstance` in a new file (e.g., `src/api/axios.ts`) configured with the backend base URL (e.g., `http://localhost:5000`).
- \* In the `/query` folder, create a `useUsers` hook that fetches `GET /api/users` using `useQuery`.
- \* In the `/query` folder, create a `useParkingSlots` hook that fetches `GET /api/parkingslots` using `useQuery`.
- \* Define TypeScript interfaces in the `/models` folder matching the backend DTOs (`UserResponseDto`, `ParkingSlotResponse`).

#20 [M3-Frontend Development] Create Application Layout and Routing

Task: Set up the main layout frame and the basic page routes for the application.

As a user, I want a consistent layout and navigation, to easily move around the application.

Acceptance Criteria:

- \* Create a `Layout` component in `/components` that includes a simple Navigation bar.
- \* Configure `react-router-dom` routes in `App.tsx`:
  - \* `/` -> Dashboard (Parking slots display)
  - \* `*` -> 404 Not Found page.

#21 [M3-Frontend Development] Build Parking Slots Dashboard

Task: Create the UI to list all parking slots and their current reservations on the dashboard.

As a user, I want to view all parking slots and see when they are booked, to plan my reservation.

Acceptance Criteria:

- \* Create a `Dashboard` component in the `/pages` folder.
- \* Utilize the `useParkingSlots` hook to fetch data.
- \* Display a loading state while fetching and an error state if the fetch fails.

- \* Render a list or grid of parking slots, displaying their `Designation`.
- \* For each slot, map over its `Reservations` and display the existing booking times.
- \* Parse the UTC `StartTime` and `EndTime` strings and format them to the user's local timezone for display (using st

---

*User prompt: Task: Create a UserController to fetch all users from the database, allowing the frontend to populate the user selection dropdown. As a frontend application, I want to fetch the list of available users, to allow simulating different reservation owners. Acceptance Criteria: Create UserResponseDto containing Id and Name. Create UserController with an [ApiController] and [Route("api/users")] attribute. Add an endpoint GET /api/users that returns all users mapped to UserResponseDto. Ensure EF Core uses .AsNoTracking() for performance.*

Response: Here is the implementation for the UserController and the corresponding DTO. This will allow the frontend to easily fetch the seeded users to populate the owner dropdown.

## 1. User Response DTO

First, create the DTO to ensure we only send the necessary data (Id and Name) to the frontend, keeping the response lightweight.

Create a new file named UserResponseDto.cs inside your /src/ParkingBackend/Parking.WebAPI/DTOs/ folder.

### UserResponseDto.cs

```
using System;

namespace Parking.WebAPI.DTOs
{
    public class UserResponseDto
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
    }
}
```

## 2. User Controller

Next, create the controller to handle the API request. We will use .AsNoTracking() to optimize the database query since we are only reading the data and not modifying it.

Create a new file named UserController.cs inside your /src/ParkingBackend/Parking.WebAPI/Controllers/ folder.

### UserController.cs

```
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOs;

namespace Parking.WebAPI.Controllers
{
    [ApiController]
    [Route("api/users")]
    public class UserController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public UserController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/users
        [HttpGet]
        public async Task<ActionResult<IEnumerable<UserResponseDto>>> GetUsers()
        {
            var users = await _context.Users
                .AsNoTracking()
                .Select(u => new UserResponseDto
                {
                    Id = u.Id,
                    Name = u.Name
                })
                .ToListAsync();

            return Ok(users);
        }
    }
}
```

---

*User prompt: Task: Write integration tests using WebApplicationFactory for the UserController. As a tester, I want to ensure the users API correctly returns the seeded data, to guarantee frontend compatibility. Acceptance Criteria: Test GET /api/users returns a 200 OK status code. Test that the response contains exactly 5 users (based on the DatabaseSeeder).*

Response: Here is the implementation of the integration tests for your UserController. This ensures that the frontend will successfully receive the 5 dummy users that our DatabaseSeeder creates on startup.

Create a new file named UserControllerTests.cs inside your Parking.WebAPI.IntegrationTests project.

#### **UserControllerTests.cs**

```
using System.Collections.Generic;
using System.Net;
using System.Net.Http.Json;
using System.Threading.Tasks;
using FluentAssertions;
using Microsoft.AspNetCore.Mvc.Testing;
using Parking.WebAPI.DTOs;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class UserControllerTests : IClassFixture<WebApplicationFactory<Program>>
    {
        private readonly WebApplicationFactory<Program> _factory;

        public UserControllerTests(WebApplicationFactory<Program> factory)
        {
            _factory = factory;
        }

        [Fact]
        public async Task GetUsers_ReturnsOkAndExactly5SeededUsers()
        {
            // Arrange
            var client = _factory.CreateClient();

            // Act
            var response = await client.GetAsync("/api/users");

            // Assert
            // 1. Test GET /api/users returns a 200 OK status code.
            response.StatusCode.Should().Be(HttpStatusCode.OK);

            // 2. Test that the response contains exactly 5 users.
            var users = await response.Content.ReadFromJsonAsync<List<UserResponseDto>>();
            users.Should().NotBeNull();
            users.Should().HaveCount(5, "because the DatabaseSeeder injects exactly 5 dummy users on startup");
        }
    }
}
```

---

*User prompt: Task: Add a Cross-Origin Resource Sharing (CORS) policy to the backend to allow local frontend development servers to make HTTP requests. As a developer, I want to allow cross-origin requests from my local Vite app, to avoid browser security blocks during development. Acceptance Criteria: Open Program.cs in the backend. Add a CORS policy that allows requests from http://localhost:5173 (Vite's default port) and http://127.0.0.1:5173. Allow any HTTP method (GET, POST, DELETE, etc.) and any header. Apply the CORS middleware via app.UseCors() before routing/authorization.*

Response: Here is the implementation to configure the CORS policy in your backend. This will ensure your Vite frontend can communicate with the API without the browser blocking the requests.

Update your /src/ParkingBackend/Parking.WebAPI/Program.cs file with the following changes.

#### **Program.cs**

```
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.Services;

var builder = WebApplication.CreateBuilder(args);

// Add services to the container.
builder.Services.AddControllers();

// Configure SQLite DbContext
builder.Services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlite(builder.Configuration.GetConnectionString("DefaultConnection")));
```

```
// Register Business Services
builder.Services.AddScoped<IReservationService, ReservationService>();

// 1. Add CORS policy configuration
builder.Services.AddCors(options =>
{
    options.AddPolicy("AllowViteFrontend", policy =>
    {
        policy.WithOrigins("http://localhost:5173", "http://127.0.0.1:5173")
            .AllowAnyHeader()
            .AllowAnyMethod();
    });
});

var app = builder.Build();

// Initialize and Seed the Database
using (var scope = app.Services.CreateScope())
{
    var services = scope.ServiceProvider;
    try
    {
        var context = services.GetRequiredService<ApplicationDbContext>();
        await DatabaseSeeder.SeedAsync(context);
    }
    catch (Exception ex)
    {
        var logger = services.GetRequiredService<ILogger<Program>>();
        logger.LogError(ex, "An error occurred while seeding the database.");
    }
}

// Configure the HTTP request pipeline.

// 2. Apply the CORS middleware before authorization and mapping controllers
app.UseCors("AllowViteFrontend");

app.UseAuthorization();
app.MapControllers();

app.Run();

// Required for integration tests
public partial class Program { }
```

## Key Implementation Details

- **builder.Services.AddCors**: Registers the CORS service and defines a specific policy named "AllowViteFrontend".
- **.WithOrigins(...)**: Explicitly whitelists the exact addresses Vite uses by default.
- **.AllowAnyMethod()** and **.AllowAnyHeader()**: Ensures GET, POST, DELETE, etc., as well as any custom headers (like Content-Type: application/json), pass the preflight checks.
- **app.UseCors(...)**: Placed strictly before **app.UseAuthorization()** and **app.MapControllers()** to guarantee that the CORS headers are attached to the response before the request is processed or rejected by the routing pipeline.

---

*User prompt: Task: Install and configure React Router, Axios, and TanStack React Query in the Vite frontend. As a frontend developer, I want to set up the core routing and data fetching libraries, to establish a scalable architecture. Acceptance Criteria: Install react-router-dom, axios, and @tanstack/react-query. Wrap the root <App /> component inside main.tsx with <BrowserRouter>. Wrap the root <App /> component with <QueryClientProvider> and initialize a new QueryClient.*

*Response: Here is the implementation for setting up your frontend architecture. This configuration wraps your application with the necessary providers to enable routing and robust data fetching.*

## 1. Install Dependencies

Open your terminal, navigate to your frontend directory (/src/ParkingFrontend), and run the following command to install the required libraries:

```
npm install react-router-dom axios @tanstack/react-query
```

*(Note: If you are using yarn or pnpm, substitute npm install with yarn add or pnpm add respectively.)*

## 2. Configure main.tsx

Update your frontend entry point to initialize the QueryClient and wrap your application in the QueryClientProvider and BrowserRouter.

Open `/src/ParkingFrontend/src/main.tsx` and replace its contents with the following:

#### main.tsx

```
import React from 'react';
import ReactDOM from 'react-dom/client';
import { BrowserRouter } from 'react-router-dom';
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import App from './App.tsx';
// import './index.css' // Uncomment if you have global styles

// Initialize the TanStack Query client
const queryClient = new QueryClient({
  defaultOptions: {
    queries: {
      refetchOnWindowFocus: false, // Prevents refetching every time you click back to the browser
      retry: 1, // Number of retry attempts on failure
    },
  },
});

ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <QueryClientProvider client={queryClient}>
      <BrowserRouter>
        <App />
      </BrowserRouter>
    </QueryClientProvider>
  </React.StrictMode>
);
```

### Key Implementation Details

- **Order of Providers:** The `<QueryClientProvider>` is placed at the top level, followed by `<BrowserRouter>`. This ensures that all routed components have access to the query cache.
- **QueryClient Defaults:** I added some sensible default options to the `QueryClient`. Disabling `refetchOnWindowFocus` prevents the API from being hit excessively during manual testing when you tab in and out of the browser.

---

*User prompt: Task: Configure the Axios instance and create React Query hooks for fetching backend data. As a frontend application, I want centralized API fetching logic, to cleanly manage server state, caching, and loading indicators. Acceptance Criteria: Create an axiosInstance in a new file (e.g., `src/api/axios.ts`) configured with the backend base URL (e.g., `http://localhost:5000`). In the `/query` folder, create a `useUsers` hook that fetches `GET /api/users` using `useQuery`. In the `/query` folder, create a `useParkingSlots` hook that fetches `GET /api/parkingslots` using `useQuery`. Define TypeScript interfaces in the `/models` folder matching the backend DTOs (`UserResponseDto`, `ParkingSlotResponseDto`, `ReservationResponseDto`).*

*Response:* Here is the implementation for your frontend API configuration, TypeScript models, and React Query hooks. This setup creates a clean boundary between your UI components and your data fetching logic.

### 1. TypeScript Models

Create a new file named `types.ts` inside your `/src/ParkingFrontend/src/models/` folder. These interfaces match your C# DTOs, ensuring type safety across your frontend.

#### src/models/types.ts

```
export interface UserResponseDto {
  id: string;
  name: string;
}

export interface ReservationResponseDto {
  id: string;
  parkingSlotId: string;
  userId: string;
  startTime: string; // Serialized as ISO 8601 string from backend
  endTime: string; // Serialized as ISO 8601 string from backend
}

export interface ParkingSlotResponseDto {
  id: string;
  designation: string;
  reservations: ReservationResponseDto[];
}
```

### 2. Axios Instance



Create a new folder named `/api` inside your `/src/ParkingFrontend/src/` directory, and create a file named `axios.ts` inside it. This centralizes the base URL configuration.

#### `src/api/axios.ts`

```
import axios from 'axios';

// Ensure the port matches your ASP.NET Core backend URL.
// 5000/5001 is standard, but check your backend console if it fails to connect.
export const axiosInstance = axios.create({
  baseURL: 'http://localhost:5000',
  headers: {
    'Content-Type': 'application/json',
  },
});
```

### 3. React Query Hooks

Create these files inside your `/src/ParkingFrontend/src/query/` folder. They wrap the Axios calls in TanStack Query hooks to handle caching, loading, and error states automatically.

#### `src/query/useUsers.ts`

```
import { useQuery } from '@tanstack/react-query';
import { axiosInstance } from '../api/axios';
import { UserResponseDto } from '../models/types';

export const useUsers = () => {
  return useQuery<UserResponseDto[], Error>({
    queryKey: ['users'],
    queryFn: async () => {
      const response = await axiosInstance.get<UserResponseDto[]>('/api/users');
      return response.data;
    },
  });
};
```

#### `src/query/useParkingSlots.ts`

```
import { useQuery } from '@tanstack/react-query';
import { axiosInstance } from '../api/axios';
import { ParkingSlotResponseDto } from '../models/types';

export const useParkingSlots = () => {
  return useQuery<ParkingSlotResponseDto[], Error>({
    queryKey: ['parkingSlots'],
    queryFn: async () => {
      const response = await axiosInstance.get<ParkingSlotResponseDto[]>('/api/parkingslots');
      return response.data;
    },
  });
};
```

---

*User prompt: Task: Set up the main layout frame and the basic page routes for the application. As a user, I want a consistent layout and navigation, to easily move around the application. Acceptance Criteria: Create a Layout component in /components that includes a simple Navigation bar. Configure react-router-dom routes in App.tsx: / -> Dashboard (Parking slots display) \* -> 404 Not Found page.*

*Response: Here is the implementation for your frontend layout and routing. This sets up a solid structural frame for your application, using React Router's <Outlet/> to nest pages within a consistent navigation bar.*

### 1. Placeholder Pages

First, let's create the basic page components so our router has something to render. Create these files inside your `/src/ParkingFrontend/src/pages/` folder.

#### `src/pages/Dashboard.tsx`

```
export const Dashboard = () => {
  return (
    <div>
      <h1>Dashboard</h1>
      <p>Parking slots will be displayed here.</p>
    </div>
  );
};
```

#### `src/pages/NotFound.tsx`

```
import { Link } from 'react-router-dom';

export const NotFound = () => {
  return (
    <div style={{ textAlign: 'center', marginTop: '50px' }}>
      <h2>404 - Page Not Found</h2>
      <p>The page you are looking for does not exist.</p>
      <Link to="/" style={{ color: 'blue', textDecoration: 'underline' }}>
        Return to Dashboard
      </Link>
    </div>
  );
};
```

## 2. Layout Component

Create a `Layout.tsx` file inside your `/src/ParkingFrontend/src/components/` folder. This component renders a navigation bar and uses the `<Outlet/>` component to inject whichever page matches the current route.

`src/components/Layout.tsx`

```
import { Outlet, Link } from 'react-router-dom';

export const Layout = () => {
  return (
    <div style={{ minHeight: '100vh', display: 'flex', flexDirection: 'column' }}>
      <nav style={{
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center',
        padding: '1rem 2rem',
        backgroundColor: '#f3f4f6',
        borderBottom: '1px solid #e5e7eb'
      }}>
        <h2 style={{ margin: 0, color: '#111827' }}>Parking Admin</h2>
        <div style={{ display: 'flex', gap: '1rem' }}>
          <Link to="/" style={{ textDecoration: 'none', color: '#374151', fontWeight: 'bold' }}>
            Dashboard
          </Link>
        </div>
      </nav>

      <main style={{ padding: '2rem', flex: 1, backgroundColor: 'ffffff' }}>
        { /* The matched child route will render inside this Outlet */ }
        <Outlet />
      </main>
    </div>
  );
};
```

## 3. Application Routing

Finally, update your `App.tsx` file to define the route hierarchy. We wrap the `Dashboard` and `NotFound` pages inside the `Layout` route.

`src/App.tsx`

```
import { Routes, Route } from 'react-router-dom';
import { Layout } from './components/Layout';
import { Dashboard } from './pages/Dashboard';
import { NotFound } from './pages/NotFound';

function App() {
  return (
    <Routes>
      { /* Parent route with the layout */ }
      <Route path="/" element={<Layout />}>
        { /* Index route renders at exactly "/" */ }
        <Route index element={<Dashboard />} />

        { /* Catch-all route for undefined paths */ }
        <Route path="*" element={<NotFound />} />
      </Route>
    </Routes>
  );
}

export default App;
```

---

*User prompt: Task: Create the UI to list all parking slots and their current reservations on the dashboard. As a user, I want to view all parking slots and see when they are booked, to plan my reservation. Acceptance Criteria: Create a Dashboard component in the /pages folder. Utilize the useParkingSlots hook to fetch data. Display a loading state while fetching and an error state if the fetch fails. Render a list or grid of parking slots, displaying their Designation. For each slot, map over its Reservations and display the existing booking times. Parse the UTC StartTime and EndTime strings and format them to the user's local timezone for display (using standard JavaScript Date methods).*

**Response:** Here is the complete implementation for the Dashboard component. It fetches the data via React Query, handles the loading and error states, and utilizes standard JavaScript Date functionality to automatically translate the backend's UTC timestamps into the user's local browser timezone.

Update the placeholder file we created earlier in /src/ParkingFrontend/src/pages/.

**src/pages/Dashboard.tsx**

```
import { useParkingSlots } from '../query/useParkingSlots';

export const Dashboard = () => {
  const { data: slots, isLoading, isError, error } = useParkingSlots();

  // 1. Loading State
  if (isLoading) {
    return (
      <div style={{ textAlign: 'center', marginTop: '3rem', color: '#4b5563' }}>
        <h2>Loading parking slots...</h2>
      </div>
    );
  }

  // 2. Error State
  if (isError) {
    return (
      <div style={{ textAlign: 'center', marginTop: '3rem', color: '#ef4444' }}>
        <h2>Failed to load data</h2>
        <p>{error?.message}</p>
      </div>
    );
  }

  // 3. Success State
  return (
    <div>
      <h2 style={{ borderBottom: '2px solid #e5e7eb', paddingBottom: '0.5rem' }}>
        Parking Slots Availability
      </h2>

      <div style={{
        display: 'grid',
        gridTemplateColumns: 'repeat(auto-fill, minmax(300px, 1fr))',
        gap: '1.5rem',
        marginTop: '1.5rem'
      }}>
        {slots?.map((slot) => (
          <div key={slot.id} style={{
            border: '1px solid #d1d5db',
            borderRadius: '8px',
            padding: '1.5rem',
            backgroundColor: '#f9fafb',
            boxShadow: '0 2px 4px rgba(0,0,0,0.05)'
          }}>
            <h3 style={{ margin: '0 0 1rem 0', color: '#1f2937' }}>
              Slot: {slot.designation}
            </h3>

            {slot.reservations.length === 0 ? (
              <p style={{ color: '#6b7280', fontStyle: 'italic', margin: 0 }}>
                Available / No reservations.
              </p>
            ) : (
              <ul style={{
                listStyleType: 'none',
                padding: 0,
                margin: 0,
                display: 'flex',
                flexDirection: 'column',
                gap: '0.75rem'
              }}>
                {slot.reservations.map((res) => {
                  // Standard JavaScript Date object automatically parses the ISO 8601 UTC string
                  // and formats it to the browser's local timezone.
                  const startLocal = new Date(res.startTime).toLocaleString();
```

```

const endLocal = new Date(res.endTime).toLocaleString();

return (
  <li key={res.id} style={{
    backgroundColor: 'ffffff',
    border: '1px solid #e5e7eb',
    padding: '0.75rem',
    borderRadius: '6px',
    fontSize: '0.875rem',
    color: '#374151'
  }}>
    <div style={{ marginBottom: '4px' }}>
      <span style={{ fontWeight: 600, color: '#10b981' }}>Start:</span> {startLocal}
    </div>
    <div>
      <span style={{ fontWeight: 600, color: '#ef4444' }}>End:</span> {endLocal}
    </div>
  </li>
);
}}}
</ul>
)}
</div>
)}
</div>
);
};

```

---

*User prompt: Next, let's head into planning the Milestone 4 of the project. Make sure the plans, issues don't have any holes in it, and every functionality, security, and logic measurements are working and well-designed!*

Response: Before we map out the specific GitHub issues for **M4-Frontend Interactions**, we need to lock down exactly how the user interface will handle the write operations (booking and canceling). Since we are simulating users and dealing with strict time rules, there are a few UI/UX design holes we need to address to ensure a smooth, foolproof experience.

## 🔴 Potential Frontend Holes & Proposals

### 1. Simulating the "Logged-In" User

- **The Hole:** You mentioned selecting the user from a dropdown. Should this dropdown be inside the reservation form itself (meaning you pick who you are booking *for* every time), or should it be a global dropdown in the Navigation bar to simulate a "logged-in" session?
- **My Proposal:** I recommend placing the User Dropdown in the Navigation bar (Layout component) and storing the selected user in a React Context. This makes the app feel more authentic, as all subsequent booking and cancellation actions will inherently belong to the "active" user.

### 2. Form Validation & User Experience

- **The Hole:** The backend strictly rejects reservations starting in the past or lasting less than 30 minutes. If we rely solely on the backend, the user will have to submit the form just to see an error.
- **My Proposal:** We should implement client-side validation on the reservation form. The inputs should prevent selecting past dates/times, and the form should disable the "Submit" button if the duration is less than 30 minutes, providing immediate UI feedback.

### 3. The Booking Flow (UI Placement)

- **The Hole:** How does a user actually initiate a booking from the Dashboard?
- **My Proposal:** Add a "Book Slot" button to each Parking Slot card. Clicking it should open a reusable Modal component containing the reservation form, passing the `ParkingSlotId` and `Designation` as props. This keeps the user on the dashboard so they can still see other bookings while deciding their time.

### 4. Cancellation Safety

- **The Hole:** The backend prevents canceling reservations that have already started. On the frontend, if a user clicks a "Cancel" button on an active/past reservation, the backend will reject it with a 400 error.
- **My Proposal:** The frontend should conditionally render the "Cancel" button. It should only appear on a reservation if:
  1. The reservation's `StartTime` is in the future.
  2. (Optional but recommended) The reservation belongs to the currently "logged-in" simulated user.

How would you like to handle the user selection (global context vs. per-form), and do you agree with using a Modal for the booking flow?

---

*User prompt: 1. I Accept your proposal 2. Add the client side validation too. 3. I Accept your proposal. 4. I accept your proposal (the optional one too)*

Response: Excellent. With those decisions locked in, we have a very solid, user-friendly design. Here are the detailed issues for **M4-Frontend Interactions**.

These issues cover the global state, the modal UI, client-side validation, and the React Query mutations to talk to your backend safely.

#22 [M4-Frontend Interactions] Implement Global User Context (Simulated Auth)

Task: Create a React Context to manage the currently "logged-in" user globally and update the navigation bar with a

As a user, I want to select my identity globally, so that all my bookings and cancellations are securely tied to my

Acceptance Criteria:

- \* Create a `UserContext` and `UserProvider` in a new `/context` folder.
- \* The context state should hold the `selectedUserId` and a method to update it.
- \* Update the `Layout` component's navigation bar to include a `<select>` dropdown.
- \* Populate the dropdown using the `useUsers` query.
- \* Set the default selected user to the first user in the fetched list.

#23 [M4-Frontend Interactions] Create Modal and Reservation Form UI

Task: Build a reusable Modal component and a Reservation Form with strict client-side validation.

As a user, I want to book a slot using a form that validates my inputs immediately, to prevent me from submitting in

Acceptance Criteria:

- \* Create a `Modal` component in `/components` that renders over the screen with a backdrop and a close button.
- \* Create a `ReservationForm` component in `/components` that takes `parkingSlotId` and `designation` as props.
- \* Include inputs for `StartTime` (datetime-local) and `EndTime` (datetime-local).
- \* Implement client-side validation:
  - `StartTime` cannot be in the past (compared to `new Date()`).
  - `EndTime` must be at least 30 minutes after `StartTime`.
- \* Disable the "Submit" button and show a UI warning text if the validation fails.

#24 [M4-Frontend Interactions] Implement Create Reservation Mutation

Task: Hook the Reservation Form up to the backend API using React Query mutations and handle API-level rejections (1

As a user, I want my valid booking request saved to the database and the dashboard updated immediately.

Acceptance Criteria:

- \* Create a `useCreateReservation` hook in `/query` using TanStack Query's `useMutation`.
- \* The mutation should `POST` to `/api/reservations` using the `axiosInstance`.
- \* Read the `UserId` directly from the `UserContext`.
- \* Convert the local form dates to UTC ISO strings (`.toISOString()`) before sending the payload.
- \* On success, use `queryClient.invalidateQueries({ queryKey: ['parkingSlots'] })` to automatically refresh the dashb
- \* On error (e.g., 400 Bad Request due to overlap), display the backend's error message in the form.
- \* Update the Dashboard cards to include a "Book Slot" button that opens the Modal containing this form.

#25 [M4-Frontend Interactions] Implement Safe Cancellation Feature

Task: Add a cancellation mutation and conditionally render a cancel button only for valid, future reservations owned

As a user, I want to cancel my upcoming reservations, but I should not be able to cancel past events or other people

Acceptance Criteria:

- \* Create a `useCancelReservation` hook in `/query` that sends a `DELETE` request to `/api/reservations/{id}`.
- \* On success, invalidate the `['parkingSlots']` query to refresh the UI.
- \* Update the Dashboard `Reservation` list items to include a "Cancel" button.
- \* **\*\*Security & UI Logic:\*\*** The Cancel button must ONLY render if BOTH of these are true:
  1. The reservation's `userId` matches the current `selectedUserId` from the `UserContext`.
  2. The reservation's `startTime` is strictly in the future (`new Date(res.startTime) > new Date()`).
- \* Show a loading state on the button while the mutation is pending.

---

User prompt: Task: Create a React Context to manage the currently "logged-in" user globally and update the navigation bar with a user selection dropdown. As a user, I want to select my identity globally, so that all my bookings and cancellations are securely tied to my simulated account. Acceptance Criteria: Create a UserContext and UserProvider in a new /context folder. The context state should hold the selectedUserId and a method to update it. Update the Layout component's navigation bar to include a <select> dropdown. Populate the dropdown using the useUsers query. Set the default selected user to the first user in the fetched list.

Response: Here is the implementation to set up the global user context and update the navigation bar to support simulating a logged-in user.

## 1. Create the User Context

Create a new folder named /context inside /src/ParkingFrontend/src/ and add the UserContext.tsx file. This file provides the context, the provider component, and a custom hook for easy consumption.

src/context/UserContext.tsx

```
import { createContext, useState, ReactNode, useContext } from 'react';

interface UserContextType {
  selectedUserId: string;
  setSelectedUserId: (id: string) => void;
}
```

```

}

export const UserContext = createContext<UserContextType | undefined>(undefined);

export const UserProvider = ({ children }: { children: ReactNode }) => {
  const [selectedUserId, setSelectedUserId] = useState<string>('');

  return (
    <UserContext.Provider value={{ selectedUserId, setSelectedUserId }}>
      {children}
    </UserContext.Provider>
  );
};

// Custom hook for easier access to the context
export const useUser = () => {
  const context = useContext(UserContext);
  if (!context) {
    throw new Error('useUser must be used within a UserProvider');
  }
  return context;
};

```

## 2. Wrap the Application with the Provider

Update your `App.tsx` file to wrap the routing logic inside the newly created `UserProvider`.

**src/App.tsx**

```

import { Routes, Route } from 'react-router-dom';
import { Layout } from '../components/Layout';
import { Dashboard } from '../pages/Dashboard';
import { NotFound } from '../pages/NotFound';
import { UserProvider } from '../context/UserContext';

function App() {
  return (
    <UserProvider>
      <Routes>
        {/* Parent route with the layout */}
        <Route path="/" element={<Layout />}>
          {/* Index route renders at exactly "/" */}
          <Route index element={<Dashboard />} />

          {/* Catch-all route for undefined paths */}
          <Route path="*" element={<NotFound />} />
        </Route>
      </Routes>
    </UserProvider>
  );
}

export default App;

```

## 3. Update the Layout Component

Now modify the `Layout.tsx` to include the user dropdown. We will use the `useUsers` hook to fetch the data and a `useEffect` to automatically select the first user once the data is loaded.

**src/components/Layout.tsx**

```

import { Outlet, Link } from 'react-router-dom';
import { useEffect } from 'react';
import { useUsers } from '../../query/useUsers';
import { useUser } from '../../context/UserContext';

export const Layout = () => {
  const { data: users, isLoading, isError } = useUsers();
  const { selectedUserId, setSelectedUserId } = useUser();

  // Set the default selected user to the first user in the fetched list
  useEffect(() => {
    if (users && users.length > 0 && !selectedUserId) {
      setSelectedUserId(users[0].id);
    }
  }, [users, selectedUserId, setSelectedUserId]);

  return (
    <div style={{ minHeight: '100vh', display: 'flex', flexDirection: 'column' }}>
      <nav style={{

```

```

display: 'flex',
justifyContent: 'space-between',
alignItems: 'center',
padding: '1rem 2rem',
backgroundColor: '#f3f4f6',
borderBottom: '1px solid #e5e7eb'
}}>
<div style={{ display: 'flex', alignItems: 'center', gap: '2rem' }}>
  <h2 style={{ margin: 0, color: '#111827' }}>Parking Admin</h2>
  <Link to="/" style={{ textDecoration: 'none', color: '#374151', fontWeight: 'bold' }}>
    Dashboard
  </Link>
</div>

{/* User Selection Dropdown */}
<div style={{ display: 'flex', alignItems: 'center', gap: '1rem' }}>
  <label htmlFor="user-select" style={{ fontSize: '0.875rem', fontWeight: 600, color: '#374151' }}>
    Simulated User:
  </label>

  {isLoading ? (
    <span style={{ fontSize: '0.875rem', color: '#6b7280' }}>Loading users...</span>
  ) : isError ? (
    <span style={{ fontSize: '0.875rem', color: '#ef4444' }}>Failed to load users</span>
  ) : (
    <select
      id="user-select"
      value={selectedUserId}
      onChange={(e) => setSelectedUserId(e.target.value)}
      style={{
        padding: '0.5rem',
        borderRadius: '6px',
        border: '1px solid #d1d5db',
        backgroundColor: 'ffffff',
        color: '#111827',
        cursor: 'pointer'
      }}
    >
      {users?.map((user) => (
        <option key={user.id} value={user.id}>
          {user.name}
        </option>
      ))}
    </select>
  )}
</div>
</nav>

<main style={{ padding: '2rem', flex: 1, backgroundColor: 'ffffff' }}>
  {/* The matched child route will render inside this Outlet */}
  <Outlet />
</main>
</div>
);
};

```

---

*User prompt: I got this error: #30 3.639 src/context/UserContext.tsx(1,35): error TS1484: 'ReactNode' is a type and must be imported using a type-only import when 'verbatimModuleSyntax' is enabled. #30 ERROR: process "/bin/sh -c npm run build" did not complete successfully: exit code: 2 Pinpoint the issue and suggest 3 solution paths*

## Response: The Issue

The error occurs because newer Vite and TypeScript templates enable a strict compiler option called "verbatimModuleSyntax" in your tsconfig.json (or tsconfig.app.json).

This rule enforces that any import used strictly as a TypeScript type (like ReactNode) must be explicitly marked as a type import. Since ReactNode doesn't exist at runtime, TypeScript wants to know it can safely strip it out during the build process.

Here are 3 ways to fix this:

### Path 1: Use an inline type modifier (Recommended)

You can keep everything on one line by adding the type keyword directly in front of ReactNode. This is the cleanest fix and adheres to your project's strict configuration.

Update line 1 of src/context/UserContext.tsx to:

```
import { createContext, useState, type ReactNode, useContext } from 'react';
```

## Path 2: Use a separate import type statement

If you prefer visual separation between your runtime code and your TypeScript types, you can split the import into two separate lines.

Replace line 1 of `src/context/UserContext.tsx` with:

```
import { createContext, useState, useContext } from 'react';
import type { ReactNode } from 'react';
```

## Path 3: Disable verbatimModuleSyntax

If you find this rule too tedious and don't want to enforce explicit type imports across your entire project, you can turn it off.

Open your `tsconfig.json` (or `tsconfig.app.json` if using a newer Vite template) and change the rule to `false`:

```
{
  "compilerOptions": {
    // ... other options
    "verbatimModuleSyntax": false
  }
}
```

(Note: Path 1 or 2 is generally preferred to keep your build step as optimized as possible, but Path 3 is perfectly valid if you want to prioritize developer convenience).

---

*User prompt: Task: Create a React Context to manage the currently "logged-in" user globally and update the navigation bar with a user selection dropdown. As a user, I want to select my identity globally, so that all my bookings and cancellations are securely tied to my simulated account. Acceptance Criteria: Create a UserContext and UserProvider in a new /context folder. The context state should hold the selectedUserId and a method to update it. Update the Layout component's navigation bar to include a <select> dropdown. Populate the dropdown using the useUsers query. Set the default selected user to the first user in the fetched list.*

*Response: Here is the updated implementation with the type ReactNode fix applied so it complies with your strict TypeScript configuration (verbatimModuleSyntax).*

## 1. Create the User Context (Fixed)

Create the `/context` folder inside `/src/ParkingFrontend/src/` and add the `UserContext.tsx` file. Notice the `type` keyword before `ReactNode` on line 1.

`src/context/UserContext.tsx`

```
import { createContext, useState, type ReactNode, useContext } from 'react';

interface UserContextType {
  selectedUserId: string;
  setSelectedUserId: (id: string) => void;
}

export const UserContext = createContext<UserContextType | undefined>(undefined);

export const UserProvider = ({ children }: { children: ReactNode }) => {
  const [selectedUserId, setSelectedUserId] = useState<string>('');

  return (
    <UserContext.Provider value={{ selectedUserId, setSelectedUserId }}>
      {children}
    </UserContext.Provider>
  );
};

// Custom hook for easier access to the context
export const useUser = () => {
  const context = useContext(UserContext);
  if (!context) {
    throw new Error('useUser must be used within a UserProvider');
  }
  return context;
};
```

## 2. Wrap the Application with the Provider

If you haven't already, update your `App.tsx` file to wrap the routing logic inside the newly created `UserProvider`.

`src/App.tsx`

```
import { Routes, Route } from 'react-router-dom';
import { Layout } from '../components/Layout';
import { Dashboard } from '../pages/Dashboard';
import { NotFound } from '../pages/NotFound';
```



```
import { UserProvider } from '../context/UserContext';

function App() {
  return (
    <UserProvider>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Dashboard />} />
          <Route path="*" element={<NotFound />} />
        </Route>
      </Routes>
    </UserProvider>
  );
}

export default App;
```

Finally, here is the updated `Layout.tsx` which includes the user dropdown. It handles the loading states gracefully and uses a `useEffect` hook to default the dropdown to the first fetched user.

```
import { Outlet, Link } from 'react-router-dom';
import { useEffect } from 'react';
import { useUsers } from '../query/useUsers';
import { useUser } from '../context/UserContext';

export const Layout = () => {
  const { data: users, isLoading, isError } = useUsers();
  const { selectedUserId, setSelectedUserId } = useUser();

  // Set the default selected user to the first user in the fetched list
  useEffect(() => {
    if (users && users.length > 0 && !selectedUserId) {
      setSelectedUserId(users[0].id);
    }
  }, [users, selectedUserId, setSelectedUserId]);

  return (
    <div style={{ minHeight: '100vh', display: 'flex', flexDirection: 'column' }}>
      <nav style={{
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center',
        padding: '1rem 2rem',
        backgroundColor: '#f3f4f6',
        borderBottom: '1px solid #e5e7eb'
      }}>
        <div style={{ display: 'flex', alignItems: 'center', gap: '2rem' }}>
          <h2 style={{ margin: 0, color: '#111827' }}>Parking Admin</h2>
          <Link to="/" style={{ textDecoration: 'none', color: '#374151', fontWeight: 'bold' }}>
            Dashboard
          </Link>
        </div>

        <div style={{ display: 'flex', alignItems: 'center', gap: '1rem' }}>
          <label htmlFor="user-select" style={{ fontSize: '0.875rem', fontWeight: 600, color: '#374151' }}>
            Simulated User:
          </label>

          {isLoading ? (
            <span style={{ fontSize: '0.875rem', color: '#6b7280' }}>Loading users...</span>
          ) : isError ? (
            <span style={{ fontSize: '0.875rem', color: '#ef4444' }}>Failed to load users</span>
          ) : (
            <select
              id="user-select"
              value={selectedUserId}
              onChange={(e) => setSelectedUserId(e.target.value)}
              style={{
                padding: '0.5rem',
                borderRadius: '6px',
                border: '1px solid #d1d5db',
                backgroundColor: 'ffffff',
                color: '#111827',
                cursor: 'pointer'
              }}
            />
          )}
        </div>
      </nav>
    </div>
  );
}
```

```

    }}
  >
    {users?.map((user) => (
      <option key={user.id} value={user.id}>
        {user.name}
      </option>
    ))}
  </select>
)}
</div>
</nav>

<main style={{ padding: '2rem', flex: 1, backgroundColor: 'ffffff' }}>
  <Outlet />
</main>
</div>
);
};

```

---

*User prompt: Task: Build a reusable Modal component and a Reservation Form with strict client-side validation. As a user, I want to book a slot using a form that validates my inputs immediately, to prevent me from submitting invalid times. Acceptance Criteria: Create a Modal component in /components that renders over the screen with a backdrop and a close button. Create a ReservationForm component in /components that takes parkingSlotId and designation as props. Include inputs for StartTime (datetime-local) and EndTime (datetime-local). Implement client-side validation: StartTime cannot be in the past (compared to new Date()). EndTime must be at least 30 minutes after StartTime. Disable the "Submit" button and show a UI warning text if the validation fails.*

Response: Here is the implementation for the reusable Modal component and the ReservationForm with strict client-side validation.

## 1. Reusable Modal Component

Create a new file named Modal.tsx inside your /src/ParkingFrontend/src/components/ folder. I am utilizing the type ReactNode import format to ensure it passes your strict TypeScript build rules.

src/components/Modal.tsx

```

import { type ReactNode } from 'react';

interface ModalProps {
  isOpen: boolean;
  onClose: () => void;
  children: ReactNode;
}

export const Modal = ({ isOpen, onClose, children }: ModalProps) => {
  if (!isOpen) return null;

  return (
    <div style={{
      position: 'fixed',
      top: 0,
      left: 0,
      right: 0,
      bottom: 0,
      backgroundColor: 'rgba(0, 0, 0, 0.6)',
      display: 'flex',
      justifyContent: 'center',
      alignItems: 'center',
      zIndex: 1000,
      padding: '1rem'
    }}>
      <div style={{
        backgroundColor: 'ffffff',
        padding: '2rem',
        borderRadius: '8px',
        width: '100%',
        maxWidth: '500px',
        position: 'relative',
        boxShadow: '0 20px 25px -5px rgba(0, 0, 0, 0.1), 0 10px 10px -5px rgba(0, 0, 0, 0.04)'
      }}>
        <button
          onClick={onClose}
          style={{
            position: 'absolute',
            top: '1rem',
            right: '1rem',
            background: 'none',
            border: 'none',
            fontSize: '1.5rem',

```

```

        lineHeight: 1,
        cursor: 'pointer',
        color: '#6b7280'
      }}
      aria-label="Close modal"
    >
      &times;
    </button>
    {children}
  </div>
</div>
);
};

```

## 2. Reservation Form Component

Create a new file named `ReservationForm.tsx` inside your `/src/ParkingFrontend/src/components/` folder. This component handles the client-side validation requirements, ensuring the user gets immediate feedback before attempting a database request.

`src/components/ReservationForm.tsx`

```

import { useState, useEffect } from 'react';

interface ReservationFormProps {
  parkingSlotId: string;
  designation: string;
  onClose: () => void;
}

export const ReservationForm = ({ parkingSlotId, designation, onClose }: ReservationFormProps) => {
  const [startTime, setStartTime] = useState('');
  const [endTime, setEndTime] = useState('');
  const [validationError, setValidationError] = useState('');

  // Re-run validation whenever start or end time changes
  useEffect(() => {
    if (!startTime || !endTime) {
      setValidationError(''); // Don't show errors until both are filled
      return;
    }

    const start = new Date(startTime);
    const end = new Date(endTime);
    const now = new Date();

    if (start < now) {
      setValidationError('Start time cannot be in the past.');
```

```

</p>

<form onSubmit={handleSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>

  <div style={{ display: 'flex', flexDirection: 'column', gap: '0.5rem' }}>
    <label htmlFor="startTime" style={{ fontSize: '0.875rem', fontWeight: 600 }}>Start Time</label>
    <input
      type="datetime-local"
      id="startTime"
      value={startTime}
      onChange={(e) => setStartTime(e.target.value)}
      style={{ padding: '0.5rem', borderRadius: '4px', border: '1px solid #d1d5db' }}
      required
    />
  </div>

  <div style={{ display: 'flex', flexDirection: 'column', gap: '0.5rem' }}>
    <label htmlFor="endTime" style={{ fontSize: '0.875rem', fontWeight: 600 }}>End Time</label>
    <input
      type="datetime-local"
      id="endTime"
      value={endTime}
      onChange={(e) => setEndTime(e.target.value)}
      style={{ padding: '0.5rem', borderRadius: '4px', border: '1px solid #d1d5db' }}
      required
    />
  </div>

  {/* Validation Warning */}
  {validationError && (
    <div style={{ color: '#ef4444', fontSize: '0.875rem', fontWeight: 500 }}>
      {validationError}
    </div>
  )}

  <div style={{ display: 'flex', justifyContent: 'flex-end', gap: '1rem', marginTop: '1rem' }}>
    <button
      type="button"
      onClick={onClose}
      style={{
        padding: '0.5rem 1rem', borderRadius: '4px', border: '1px solid #d1d5db',
        backgroundColor: 'ffffff', cursor: 'pointer'
      }}
    >
      Cancel
    </button>
    <button
      type="submit"
      disabled={!isFormValid}
      style={{
        padding: '0.5rem 1rem', borderRadius: '4px', border: 'none',
        backgroundColor: isFormValid ? '#3b82f6' : '#9ca3af',
        color: 'ffffff', cursor: isFormValid ? 'pointer' : 'not-allowed'
      }}
    >
      Submit
    </button>
  </div>
</form>
</div>
);
};

```

*User prompt: Task: Hook the Reservation Form up to the backend API using React Query mutations and handle API-level rejections (like overlaps). As a user, I want my valid booking request saved to the database and the dashboard updated immediately. Acceptance Criteria: Create a useCreateReservation hook in /query using TanStack Query's useMutation. The mutation should POST to /api/reservations using the axiosInstance. Read the UserId directly from the UserContext. Convert the local form dates to UTC ISO strings (.toISOString()) before sending the payload. On success, use queryClient.invalidateQueries({ queryKey: ['parkingSlots'] }) to automatically refresh the dashboard. On error (e.g., 400 Bad Request due to overlap), display the backend's error message in the form. Update the Dashboard cards to include a "Book Slot" button that opens the Modal containing this form.*

Response: Here is the implementation to connect your frontend form to the backend API. We will create the mutation hook, update the form to submit the request safely, and update the dashboard to trigger the modal.

## 1. Create the Mutation Hook

Create a new file named useCreateReservation.ts inside your /src/ParkingFrontend/src/query/ folder. This hook uses TanStack Query's useMutation and automatically invalidates the dashboard's cache upon success.

### src/query/useCreateReservation.ts

```
import { useMutation, useQueryClient } from '@tanstack/react-query';
import { AxiosError } from 'axios';
import { axiosInstance } from '../api/axios';
import { ReservationResponseDto } from '../models/types';

export interface ReservationRequestDto {
  parkingSlotId: string;
  userId: string;
  startTime: string;
  endTime: string;
}

interface ApiError {
  message: string;
}

export const useCreateReservation = () => {
  const queryClient = useQueryClient();

  return useMutation<ReservationResponseDto, AxiosError<ApiError>, ReservationRequestDto>({
    mutationFn: async (request) => {
      const response = await axiosInstance.post<ReservationResponseDto>('/api/reservations', request);
      return response.data;
    },
    onSuccess: () => {
      // Invalidate the 'parkingSlots' query to force the dashboard to fetch the fresh data
      queryClient.invalidateQueries({ queryKey: ['parkingSlots'] });
    },
  });
};
```

## 2. Update the Reservation Form

Update your existing ReservationForm.tsx. We will integrate the useUser context to grab the simulated user, format the dates to UTC ISO strings, and handle the mutation states (loading and error).

### src/components/ReservationForm.tsx

```
import { useState, useEffect } from 'react';
import { useUser } from '../context/UserContext';
import { useCreateReservation } from '../query/useCreateReservation';

interface ReservationFormProps {
  parkingSlotId: string;
  designation: string;
  onClose: () => void;
}

export const ReservationForm = ({ parkingSlotId, designation, onClose }: ReservationFormProps) => {
  const [startTime, setStartTime] = useState('');
  const [endTime, setEndTime] = useState('');
  const [validationError, setValidationError] = useState('');
  const [apiError, setApiError] = useState('');

  const { selectedUserId } = useUser();
  const createReservation = useCreateReservation();

  useEffect(() => {
    setApiError(''); // Clear API error when inputs change

    if (!startTime || !endTime) {
      setValidationError('');
      return;
    }

    const start = new Date(startTime);
    const end = new Date(endTime);
    const now = new Date();

    if (start < now) {
      setValidationError('Start time cannot be in the past.');
```

```

    return;
  }

  setValidationError('');
}, [startTime, endTime]);

const handleSubmit = (e: React.FormEvent) => {
  e.preventDefault();
  if (validationError || !startTime || !endTime) return;

  if (!selectedUserId) {
    setApiError('No user is currently selected. Please select one from the navigation bar.');
```

return;

```
  }

  const request = {
    parkingSlotId,
    userId: selectedUserId,
    // Convert browser local time to strictly formatted UTC strings for the backend
    startTime: new Date(startTime).toISOString(),
    endTime: new Date(endTime).toISOString(),
  };

  createReservation.mutate(request, {
    onSuccess: () => {
      onClose();
    },
    onError: (error) => {
      const message = error.response?.data?.message || 'An unexpected error occurred.';
      setApiError(message);
    }
  });
});

const isFormValid = startTime && endTime && !validationError && !createReservation.isPending;

return (
  <div>
    <h2 style={{ marginTop: 0, marginBottom: '0.5rem', color: '#111827' }}>Book Slot</h2>
    <p style={{ color: '#6b7280', marginBottom: '1.5rem' }}>
      Reserving slot: <strong>{designation}</strong>
    </p>

    <form onSubmit={handleSubmit} style={{ display: 'flex', flexDirection: 'column', gap: '1rem' }}>

      <div style={{ display: 'flex', flexDirection: 'column', gap: '0.5rem' }}>
        <label htmlFor="startTime" style={{ fontSize: '0.875rem', fontWeight: 600 }}>Start Time</label>
        <input
          type="datetime-local"
          id="startTime"
          value={startTime}
          onChange={(e) => setStartTime(e.target.value)}
          style={{ padding: '0.5rem', borderRadius: '4px', border: '1px solid #d1d5db' }}
          required
          disabled={createReservation.isPending}
        />
      </div>

      <div style={{ display: 'flex', flexDirection: 'column', gap: '0.5rem' }}>
        <label htmlFor="endTime" style={{ fontSize: '0.875rem', fontWeight: 600 }}>End Time</label>
        <input
          type="datetime-local"
          id="endTime"
          value={endTime}
          onChange={(e) => setEndTime(e.target.value)}
          style={{ padding: '0.5rem', borderRadius: '4px', border: '1px solid #d1d5db' }}
          required
          disabled={createReservation.isPending}
        />
      </div>

      {/* Client-side Validation Warning */}
      {validationError && (
        <div style={{ color: '#ef4444', fontSize: '0.875rem', fontWeight: 500 }}>
          {validationError}
        </div>
      )}

      {/* Backend API Error Warning */}

```

```

    {apiError && (
      <div style={{ color: '#b91c1c', fontSize: '0.875rem', fontWeight: 500, backgroundColor: '#fee2e2', padding: 10px }}>
        {apiError}
      </div>
    )}

    <div style={{ display: 'flex', justifyContent: 'flex-end', gap: '1rem', marginTop: '1rem' }}>
      <button
        type="button"
        onClick={onClose}
        disabled={createReservation.isPending}
        style={{
          padding: '0.5rem 1rem', borderRadius: '4px', border: '1px solid #d1d5db',
          backgroundColor: '#ffffff', cursor: createReservation.isPending ? 'not-allowed' : 'pointer'
        }}
      >
        Cancel
      </button>
      <button
        type="submit"
        disabled={!isFormValid}
        style={{
          padding: '0.5rem 1rem', borderRadius: '4px', border: 'none',
          backgroundColor: isFormValid ? '#3b82f6' : '#9ca3af',
          color: '#ffffff', cursor: isFormValid ? 'pointer' : 'not-allowed'
        }}
      >
        {createReservation.isPending ? 'Submitting...' : 'Submit'}
      </button>
    </div>
  </form>
</div>
);
};

```

### 3. Update the Dashboard Component

Update your existing `Dashboard.tsx` to include the new "Book Slot" buttons, manage the `Modal` state, and inject the `ReservationForm` when a user clicks the button.

`src/pages/Dashboard.tsx`

```

import { useState } from 'react';
import { useParkingSlots } from '../query/useParkingSlots';
import { Modal } from '../components/Modal';
import { ReservationForm } from '../components/ReservationForm';

export const Dashboard = () => {
  const { data: slots, isLoading, isError, error } = useParkingSlots();

  // State to handle which slot the user is currently booking
  const [isModalOpen, setIsModalOpen] = useState(false);
  const [selectedSlot, setSelectedSlot] = useState<{ id: string; designation: string } | null>(null);

  if (isLoading) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#4b5563' }}><h2>Loading parking slots</h2></div>
  if (isError) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#ef4444' }}><h2>Failed to load parking slots</h2></div>

  const handleOpenModal = (id: string, designation: string) => {
    setSelectedSlot({ id, designation });
    setIsModalOpen(true);
  };

  const handleCloseModal = () => {
    setIsModalOpen(false);
    setSelectedSlot(null);
  };

  return (
    <div>
      <h2 style={{ borderBottom: '2px solid #e5e7eb', padding: '0.5rem 0' }}>
        Parking Slots Availability
      </h2>

      <div style={{
        display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(320px, 1fr))',
        gap: '1.5rem', marginTop: '1.5rem'
      }}>
        {slots?.map((slot) => (
          <div key={slot.id} style={{

```

```

border: '1px solid #d1d5db', borderRadius: '8px', padding: '1.5rem',
backgroundColor: '#f9fafb', boxShadow: '0 2px 4px rgba(0,0,0,0.05)'
}}>
<div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '1re
  <h3 style={{ margin: 0, color: '#1f2937' }}>Slot: {slot.designation}</h3>
  <button
    onClick={() => handleOpenModal(slot.id, slot.designation)}
    style={{
      padding: '0.5rem 1rem', backgroundColor: '#10b981', color: 'white',
      border: 'none', borderRadius: '4px', cursor: 'pointer', fontWeight: 600, fontSize: '0.875rem'
    }}
  >
    Book Slot
  </button>
</div>

{slot.reservations.length === 0 ? (
  <p style={{ color: '#6b7280', fontStyle: 'italic', margin: 0 }}>Available / No reservations.</p>
) : (
  <ul style={{ listStyleType: 'none', padding: 0, margin: 0, display: 'flex', flexDirection: 'column', g
    {slot.reservations.map((res) => {
      const startLocal = new Date(res.startTime).toLocaleString();
      const endLocal = new Date(res.endTime).toLocaleString();

      return (
        <li key={res.id} style={{
          backgroundColor: 'ffffff', border: '1px solid #e5e7eb', padding: '0.75rem',
          borderRadius: '6px', fontSize: '0.875rem', color: '#374151'
        }}>
          <div style={{ marginBottom: '4px' }}><span style={{ fontWeight: 600, color: '#10b981' }}>Start
            <div><span style={{ fontWeight: 600, color: '#ef4444' }}>End:</span> {endLocal}</div>
          </li>
        </li>
      );
    })}
  </ul>
)}
</div>
))}
</div>

<Modal isOpen={isModalOpen} onClose={handleCloseModal}>
  {selectedSlot && (
    <ReservationForm
      parkingSlotId={selectedSlot.id}
      designation={selectedSlot.designation}
      onClose={handleCloseModal}
    />
  )}
</Modal>
</div>
);
};

```

*User prompt: Task: Add a cancellation mutation and conditionally render a cancel button only for valid, future reservations owned by the current user. As a user, I want to cancel my upcoming reservations, but I should not be able to cancel past events or other people's bookings. Acceptance Criteria: Create a useCancelReservation hook in /query that sends a DELETE request to /api/reservations/{id}. On success, invalidate the [parkingSlots] query to refresh the UI. Update the Dashboard Reservation list items to include a "Cancel" button. Security & UI Logic: The Cancel button must ONLY render if BOTH of these are true: The reservation's userId matches the current selectedUserId from the UserContext. The reservation's startTime is strictly in the future (new Date(res.startTime) > new Date()). Show a loading state on the button while the mutation is pending.*

**Response:** Here is the implementation to complete the cancellation feature. This will add the React Query mutation for the DELETE request and apply the strict frontend security/UI rules to conditionally display the "Cancel" button.

## 1. Create the Cancellation Hook

Create a new file named useCancelReservation.ts inside your /src/ParkingFrontend/src/query/ folder.

**src/query/useCancelReservation.ts**

```

import { useMutation, useQueryClient } from '@tanstack/react-query';
import { AxiosError } from 'axios';
import { axiosInstance } from '../api/axios';

interface ApiError {
  message: string;
}

```



```

export const useCancelReservation = () => {
  const queryClient = useQueryClient();

  return useMutation<void, AxiosError<ApiError>, string>({
    mutationFn: async (reservationId: string) => {
      await axiosInstance.delete(`/api/reservations/${reservationId}`);
    },
    onSuccess: () => {
      // Refresh the dashboard data automatically
      queryClient.invalidateQueries({ queryKey: ['parkingSlots'] });
    },
  });
};

```

## 2. Update the Dashboard Component

Update your existing Dashboard.tsx. We will import the useUser context to verify ownership and the useCancelReservation hook to process the deletion.

src/pages/Dashboard.tsx

```

import { useState } from 'react';
import { useParkingSlots } from '../query/useParkingSlots';
import { Modal } from '../components/Modal';
import { ReservationForm } from '../components/ReservationForm';
import { useUser } from '../context/UserContext';
import { useCancelReservation } from '../query/useCancelReservation';

export const Dashboard = () => {
  const { data: slots, isLoading, isError, error } = useParkingSlots();
  const { selectedUserId } = useUser();
  const cancelReservation = useCancelReservation();

  const [isModalOpen, setIsModalOpen] = useState(false);
  const [selectedSlot, setSelectedSlot] = useState<{ id: string; designation: string } | null>(null);

  if (isLoading) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#4b5563' }}><h2>Loading parking
  if (isError) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#ef4444' }}><h2>Failed to load d

  const handleOpenModal = (id: string, designation: string) => {
    setSelectedSlot({ id, designation });
    setIsModalOpen(true);
  };

  const handleCloseModal = () => {
    setIsModalOpen(false);
    setSelectedSlot(null);
  };

  const handleCancel = (reservationId: string) => {
    if (window.confirm('Are you sure you want to cancel this reservation?')) {
      cancelReservation.mutate(reservationId, {
        onError: (err) => {
          alert(err.response?.data?.message || 'Failed to cancel reservation');
        }
      });
    }
  };

  return (
    <div>
      <h2 style={{ borderBottom: '2px solid #e5e7eb', paddingBottom: '0.5rem' }}>
        Parking Slots Availability
      </h2>

      <div style={{
        display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(320px, 1fr))',
        gap: '1.5rem', marginTop: '1.5rem'
      }}>
        {slots?.map((slot) => (
          <div key={slot.id} style={{
            border: '1px solid #d1d5db', borderRadius: '8px', padding: '1.5rem',
            backgroundColor: '#f9fafb', boxShadow: '0 2px 4px rgba(0,0,0,0.05)'
          }}>
            <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '1re
              <h3 style={{ margin: 0, color: '#1f2937' }}>Slot: {slot.designation}</h3>
              <button
                onClick={() => handleOpenModal(slot.id, slot.designation)}
                style={{

```

```

padding: '0.5rem 1rem', backgroundColor: '#10b981', color: 'white',
border: 'none', borderRadius: '4px', cursor: 'pointer', fontWeight: 600, fontSize: '0.875rem'
}}
>
  Book Slot
</button>
</div>

{slot.reservations.length === 0 ? (
  <p style={{ color: '#6b7280', fontStyle: 'italic', margin: 0 }}>Available / No reservations.</p>
) : (
  <ul style={{ listStyleType: 'none', padding: 0, margin: 0, display: 'flex', flexDirection: 'column', gap: 10px }}>
    {slot.reservations.map((res) => {
      const startLocal = new Date(res.startTime).toLocaleString();
      const endLocal = new Date(res.endTime).toLocaleString();

      // Security & UI Logic
      const isFuture = new Date(res.startTime) > new Date();
      const isOwner = res.userId === selectedUserId;
      const canCancel = isFuture && isOwner;

      // Track loading state for this specific button
      const isCancelingThis = cancelReservation.isPending && cancelReservation.variables === res.id;

      return (
        <li key={res.id} style={{
          backgroundColor: 'white', border: '1px solid #e5e7eb', padding: '0.75rem',
          borderRadius: '6px', fontSize: '0.875rem', color: '#374151',
          display: 'flex', justifyContent: 'space-between', alignItems: 'center'
        }}>
          <div>
            <div style={{ marginBottom: '4px' }}><span style={{ fontWeight: 600, color: '#10b981' }}>Start</span> {startLocal}</div>
            <div><span style={{ fontWeight: 600, color: '#ef4444' }}>End</span> {endLocal}</div>
          </div>

          {/* Conditionally Render Cancel Button */}
          {canCancel && (
            <button
              onClick={() => handleCancel(res.id)}
              disabled={isCancelingThis}
              style={{
                padding: '0.4rem 0.75rem',
                backgroundColor: isCancelingThis ? '#f3f4f6' : '#fee2e2',
                color: isCancelingThis ? '#9ca3af' : '#b91c1c',
                border: `1px solid ${isCancelingThis ? '#d1d5db' : '#fca5a5'}`,
                borderRadius: '4px',
                cursor: isCancelingThis ? 'not-allowed' : 'pointer',
                fontWeight: 600,
                fontSize: '0.75rem',
                transition: 'background-color 0.2s'
              }}
            >
              {isCancelingThis ? 'Canceling...' : 'Cancel'}
            </button>
          )}
        </li>
      );
    })}
  </ul>
)}
</div>
))}
</div>

<Modal isOpen={isModalOpen} onClose={handleCloseModal}>
  {selectedSlot && (
    <ReservationForm
      parkingSlotId={selectedSlot.id}
      designation={selectedSlot.designation}
      onClose={handleCloseModal}
    />
  )}
</Modal>
</div>
);
};

```

User prompt: I have a few more additional implementation ideas. 1) I don't want to be able to cancel other user's reservations! Block it on the backend too. 2) I want to have a page on the frontend project, which displays all the reservations made by the selected user. 3) I want to have special types of parking slots: VIP, Electric cars, Accessible slots. For these slots, I only want specific users to be able to make a reservation. Each user should have a new property, noting whether they are authorized to use the special slots. First, make design specific questions, if there are any. Next, create the issues for the new developments.

Response: These are excellent additions! They add realistic business rules, improve security, and flesh out the user experience.

Before we finalize the exact implementation details, I have a few design-specific questions to ensure we build this exactly how you envision it.



## Design & Architecture Questions

**1. Passing the User Context for Backend Security (Feature 1)** Since we don't have a real authentication system (like JWT tokens), how should we tell the backend *who* is requesting the cancellation?

- **My Proposal:** We can configure the frontend's Axios instance to send a custom HTTP header (e.g., X-User-Id) with every request, mapped to the currently selected user in the context. The backend will read this header to verify ownership before canceling. Does this work for you?

**2. Modeling Special Slot Authorizations (Feature 3)** How should we model the user permissions in the database?

- **My Proposal:** We create a SlotType enum (Regular, VIP, Electric, Accessible). The ParkingSlot entity gets a Type property. The User entity gets a collection of authorized SlotTypes (e.g., User A is authorized for Regular and Electric). This is highly scalable if you add more types later.

**3. Fetching "My Reservations" (Feature 2)** Currently, we only fetch reservations grouped by parking slots.

- **My Proposal:** We should create a new backend endpoint: GET /api/users/{userId}/reservations that returns a flat list of a specific user's bookings, ordered by time. This is much more efficient than fetching all slots on the frontend and filtering them manually.

**4. Frontend UX for Special Slots (Feature 3)** If a user is selected who *does not* have access to a VIP slot, how should the UI react?

- **My Proposal:** We render a visual badge (e.g., ⚡ Electric, ♿ Accessible, 🏠 VIP) on the dashboard cards. If the user lacks the authorization, we disable the "Book Slot" button completely and show a small tooltip explaining why.



## Proposed GitHub Issues (Milestone 5)

Assuming you agree with the proposals above, here is the complete set of issues to build these features. (If you want to change any of the approaches, we can easily tweak these!)

#26 [M5-Advanced Features] Backend: Enforce Cancellation Ownership

Task: Prevent users from canceling reservations that do not belong to them.

As a secure API, I want to ensure the requester owns the reservation they are trying to delete.

Acceptance Criteria:

- \* Update `IReservationService.CancelReservationAsync(Guid id, Guid requestingUserId)` to require the user ID.
- \* Inside the service, throw an `UnauthorizedAccessException` if the reservation's `UserId` doesn't match the `requestingUserId`.
- \* Update `ReservationController.CancelReservation` to extract the `requestingUserId` from a custom HTTP header: `X-User-Id`.
- \* Catch `UnauthorizedAccessException` in the controller and return a `403 Forbidden`.

#27 [M5-Advanced Features] Backend: Add "User Reservations" Endpoint

Task: Create an endpoint to retrieve all reservations for a specific user.

As a frontend application, I want to fetch a user's reservations directly, to populate a dedicated profile page.

Acceptance Criteria:

- \* Create a new endpoint in `UserController` (or `ReservationController`): `GET /api/users/{userId}/reservations`.
- \* Return a list of `ReservationResponseDto`.
- \* The query should include the `ParkingSlot` data (to display the designation to the user) and be ordered by `StartTime`.
- \* Ensure it uses `.AsNoTracking()` for performance.

#28 [M5-Advanced Features] Backend: Implement Special Parking Slot Types

Task: Update the database schema to support slot types and user authorizations, and enforce these rules during booking.

As a parking administrator, I want to restrict certain slots to authorized users only.

Acceptance Criteria:

- \* Create a `SlotType` Enum (`Regular`, `VIP`, `Electric`, `Accessible`).
- \* Add `SlotType` to the `ParkingSlot` entity.
- \* Add a collection (or flags) to the `User` entity to represent their authorized slot types.
- \* Update `DatabaseSeeder` to assign random types to slots and random authorizations to the dummy users.
- \* Update `ReservationService.CreateReservationAsync`: Before creating, verify the user has the required authorization for the slot type.
- \* Update all corresponding DTOs to include these new fields.

#29 [M5-Advanced Features] Frontend: Update Global Axios Interceptor

Task: Automatically append the selected user ID to API requests.

As a frontend application, I want to send my simulated authentication state to the backend on every request.

Acceptance Criteria:

- \* Update the `axiosInstance` configuration or add an Axios interceptor.

- \* Read the `selectedUserId` from local storage or context.
- \* Inject the `X-User-Id` header into all outgoing requests.

#30 [M5-Advanced Features] Frontend: Build "My Reservations" Page  
Task: Create a new page displaying a flat list of the active user's bookings.

As a user, I want a dedicated page to see my own bookings, making it easy to manage my schedule.

Acceptance Criteria:

- \* Create a `useUserReservations` query hook targeting the new backend endpoint.
- \* Create a `MyReservations.tsx` page and map it to the `/my-reservations` route in `App.tsx`.
- \* Add a "My Reservations" link to the Navigation bar.
- \* Display the reservations in a list/table, showing the Slot Designation, Start Time, and End Time (formatted to local time).
- \* Reuse the `useCancelReservation` hook to allow users to cancel directly from this list (only for future reservations).

#31 [M5-Advanced Features] Frontend: Special Slots UX & Authorization Checks  
Task: Display slot types visually and prevent unauthorized booking attempts.

As a user, I want to clearly see which slots require special authorization and be prevented from booking them if I am not authorized.

Acceptance Criteria:

- \* Update `types.ts` to include the new `SlotType` fields on the DTOs.
- \* Update the `Dashboard` cards to display a visual badge indicating the slot type (e.g., a crown for VIP, a plug for Electric).
- \* Read the current user's authorized slot types (may require updating the `useUsers` hook or `UserContext`).
- \* Visually disable the "Book Slot" button if the current user lacks the required authorization, and add a `title` to the button.

*User prompt: 1. Put the X-User-Id in the HTTP headers! 2. I accept the proposal. 3. I accept the proposal. 4. I accept the proposal. Also, Include the icons in the user dropdown too, after the name in parentheses, like (⚡🔌👑). Also there is one issue on the frontend: The reservation dashboard displays the time incorrectly, the local timezone is not working. Also, I want the reservation date to be autofilled to today's date. The time does not need to be prefilled.*

**Response:** These are fantastic refinements. Using Unicode emojis (⚡, 🔌, 👑) directly inside the standard HTML `<option>` tags is a brilliant, lightweight way to show authorizations in the dropdown without needing a complex custom component.

**Regarding the `timezone` bug:** This is a classic issue when using SQLite and .NET. EF Core often pulls `DateTime` out of SQLite as `DateTimeKind.Unspecified`. When the API sends this to the frontend, it omits the Z at the end of the ISO string, causing the browser to assume the time is *already* local. We will add an issue to fix this serialization.

**Regarding the `date` autofill:** The HTML `<input type="datetime-local">` requires both date and time to be set simultaneously. Since you only want the date prefilled, we will split the form into one `<input type="date">` (defaulted to today) and two `<input type="time">` fields (left blank).

Here are the finalized issues for **Milestone 5 - Advanced Features & Bug Fixes**:

#26 [M5-Advanced Features] Backend: Enforce Cancellation Ownership  
Task: Prevent users from canceling reservations that do not belong to them.

As a secure API, I want to ensure the requester owns the reservation they are trying to delete using HTTP headers.

Acceptance Criteria:

- \* Update `IReservationService.CancelReservationAsync(Guid id, Guid requestingUserId)` to require the user ID.
- \* Inside the service, throw an `UnauthorizedAccessException` if the reservation's `UserId` doesn't match the `requestingUserId`.
- \* Update `ReservationController.CancelReservation` to extract the `requestingUserId` from the `X-User-Id` HTTP header.
- \* Catch `UnauthorizedAccessException` in the controller and return a `403 Forbidden`.

#27 [M5-Advanced Features] Backend: Implement Special Parking Slot Types  
Task: Update the database schema to support slot types and user authorizations, and enforce these rules during booking.

As a parking administrator, I want to restrict certain slots to authorized users only.

Acceptance Criteria:

- \* Create a `SlotType` Enum (`Regular = 0`, `VIP = 1`, `Electric = 2`, `Accessible = 3`).
- \* Add `SlotType` to the `ParkingSlot` entity.
- \* Add a `List<SlotType> AuthorizedSlotTypes` to the `User` entity (stored appropriately in EF Core).
- \* Update `DatabaseSeeder` to assign random types to slots and random authorizations to the dummy users.
- \* Update `ReservationService.CreateReservationAsync`: Throw an `InvalidOperationException` if the user lacks the required authorization for the slot type.
- \* Update all corresponding DTOs (`UserResponseDto`, `ParkingSlotResponseDto`) to include these new fields.

#28 [M5-Advanced Features] Backend: Add "User Reservations" Endpoint  
Task: Create an endpoint to retrieve all reservations for a specific user.

As a frontend application, I want to fetch a user's reservations directly, to populate a dedicated profile page.

Acceptance Criteria:

- \* Create a new endpoint: `GET /api/users/{userId}/reservations`.
- \* Return a list of `ReservationResponseDto` mapped from the database.
- \* The query should include the associated `ParkingSlot` data and be ordered by `StartTime` descending.
- \* Ensure EF Core uses `.AsNoTracking()`.

#29 [M5-Advanced Features] Frontend: Update Axios to Send X-User-Id

Task: Automatically append the selected user ID to API requests via HTTP headers.

As a frontend application, I want to send my simulated authentication state to the backend on every request.

Acceptance Criteria:

- \* Update `src/api/axios.ts` to utilize an Axios interceptor.
- \* Before a request is sent, read the current `selectedUserId` (this may require exposing a getter from a store or ad
- \* Inject the `X-User-Id` header into all outgoing requests.

#30 [M5-Advanced Features] Frontend: Build "My Reservations" Page

Task: Create a new page displaying a flat list of the active user's bookings.

As a user, I want a dedicated page to see my own bookings, making it easy to manage my schedule.

Acceptance Criteria:

- \* Create a `useUserReservations` query hook targeting the new `GET /api/users/{userId}/reservations` endpoint.
- \* Create a `MyReservations.tsx` page and map it to the `/my-reservations` route in `App.tsx`.
- \* Add a "My Reservations" link to the Navigation bar in `Layout.tsx`.
- \* Display the reservations, showing the Slot Designation, Start Time, and End Time.
- \* Reuse the `useCancelReservation` hook to allow users to cancel directly from this list (only for future reservatio

#31 [M5-Advanced Features] Frontend: Special Slots UX & User Dropdown Icons

Task: Display slot types visually, add emojis to the user dropdown, and prevent unauthorized booking attempts.

As a user, I want to clearly see which slots require special authorization and my own permissions.

Acceptance Criteria:

- \* Update `types.ts` to include `authorizedSlotTypes` on the User, and `type` on the ParkingSlot.
- \* **Layout.tsx**: Append Unicode emojis to the user's name in the dropdown based on their authorizations (e.g., if t
  - Electric: ⚡
  - Accessible: ♿
  - VIP: 👑
- \* **Dashboard.tsx**: Display matching emojis next to the slot designations on the cards.
- \* Visually disable the "Book Slot" button if the current user lacks the required authorization for that slot, and ad

#32 [M5-Advanced Features] Frontend Bug Fix: Timezone and Form Autofill

Task: Fix the backend-to-frontend UTC serialization issue and improve the Reservation Form UX.

As a user, I want to see the correct local time for reservations and have the date prefilled when booking.

Acceptance Criteria:

- \* **Timezone Fix**: Update the backend DTO mapping (or JSON serialization settings) to explicitly format `DateTime`
- \* **Form Refactor**: Update `ReservationForm.tsx` to replace `datetime-local` inputs with one `- \* **Autofill Date**: Initialize the `date` input state to today's date (`new Date().toISOString().split('T')[0]`).
- \* Re-implement the client-side validation to combine the Date and Time inputs into standard Date objects before vali

Shall we begin implementing these by starting with the backend changes?

---

*User prompt: Task: Prevent users from canceling reservations that do not belong to them. As a secure API, I want to ensure the requester owns the reservation they are trying to delete using HTTP headers. Acceptance Criteria: Update IReservationService.CancelReservationAsync(Guid id, Guid requestingUserId) to require the user ID. Inside the service, throw an UnauthorizedAccessException if the reservation's UserId doesn't match the requestingUserId. Update ReservationController.CancelReservation to extract the requestingUserId from the X-User-Id HTTP header. Catch UnauthorizedAccessException in the controller and return a 403 Forbidden.*

Response: Here is the implementation to enforce cancellation ownership on the backend. We will update the service interface, the service implementation, and the controller to extract the custom HTTP header.

## 1. Update the Service Interface

Modify the IReservationService interface to require the requestingUserId parameter.

src/ParkingBackend/Parking.WebAPI/Services/IReservationService.cs

```
using System;
using System.Threading.Tasks;
using Parking.WebAPI.DTOs;

namespace Parking.WebAPI.Services
{
    public interface IReservationService
    {
        Task<ReservationResponseDto> CreateReservationAsync(ReservationRequestDto request);

        // Updated to include the requesting user's ID
        Task CancelReservationAsync(Guid id, Guid requestingUserId);
    }
}
```

## 2. Update the Service Implementation

Update ReservationService to perform the authorization check before allowing the deletion.

src/ParkingBackend/Parking.WebAPI/Services/ReservationService.cs

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOs;

namespace Parking.WebAPI.Services
{
    public class ReservationService : IReservationService
    {
        private readonly ApplicationDbContext _context;

        public ReservationService(ApplicationDbContext context)
        {
            _context = context;
        }

        public async Task<ReservationResponseDto> CreateReservationAsync(ReservationRequestDto request)
        {
            // ... (Existing CreateReservationAsync logic remains untouched)
            throw new NotImplementedException();
        }

        public async Task CancelReservationAsync(Guid id, Guid requestingUserId)
        {
            var reservation = await _context.Reservations.FindAsync(id);
            if (reservation == null)
            {
                throw new KeyNotFoundException("Reservation not found.");
            }

            // Security: Ensure the requester actually owns this reservation
            if (reservation.UserId != requestingUserId)
            {
                throw new UnauthorizedAccessException("You are not authorized to cancel this reservation.");
            }

            // Time Logic: Cannot cancel past/active reservations
            if (reservation.StartTime <= DateTime.UtcNow)
            {
                throw new InvalidOperationException("Cannot cancel a reservation that has already started or is in t");
            }

            _context.Reservations.Remove(reservation);
            await _context.SaveChangesAsync();
        }
    }
}
```

## 3. Update the Reservation Controller

Update the CancelReservation endpoint to read the X-User-Id header using ASP.NET Core's [FromHeader] attribute, and handle the newly introduced UnauthorizedAccessException.

src/ParkingBackend/Parking.WebAPI/Controllers/ReservationController.cs

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Parking.WebAPI.DTOs;
using Parking.WebAPI.Services;

namespace Parking.WebAPI.Controllers
{
    [ApiController]
    [Route("api/reservations")]
    public class ReservationController : ControllerBase
    {
        private readonly IReservationService _reservationService;

        public ReservationController(IReservationService reservationService)
        {

```

```

        _reservationService = reservationService;
    }

    // POST: api/reservations
    [HttpPost]
    public async Task<ActionResult<ReservationResponseDto>> CreateReservation([FromBody] ReservationRequestDto r
    {
        // ... (Existing CreateReservation logic remains untouched)
        return Ok();
    }

    // DELETE: api/reservations/{id}
    [HttpDelete("{id}")]
    public async Task<IActionResult> CancelReservation(
        Guid id,
        [FromHeader(Name = "X-User-Id")] Guid requestingUserId)
    {
        try
        {
            if (requestingUserId == Guid.Empty)
            {
                return BadRequest(new { message = "The X-User-Id header is missing or invalid." });
            }

            await _reservationService.CancelReservationAsync(id, requestingUserId);
            return NoContent();
        }
        catch (UnauthorizedAccessException ex)
        {
            // Return 403 Forbidden if the user doesn't own the reservation
            return StatusCode(403, new { message = ex.Message });
        }
        catch (KeyNotFoundException ex)
        {
            return NotFound(new { message = ex.Message });
        }
        catch (InvalidOperationException ex)
        {
            return BadRequest(new { message = ex.Message });
        }
        catch (Exception)
        {
            return StatusCode(500, new { message = "An unexpected error occurred while canceling the reservation"
        }
    }
}
}

```

---

*User prompt: Task: Update the database schema to support slot types and user authorizations, and enforce these rules during booking. As a parking administrator, I want to restrict certain slots to authorized users only. Acceptance Criteria: Create a SlotType Enum (Regular = 0, VIP = 1, Electric = 2, Accessible = 3). Add SlotType Type to the ParkingSlot entity. Add a List<SlotType> AuthorizedSlotTypes to the User entity (stored appropriately in EF Core). Update DatabaseSeeder to assign random types to slots and random authorizations to the dummy users. Update ReservationService.CreateReservationAsync: Throw an InvalidOperationException if the user lacks the authorization for the slot's Type. Update all corresponding DTOs (UserResponseDto, ParkingSlotResponseDto) to include these new fields.*

**Response:** Here is the implementation to support special parking slot types and enforce authorization rules. We will create the new Enum, update the Entities, configure EF Core to store the list of types, update the Seeder, and apply the validation in the Service.

## 1. Create the Enum

Create a new file named SlotType.cs in the /src/ParkingBackend/Parking.WebAPI/Models/ (or wherever your entities are stored) folder.

**src/ParkingBackend/Parking.WebAPI/Models/SlotType.cs**

```

namespace Parking.WebAPI.Models
{
    public enum SlotType
    {
        Regular = 0,
        VIP = 1,
        Electric = 2,
        Accessible = 3
    }
}

```

## 2. Update Entities & Database Context

Update the ParkingSlot and User entities, and configure EF Core in ApplicationDbContext to store the List<SlotType> as a JSON string in SQLite.

**src/ParkingBackend/Parking.WebAPI/Models/ParkingSlot.cs**

```
using System;
using System.Collections.Generic;

namespace Parking.WebAPI.Models
{
    public class ParkingSlot
    {
        public Guid Id { get; set; }
        public string Designation { get; set; } = string.Empty;
        public SlotType Type { get; set; } = SlotType.Regular;

        public ICollection<Reservation> Reservations { get; set; } = new List<Reservation>();
    }
}
```

**src/ParkingBackend/Parking.WebAPI/Models/User.cs**

```
using System;
using System.Collections.Generic;

namespace Parking.WebAPI.Models
{
    public class User
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;

        // List of slot types this user is authorized to book
        public List<SlotType> AuthorizedSlotTypes { get; set; } = new List<SlotType> { SlotType.Regular };

        public ICollection<Reservation> Reservations { get; set; } = new List<Reservation>();
    }
}
```

**src/ParkingBackend/Parking.WebAPI/Data/ApplicationDbContext.cs**

```
using System.Collections.Generic;
using System.Text.Json;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.Data
{
    public class ApplicationDbContext : DbContext
    {
        public ApplicationDbContext(DbContextOptions<ApplicationDbContext> options) : base(options) { }

        public DbSet<User> Users { get; set; }
        public DbSet<ParkingSlot> ParkingSlots { get; set; }
        public DbSet<Reservation> Reservations { get; set; }

        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            base.OnModelCreating(modelBuilder);

            // Configure EF Core to store the List<SlotType> as a JSON string
            modelBuilder.Entity<User>()
                .Property(u => u.AuthorizedSlotTypes)
                .HasConversion(
                    v => JsonSerializer.Serialize(v, (JsonSerializerOptions)null),
                    v => JsonSerializer.Deserialize<List<SlotType>>(v, (JsonSerializerOptions)null) ?? new List<SlotType>()
                );
        }
    }
}
```

### 3. Update the DTOs

Update both DTOs to include the new fields so the frontend can receive them.

**src/ParkingBackend/Parking.WebAPI/DTOs/UserResponseDto.cs**

```
using System;
using System.Collections.Generic;
using Parking.WebAPI.Models;
```



```

namespace Parking.WebAPI.DTOS
{
    public class UserResponseDto
    {
        public Guid Id { get; set; }
        public string Name { get; set; } = string.Empty;
        public List<SlotType> AuthorizedSlotTypes { get; set; } = new();
    }
}

```

**src/ParkingBackend/Parking.WebAPI/DTOS/ParkingSlotResponseDto.cs**

```

using System;
using System.Collections.Generic;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.DTOS
{
    public class ParkingSlotResponseDto
    {
        public Guid Id { get; set; }
        public string Designation { get; set; } = string.Empty;
        public SlotType Type { get; set; }
        public List<ReservationResponseDto> Reservations { get; set; } = new();
    }
}

```

*(Note: You will also need to update the UserController and ParkingSlotController to map these new fields into the DTOS).*

#### 4. Update the Database Seeder

Modify the seeder to distribute types and authorizations randomly (or fixed, for easier testing).

**src/ParkingBackend/Parking.WebAPI/Data/DatabaseSeeder.cs**

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.Data
{
    public static class DatabaseSeeder
    {
        public static async Task SeedAsync(ApplicationDbContext context)
        {
            await context.Database.EnsureCreatedAsync();

            if (!context.Users.Any())
            {
                var users = new List<User>
                {
                    new User { Id = Guid.NewGuid(), Name = "Alice (Admin)", AuthorizedSlotTypes = new List<SlotType>
                    new User { Id = Guid.NewGuid(), Name = "Bob (Electric)", AuthorizedSlotTypes = new List<SlotType>
                    new User { Id = Guid.NewGuid(), Name = "Charlie (Accessible)", AuthorizedSlotTypes = new List<SlotType>
                    new User { Id = Guid.NewGuid(), Name = "Dave (Regular)", AuthorizedSlotTypes = new List<SlotType>
                    new User { Id = Guid.NewGuid(), Name = "Eve (VIP)", AuthorizedSlotTypes = new List<SlotType> { S
                };
                await context.Users.AddRangeAsync(users);
                await context.SaveChangesAsync();
            }

            if (!context.ParkingSlots.Any())
            {
                var slots = new List<ParkingSlot>
                {
                    new ParkingSlot { Id = Guid.NewGuid(), Designation = "A1", Type = SlotType.VIP },
                    new ParkingSlot { Id = Guid.NewGuid(), Designation = "A2", Type = SlotType.Electric },
                    new ParkingSlot { Id = Guid.NewGuid(), Designation = "A3", Type = SlotType.Accessible },
                    new ParkingSlot { Id = Guid.NewGuid(), Designation = "B1", Type = SlotType.Regular },
                    new ParkingSlot { Id = Guid.NewGuid(), Designation = "B2", Type = SlotType.Regular },
                    new ParkingSlot { Id = Guid.NewGuid(), Designation = "B3", Type = SlotType.Regular }
                };
                await context.ParkingSlots.AddRangeAsync(slots);
                await context.SaveChangesAsync();
            }
        }
    }
}

```

```
}
}
```

## 5. Update Reservation Service Validation

Add the authorization check into the `CreateReservationAsync` method.

`src/ParkingBackend/Parking.WebAPI/Services/ReservationService.cs` (Inside your `CreateReservationAsync` method...)

```
public async Task<ReservationResponseDto> CreateReservationAsync(ReservationRequestDto request)
{
    var slot = await _context.ParkingSlots.FindAsync(request.ParkingSlotId);
    if (slot == null)
        throw new KeyNotFoundException("Parking slot not found.");

    var user = await _context.Users.FindAsync(request.UserId);
    if (user == null)
        throw new KeyNotFoundException("User not found.");

    // NEW: Authorization Check
    if (slot.Type != SlotType.Regular && !user.AuthorizedSlotTypes.Contains(slot.Type))
    {
        throw new InvalidOperationException($"User is not authorized to book this {slot.Type} parking slot.");
    }

    // ... (Existing overlapping and time validation logic remains untouched)

    // Create and save the reservation...
}
```

*Don't forget to run `dotnet ef migrations add AddSpecialSlotTypes` and `dotnet ef database update` (or just delete your local SQLite database so the app recreates it on next run with the new schema).*

*User prompt: Task: Create an endpoint to retrieve all reservations for a specific user. As a frontend application, I want to fetch a user's reservations directly, to populate a dedicated profile page. Acceptance Criteria: Create a new endpoint: `GET /api/users/{userId}/reservations`. Return a list of `ReservationResponseDto` mapped from the database. The query should include the associated `ParkingSlot` data and be ordered by `StartTime` descending. Ensure EF Core uses `.AsNoTracking()`.*

Response: Here is the implementation to retrieve a user's reservations. Since the frontend needs to display the parking slot designation on the "My Reservations" page, we will first add an optional `ParkingSlotDesignation` property to the `ReservationResponseDto`. Then, we'll create the new endpoint in the `UserController`.

### 1. Update the Reservation DTO

Update your `ReservationResponseDto` to include the parking slot designation. This allows the frontend to show "A1" or "B2" instead of just a raw GUID.

`src/ParkingBackend/Parking.WebAPI/DTOs/ReservationResponseDto.cs`

```
using System;

namespace Parking.WebAPI.DTOs
{
    public class ReservationResponseDto
    {
        public Guid Id { get; set; }
        public Guid ParkingSlotId { get; set; }
        public string ParkingSlotDesignation { get; set; } = string.Empty; // Added for UI display
        public Guid UserId { get; set; }
        public DateTime StartTime { get; set; }
        public DateTime EndTime { get; set; }
    }
}
```

### 2. Update the User Controller

Add the new `GET /api/users/{userId}/reservations` endpoint to the `UserController`. It will include the `ParkingSlot` reference, filter by the requested user, and sort the results by `StartTime` descending.

`src/ParkingBackend/Parking.WebAPI/Controllers/UserController.cs`

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
```

```

using Parking.WebAPI.DTOs;
using Parking.WebAPI.Models;

namespace Parking.WebAPI.Controllers
{
    [ApiController]
    [Route("api/users")]
    public class UserController : ControllerBase
    {
        private readonly ApplicationDbContext _context;

        public UserController(ApplicationDbContext context)
        {
            _context = context;
        }

        // GET: api/users
        [HttpGet]
        public async Task<ActionResult<IEnumerable<UserResponseDto>>> GetUsers()
        {
            var users = await _context.Users
                .AsNoTracking()
                .Select(u => new UserResponseDto
                {
                    Id = u.Id,
                    Name = u.Name,
                    AuthorizedSlotTypes = u.AuthorizedSlotTypes
                })
                .ToListAsync();

            return Ok(users);
        }

        // GET: api/users/{userId}/reservations
        [HttpGet("{userId:guid}/reservations")]
        public async Task<ActionResult<IEnumerable<ReservationResponseDto>>> GetUserReservations(Guid userId)
        {
            // First, verify the user exists
            var userExists = await _context.Users.AnyAsync(u => u.Id == userId);
            if (!userExists)
            {
                return NotFound(new { message = "User not found." });
            }

            // Fetch, map, and sort their reservations
            var reservations = await _context.Reservations
                .AsNoTracking()
                .Include(r => r.ParkingSlot) // Include to access the Designation
                .Where(r => r.UserId == userId)
                .OrderByDescending(r => r.StartTime)
                .Select(r => new ReservationResponseDto
                {
                    Id = r.Id,
                    ParkingSlotId = r.ParkingSlotId,
                    ParkingSlotDesignation = r.ParkingSlot.Designation, // Map the designation for the UI
                    UserId = r.UserId,
                    StartTime = r.StartTime,
                    EndTime = r.EndTime
                })
                .ToListAsync();

            return Ok(reservations);
        }
    }
}

```

(Note: Depending on how you implemented the `ParkingSlotController` and `ReservationService` earlier, you might need to make sure that wherever you return a `ReservationResponseDto`, you populate the `ParkingSlotDesignation` property there as well to avoid null/empty values on the Dashboard.)

---

*User prompt: Task: Automatically append the selected user ID to API requests via HTTP headers. As a frontend application, I want to send my simulated authentication state to the backend on every request. Acceptance Criteria: Update `src/api/axios.ts` to utilize an Axios interceptor. Before a request is sent, read the current `selectedUserId` (this may require exposing a getter from a store or adjusting how the Axios instance accesses the context). Inject the `X-User-Id` header into all outgoing requests.*

*Response: Here is the implementation to automatically inject the simulated authentication state (`X-User-Id` header) into every outgoing request. We will achieve this by adding an Axios request interceptor in your existing `axios.ts` file.*

Since the selected user ID is currently stored in React state (via `UserContext`), it's tricky to read it directly inside a standard TypeScript file outside of the React component tree. The simplest and most robust solution is to sync the `selectedUserId` to `localStorage` within the context, and have `Axios` read it from there.

## 1. Update the `UserContext` to use `LocalStorage`

Modify your `UserContext.tsx` to save the selected user ID to the browser's local storage whenever it changes.

**src/context/UserContext.tsx**

```
import { createContext, useState, type ReactNode, useContext, useEffect } from 'react';

interface UserContextType {
  selectedUserId: string;
  setSelectedUserId: (id: string) => void;
}

export const UserContext = createContext<UserContextType | undefined>(undefined);

export const UserProvider = ({ children }: { children: ReactNode }) => {
  // Initialize state from local storage if available
  const [selectedUserId, setSelectedUserId] = useState<string>(
    localStorage.getItem('selectedUserId') || ''
  );

  // Sync state to local storage whenever it changes
  useEffect(() => {
    if (selectedUserId) {
      localStorage.setItem('selectedUserId', selectedUserId);
    } else {
      localStorage.removeItem('selectedUserId');
    }
  }, [selectedUserId]);

  return (
    <UserContext.Provider value={{ selectedUserId, setSelectedUserId }}>
      {children}
    </UserContext.Provider>
  );
};

export const useUser = () => {
  const context = useContext(UserContext);
  if (!context) {
    throw new Error('useUser must be used within a UserProvider');
  }
  return context;
};
```

## 2. Update the `Axios` Configuration

Now modify your `axios.ts` file to add a request interceptor. This interceptor runs immediately before any request is sent, grabs the ID from local storage, and appends the header.

**src/api/axios.ts**

```
import axios from 'axios';

export const axiosInstance = axios.create({
  baseURL: 'http://localhost:5000',
  headers: {
    'Content-Type': 'application/json',
  },
});

// Add a request interceptor
axiosInstance.interceptors.request.use(
  (config) => {
    // Read the current user ID from local storage
    const userId = localStorage.getItem('selectedUserId');

    // If we have an ID, inject the custom header
    if (userId) {
      config.headers['X-User-Id'] = userId;
    }

    return config;
  },
  (error) => {
    // Handle request errors here
  }
);
```

```

    return Promise.reject(error);
  }
});

```

---

*User prompt: Task: Create a new page displaying a flat list of the active user's bookings. As a user, I want a dedicated page to see my own bookings, making it easy to manage my schedule. Acceptance Criteria: Create a useUserReservations query hook targeting the new GET /api/users/{userId}/reservations endpoint. Create a MyReservations.tsx page and map it to the /my-reservations route in App.tsx. Add a "My Reservations" link to the Navigation bar in Layout.tsx. Display the reservations, showing the Slot Designation, Start Time, and End Time. Reuse the useCancelReservation hook to allow users to cancel directly from this list (only for future reservations).*

**Response:** Here is the implementation for the "My Reservations" page. This creates the new query hook, builds the page component, and wires it up to your routing and layout.

## 1. Create the Query Hook

Create a new file named `useUserReservations.ts` inside your `/src/ParkingFrontend/src/query/` folder. This hook will fetch the reservations for the provided user ID.

**src/query/useUserReservations.ts**

```

import { useQuery } from '@tanstack/react-query';
import { axiosInstance } from '../api/axios';
import { ReservationResponseDto } from '../models/types';

export const useUserReservations = (userId: string) => {
  return useQuery<ReservationResponseDto[], Error>({
    queryKey: ['userReservations', userId],
    queryFn: async () => {
      const response = await axiosInstance.get<ReservationResponseDto[]>(`/api/users/${userId}/reservations`);
      return response.data;
    },
    // Don't run the query if there is no user selected
    enabled: !!userId,
  });
};

```

*(Note: You should also quickly update your useCancelReservation.ts and useCreateReservation.ts onSuccess callbacks to invalidate this new query key: queryClient.invalidateQueries({ queryKey: ['userReservations'] }); so the page updates automatically).*

## 2. Create the My Reservations Page

Create a new file named `MyReservations.tsx` inside your `/src/ParkingFrontend/src/pages/` folder.

**src/pages/MyReservations.tsx**

```

import { useUser } from '../context/UserContext';
import { useUserReservations } from '../query/useUserReservations';
import { useCancelReservation } from '../query/useCancelReservation';

export const MyReservations = () => {
  const { selectedUserId } = useUser();
  const { data: reservations, isLoading, isError, error } = useUserReservations(selectedUserId);
  const cancelReservation = useCancelReservation();

  if (!selectedUserId) {
    return <div style={{ textAlign: 'center', marginTop: '3rem' }}>Please select a user to view their reservations.</div>
  }

  if (isLoading) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#4b5563' }}><h2>Loading your r</h2></div>
  if (isError) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#ef4444' }}><h2>Failed to load d</h2></div>

  const handleCancel = (reservationId: string) => {
    if (window.confirm('Are you sure you want to cancel this reservation?')) {
      cancelReservation.mutate(reservationId, {
        onError: (err) => {
          alert(err.response?.data?.message || 'Failed to cancel reservation');
        }
      });
    }
  };

  return (
    <div>
      <h2 style={{ borderBottom: '2px solid #e5e7eb', paddingBottom: '0.5rem' }}>
        My Reservations
      </h2>

      {reservations?.length === 0 ? (

```

```

    <p style={{ color: '#6b7280', marginTop: '1.5rem' }}>You have no reservations.</p>
  ) : (
    <div style={{ display: 'flex', flexDirection: 'column', gap: '1rem', marginTop: '1.5rem' }}>
      {reservations?.map((res) => {
        const startLocal = new Date(res.startTime).toLocaleString();
        const endLocal = new Date(res.endTime).toLocaleString();

        // UI Logic: Only future reservations can be canceled
        const isFuture = new Date(res.startTime) > new Date();
        const isCancelingThis = cancelReservation.isPending && cancelReservation.variables === res.id;

        return (
          <div key={res.id} style={{
            display: 'flex', justifyContent: 'space-between', alignItems: 'center',
            border: '1px solid #d1d5db', borderRadius: '8px', padding: '1rem 1.5rem',
            backgroundColor: '#f9fafb', boxShadow: '0 1px 3px rgba(0,0,0,0.05)'
          }}>
            <div>
              <h3 style={{ margin: '0 0 0.5rem 0', color: '#1f2937' }}>
                Slot: {res.parkingSlotDesignation || res.parkingSlotId}
              </h3>
              <div style={{ fontSize: '0.875rem', color: '#374151' }}>
                <div style={{ marginBottom: '4px' }}><span style={{ fontWeight: 600, color: '#10b981' }}>Start:<
                  <div><span style={{ fontWeight: 600, color: '#ef4444' }}>End:</span> {endLocal}</div>
                </div>
              </div>
            </div>

            {isFuture && (
              <button
                onClick={() => handleCancel(res.id)}
                disabled={isCancelingThis}
                style={{
                  padding: '0.5rem 1rem',
                  backgroundColor: isCancelingThis ? '#f3f4f6' : '#fee2e2',
                  color: isCancelingThis ? '#9ca3af' : '#b91c1c',
                  border: `1px solid ${isCancelingThis ? '#d1d5db' : '#fca5a5'}`,
                  borderRadius: '4px',
                  cursor: isCancelingThis ? 'not-allowed' : 'pointer',
                  fontWeight: 600,
                  fontSize: '0.875rem',
                  transition: 'background-color 0.2s'
                }}
              >
                {isCancelingThis ? 'Canceling...' : 'Cancel'}
              </button>
            )}
          </div>
        );
      })}
    </div>
  )}
</div>
);
};

```

### 3. Update the Router

Map the new page to a route in your App.tsx.

src/App.tsx

```

import { Routes, Route } from 'react-router-dom';
import { Layout } from './components/Layout';
import { Dashboard } from './pages/Dashboard';
import { MyReservations } from './pages/MyReservations';
import { NotFound } from './pages/NotFound';
import { UserProvider } from './context/UserContext';

function App() {
  return (
    <UserProvider>
      <Routes>
        <Route path="/" element={<Layout />}>
          <Route index element={<Dashboard />} />
          { /* New route for My Reservations */ }
          <Route path="my-reservations" element={<MyReservations />} />
          <Route path="*" element={<NotFound />} />
        </Route>
      </Routes>
    </UserProvider>
  );
}

```

```

    </AuthProvider>
  );
}

```

```
export default App;
```

#### 4. Update the Navigation Bar

Add a link to the new route in the `Layout.tsx` file.

`src/components/Layout.tsx` (Inside the nav, update the links container)

```

<div style={{ display: 'flex', alignItems: 'center', gap: '2rem' }}>
  <h2 style={{ margin: 0, color: '#111827' }}>Parking Admin</h2>
  <div style={{ display: 'flex', gap: '1rem' }}>
    <Link to="/" style={{ textDecoration: 'none', color: '#374151', fontWeight: 'bold' }}>
      Dashboard
    </Link>
    <Link to="/my-reservations" style={{ textDecoration: 'none', color: '#374151', fontWeight: 'bold' }}>
      My Reservations
    </Link>
  </div>
</div>

```

---

*User prompt: `src/models/types.ts` should be updated. create the modified code to include the enum and newly added fields.*

Response: Here is the updated implementation for your TypeScript models. This includes the new `SlotType` enum and the corresponding properties added to your DTOs on the backend.

`src/models/types.ts`

```

export enum SlotType {
  Regular = 0,
  VIP = 1,
  Electric = 2,
  Accessible = 3
}

export interface UserResponseDto {
  id: string;
  name: string;
  authorizedSlotTypes: SlotType[];
}

export interface ReservationResponseDto {
  id: string;
  parkingSlotId: string;
  parkingSlotDesignation?: string; // Added to display slot names in "My Reservations"
  userId: string;
  startTime: string; // Serialized as ISO 8601 string from backend
  endTime: string; // Serialized as ISO 8601 string from backend
}

export interface ParkingSlotResponseDto {
  id: string;
  designation: string;
  type: SlotType;
  reservations: ReservationResponseDto[];
}

```

---

*User prompt: There is a lint issue: This syntax is not allowed when 'erasableSyntaxOnly' is enabled.*

Response: The issue is that TypeScript enums are not purely type-level constructs—they actually generate JavaScript code at runtime. Newer, strict TypeScript configurations (like `erasableSyntaxOnly` or bundlers using isolated modules) forbid TypeScript-specific features that emit runtime code.

To fix this, we can replace the enum with a standard JavaScript `const` object and derive an *erasable* TypeScript type from it. This gives you the exact same developer experience (you can still type `SlotType.VIP`) but complies with your strict build rules.

Here is the updated `types.ts` file:

`src/models/types.ts`

```

// 1. Define a standard JavaScript constant object
export const SlotType = {
  Regular: 0,
  VIP: 1,

```

```

    Electric: 2,
    Accessible: 3,
  } as const;

// 2. Extract an erasable type from the object's values (0 | 1 | 2 | 3)
export type SlotType = (typeof SlotType)[keyof typeof SlotType];

export interface UserResponseDto {
  id: string;
  name: string;
  authorizedSlotTypes: SlotType[];
}

export interface ReservationResponseDto {
  id: string;
  parkingSlotId: string;
  parkingSlotDesignation?: string;
  userId: string;
  startTime: string;
  endTime: string;
}

export interface ParkingSlotResponseDto {
  id: string;
  designation: string;
  type: SlotType;
  reservations: ReservationResponseDto[];
}

```

Because the const object is standard JavaScript and the type alias is completely stripped out during the build process, the `erasableSyntaxOnly` rule will now pass.

---

*User prompt: Task: Display slot types visually, add emojis to the user dropdown, and prevent unauthorized booking attempts. As a user, I want to clearly see which slots require special authorization and my own permissions. Acceptance Criteria: Update types.ts to include authorizedSlotTypes on the User, and type on the ParkingSlot. Layout.tsx: Append Unicode emojis to the user's name in the dropdown based on their authorizations (e.g., if they have VIP and Electric, show Name (👑⚡). Electric: ⚡ Accessible: ♿ VIP: 👑 Dashboard.tsx: Display matching emojis next to the slot designations on the cards. Visually disable the "Book Slot" button if the current user lacks the required authorization for that slot, and add a title tooltip explaining why.*

Response: Here is the implementation to visually represent the special slot types and enforce the authorization rules on the frontend. We will create a small utility file for the emoji mappings and update both the Layout and Dashboard components.

## 1. Create a Formatter Utility for Emojis

To keep our components clean and ensure the emojis are consistent across the app, let's create a small helper file.

Create a new file named `formatters.ts` inside a new (or existing) `/src/ParkingFrontend/src/utils/` folder.

**src/utils/formatters.ts**

```

import { SlotType } from '../models/types';

export const getSlotEmoji = (type: SlotType): string => {
  switch (type) {
    case SlotType.VIP: return '👑';
    case SlotType.Electric: return '⚡';
    case SlotType.Accessible: return '♿';
    default: return ''; // Regular slots have no special emoji
  }
};

export const getUserEmojis = (authorizedTypes: SlotType[]): string => {
  const emojis = authorizedTypes
    .map(getSlotEmoji)
    .filter(e => e !== '') // Remove empty strings (Regular)
    .join('');

  return emojis ? ` (${emojis})` : '';
};

```

## 2. Update the Layout Component

Update `Layout.tsx` to utilize the `getUserEmojis` helper, appending the badges to each user's name inside the dropdown option.

**src/components/Layout.tsx**

```

import { Outlet, Link } from 'react-router-dom';
import { useEffect } from 'react';

```



```

import { useUsers } from '../query/useUsers';
import { useUser } from '../context/UserContext';
import { getUserEmojis } from '../utils/formatters';

export const Layout = () => {
  const { data: users, isLoading, isError } = useUsers();
  const { selectedUserId, setSelectedUserId } = useUser();

  // Set the default selected user to the first user in the fetched list
  useEffect(() => {
    if (users && users.length > 0 && !selectedUserId) {
      setSelectedUserId(users[0].id);
    }
  }, [users, selectedUserId, setSelectedUserId]);

  return (
    <div style={{ minHeight: '100vh', display: 'flex', flexDirection: 'column' }}>
      <nav style={{
        display: 'flex',
        justifyContent: 'space-between',
        alignItems: 'center',
        padding: '1rem 2rem',
        backgroundColor: '#f3f4f6',
        borderBottom: '1px solid #e5e7eb'
      }}>
        <div style={{ display: 'flex', alignItems: 'center', gap: '2rem' }}>
          <h2 style={{ margin: 0, color: '#111827' }}>Parking Admin</h2>
          <div style={{ display: 'flex', gap: '1rem' }}>
            <Link to="/" style={{ textDecoration: 'none', color: '#374151', fontWeight: 'bold' }}>
              Dashboard
            </Link>
            <Link to="/my-reservations" style={{ textDecoration: 'none', color: '#374151', fontWeight: 'bold' }}>
              My Reservations
            </Link>
          </div>
        </div>
      </nav>

      <div style={{ display: 'flex', alignItems: 'center', gap: '1rem' }}>
        <label htmlFor="user-select" style={{ fontSize: '0.875rem', fontWeight: 600, color: '#374151' }}>
          Simulated User:
        </label>

        {isLoading ? (
          <span style={{ fontSize: '0.875rem', color: '#6b7280' }}>Loading users...</span>
        ) : isError ? (
          <span style={{ fontSize: '0.875rem', color: '#ef4444' }}>Failed to load users</span>
        ) : (
          <select
            id="user-select"
            value={selectedUserId}
            onChange={(e) => setSelectedUserId(e.target.value)}
            style={{
              padding: '0.5rem',
              borderRadius: '6px',
              border: '1px solid #d1d5db',
              backgroundColor: 'ffffff',
              color: '#111827',
              cursor: 'pointer'
            }}
          >
            {users?.map((user) => (
              <option key={user.id} value={user.id}>
                {user.name}{getUserEmojis(user.authorizedSlotTypes)}
              </option>
            ))}
          </select>
        )}
      </div>

      <main style={{ padding: '2rem', flex: 1, backgroundColor: 'ffffff' }}>
        <Outlet />
      </main>
    </div>
  );
};

```

### 3. Update the Dashboard Component

Update Dashboard.tsx to read the active user's permissions, append emojis to the slot headers, and disable the "Book Slot" button with a helpful tooltip if they lack the required clearance.

#### src/pages/Dashboard.tsx

```
import { useState } from 'react';
import { useParkingSlots } from '../query/useParkingSlots';
import { useUsers } from '../query/useUsers';
import { Modal } from '../components/Modal';
import { ReservationForm } from '../components/ReservationForm';
import { useUser } from '../context/UserContext';
import { useCancelReservation } from '../query/useCancelReservation';
import { SlotType } from '../models/types';
import { getSlotEmoji } from '../utils/formatters';

export const Dashboard = () => {
  const { data: slots, isLoading: isLoadingSlots, isError, error } = useParkingSlots();
  const { data: users } = useUsers(); // Fetch users to check active user's permissions
  const { selectedUserId } = useUser();
  const cancelReservation = useCancelReservation();

  const [isModalOpen, setIsModalOpen] = useState(false);
  const [selectedSlot, setSelectedSlot] = useState<{ id: string; designation: string } | null>(null);

  // Find the currently active user to verify their authorizations
  const activeUser = users?.find(u => u.id === selectedUserId);

  if (isLoadingSlots) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#4b5563' }}><h2>Loading p
  if (isError) return <div style={{ textAlign: 'center', marginTop: '3rem', color: '#ef4444' }}><h2>Failed to load d

  const handleOpenModal = (id: string, designation: string) => {
    setSelectedSlot({ id, designation });
    setIsModalOpen(true);
  };

  const handleCloseModal = () => {
    setIsModalOpen(false);
    setSelectedSlot(null);
  };

  const handleCancel = (reservationId: string) => {
    if (window.confirm('Are you sure you want to cancel this reservation?')) {
      cancelReservation.mutate(reservationId, {
        onError: (err) => {
          alert(err.response?.data?.message || 'Failed to cancel reservation');
        }
      });
    }
  };

  return (
    <div>
      <h2 style={{ borderBottom: '2px solid #e5e7eb', paddingBottom: '0.5rem' }}>
        Parking Slots Availability
      </h2>

      <div style={{
        display: 'grid', gridTemplateColumns: 'repeat(auto-fill, minmax(320px, 1fr))',
        gap: '1.5rem', marginTop: '1.5rem'
      }}>
        {slots?.map((slot) => {
          // UI Logic: Check if the user is authorized to book this slot type
          const isAuthorized = slot.type === SlotType.Regular || (activeUser?.authorizedSlotTypes.includes(slot.type)
          const slotEmoji = getSlotEmoji(slot.type);

          return (
            <div key={slot.id} style={{
              border: '1px solid #d1d5db', borderRadius: '8px', padding: '1.5rem',
              backgroundColor: '#f9fafb', boxShadow: '0 2px 4px rgba(0,0,0,0.05)'
            }}>
              <div style={{ display: 'flex', justifyContent: 'space-between', alignItems: 'center', marginBottom: '1'
                <h3 style={{ margin: 0, color: '#1f2937' }}>
                  Slot: {slot.designation} <span title="Slot Type">{slotEmoji}</span>
                </h3>

                <button
                  onClick={() => handleOpenModal(slot.id, slot.designation)}
                  disabled={!isAuthorized}
                  title={!isAuthorized ? 'You do not have the required authorization to book this slot.' : 'Book thi
                  style={{
```

```

padding: '0.5rem 1rem',
backgroundColor: isAuthorized ? '#10b981' : '#d1d5db',
color: isAuthorized ? 'white' : '#6b7280',
border: 'none', borderRadius: '4px',
cursor: isAuthorized ? 'pointer' : 'not-allowed',
fontWeight: 600, fontSize: '0.875rem'
}}
>
  Book Slot
</button>
</div>

{slot.reservations.length === 0 ? (
  <p style={{ color: '#6b7280', fontStyle: 'italic', margin: 0 }}>Available / No reservations.</p>
) : (
  <ul style={{ listStyleType: 'none', padding: 0, margin: 0, display: 'flex', flexDirection: 'column',
    {slot.reservations.map((res) => {
      const startLocal = new Date(res.startTime).toLocaleString();
      const endLocal = new Date(res.endTime).toLocaleString();

      const isFuture = new Date(res.startTime) > new Date();
      const isOwner = res.userId === selectedUserId;
      const canCancel = isFuture && isOwner;
      const isCancelingThis = cancelReservation.isPending && cancelReservation.variables === res.id;

      return (
        <li key={res.id} style={{
          backgroundColor: 'ffffff', border: '1px solid #e5e7eb', padding: '0.75rem',
          borderRadius: '6px', fontSize: '0.875rem', color: '#374151',
          display: 'flex', justifyContent: 'space-between', alignItems: 'center'
        }}>
          <div>
            <div style={{ marginBottom: '4px' }}><span style={{ fontWeight: 600, color: '#10b981' }}>S
            <div><span style={{ fontWeight: 600, color: '#ef4444' }}>End:</span> {endLocal}</div>
          </div>

          {canCancel && (
            <button
              onClick={() => handleCancel(res.id)}
              disabled={isCancelingThis}
              style={{
                padding: '0.4rem 0.75rem',
                backgroundColor: isCancelingThis ? '#f3f4f6' : '#fee2e2',
                color: isCancelingThis ? '#9ca3af' : '#b91c1c',
                border: `1px solid ${isCancelingThis ? '#d1d5db' : '#fca5a5'}`,
                borderRadius: '4px',
                cursor: isCancelingThis ? 'not-allowed' : 'pointer',
                fontWeight: 600,
                fontSize: '0.75rem',
                transition: 'background-color 0.2s'
              }}
            >
              {isCancelingThis ? 'Canceling...' : 'Cancel'}
            </button>
          )}
        </li>
      );
    )}
  </ul>
)}
</div>
);
}}
</div>

<Modal isOpen={isModalOpen} onClose={handleCloseModal}>
  {selectedSlot && (
    <ReservationForm
      parkingSlotId={selectedSlot.id}
      designation={selectedSlot.designation}
      onClose={handleCloseModal}
    />
  )}
</Modal>
</div>
);
};

```

*User prompt: Task: Fix the backend-to-frontend UTC serialization issue and improve the Reservation Form UX. As a user, I want to see the correct local time for reservations and have the date prefilled when booking. Acceptance Criteria: Timezone Fix: Update the backend DTO mapping (or JSON serialization settings) to explicitly format DateTime properties with the Z suffix (e.g., ToString("yyyy-MM-ddTHH:mm:ssZ")), ensuring the browser parses it as UTC. Form Refactor: Update ReservationForm.tsx to replace datetime-local inputs with one <input type="date"> and two <input type="time"> (Start Time and End Time). Autofill Date: Initialize the date input state to today's date (new Date().toISOString().split('T')[0]). Re-implement the client-side validation to combine the Date and Time inputs into standard Date objects before validating the 30-minute rule and past-time rule.*

Response: Here is the implementation to resolve the UTC timezone bug on the backend and vastly improve the UX of the reservation form on the frontend.

## 1. Fix the Backend UTC Serialization

Because Entity Framework retrieves DateTime from SQLite as DateTimeKind.Unspecified, standard JSON serialization does not append the Z (Zulu/UTC) indicator. We can fix this globally by creating a custom JSON converter and registering it.

**Create** src/ParkingBackend/Parking.WebAPI/Converters/UtcDateTimeConverter.cs

```
using System;
using System.Text.Json;
using System.Text.Json.Serialization;

namespace Parking.WebAPI.Converters
{
    public class UtcDateTimeConverter : JsonConverter
    {
        public override DateTime Read(ref Utf8JsonReader reader, Type typeToConvert, JsonSerializerOptions options)
        {
            return DateTime.Parse(reader.GetString() ?? string.Empty);
        }

        public override void Write(Utf8JsonWriter writer, DateTime value, JsonSerializerOptions options)
        {
            // Explicitly format with the "Z" suffix so the browser always interprets it as UTC
            writer.WriteStringValue(value.ToString("yyyy-MM-ddTHH:mm:ssZ"));
        }
    }
}
```

**Update** src/ParkingBackend/Parking.WebAPI/Program.cs Add the converter to your controller's JSON options.

```
using Parking.WebAPI.Converters; // Ensure this is imported

// Find your builder.Services.AddControllers() line and modify it:
builder.Services.AddControllers()
    .AddJsonOptions(options =>
    {
        // Add the custom converter for DateTime
        options.JsonSerializerOptions.Converters.Add(new UtcDateTimeConverter());
    });
```

## 2. Refactor the Frontend Reservation Form

We will update ReservationForm.tsx to utilize three separate inputs. This allows us to autofill the date, making it much faster for users to book a time today.

**src/components/ReservationForm.tsx**

```
import { useState, useEffect } from 'react';
import { useUser } from '../context/UserContext';
import { useCreateReservation } from '../query/useCreateReservation';

interface ReservationFormProps {
    parkingSlotId: string;
    designation: string;
    onClose: () => void;
}

export const ReservationForm = ({ parkingSlotId, designation, onClose }: ReservationFormProps) => {
    // Initialize date to today using the YYYY-MM-DD format
    const [date, setDate] = useState(new Date().toISOString().split('T')[0]);
    const [startTime, setStartTime] = useState('');
    const [endTime, setEndTime] = useState('');
    const [validationError, setValidationError] = useState('');
    const [apiError, setApiError] = useState('');

    const { selectedUserId } = useUser();
    const createReservation = useCreateReservation();

    // Re-run validation whenever date, start time, or end time changes
    useEffect(() => {
```

```

setApiError(''); // Clear API error when inputs change

if (!date || !startTime || !endTime) {
  setValidationError('');
  return;
}

// Safely combine the separate date and time strings into valid Date objects
const start = new Date(`${date}T${startTime}`);
const end = new Date(`${date}T${endTime}`);
const now = new Date();

if (start < now) {
  setValidationError('Start time cannot be in the past.');
```

```

</div>

<div style={{ display: 'flex', gap: '1rem' }}>
  <div style={{ display: 'flex', flexDirection: 'column', gap: '0.5rem', flex: 1 }}>
    <label htmlFor="startTime" style={{ fontSize: '0.875rem', fontWeight: 600 }}>Start Time</label>
    <input
      type="time"
      id="startTime"
      value={startTime}
      onChange={(e) => setStartTime(e.target.value)}
      style={{ padding: '0.5rem', borderRadius: '4px', border: '1px solid #d1d5db', width: '100%', boxSizing
        required
        disabled={createReservation.isPending}
      />
    </div>
  </div>

  <div style={{ display: 'flex', flexDirection: 'column', gap: '0.5rem', flex: 1 }}>
    <label htmlFor="endTime" style={{ fontSize: '0.875rem', fontWeight: 600 }}>End Time</label>
    <input
      type="time"
      id="endTime"
      value={endTime}
      onChange={(e) => setEndTime(e.target.value)}
      style={{ padding: '0.5rem', borderRadius: '4px', border: '1px solid #d1d5db', width: '100%', boxSizing
        required
        disabled={createReservation.isPending}
      />
    </div>
  </div>

  </div>

  <div style={{ display: 'flex', justify-content: 'flex-end', gap: '1rem', marginTop: '1rem' }}>
    <button
      type="button"
      onClick={onClose}
      disabled={createReservation.isPending}
      style={{
        padding: '0.5rem 1rem', borderRadius: '4px', border: '1px solid #d1d5db',
        backgroundColor: 'ffffff', cursor: createReservation.isPending ? 'not-allowed' : 'pointer'
      }}
    >
      Cancel
    </button>
    <button
      type="submit"
      disabled={!isFormValid}
      style={{
        padding: '0.5rem 1rem', borderRadius: '4px', border: 'none',
        backgroundColor: isFormValid ? '#3b82f6' : '#9ca3af',
        color: 'ffffff', cursor: isFormValid ? 'pointer' : 'not-allowed'
      }}
    >
      {createReservation.isPending ? 'Submitting...' : 'Submit'}
    </button>
  </div>
</form>
</div>
);
};

```

User prompt: Last task is to adjust the test cases to match the newly added functionalities.

Response: Here are the updated unit tests for the backend to ensure they cover the new SlotType authorizations and the X-User-Id ownership checks for cancellations.

## 1. Update the Reservation Service Tests

We need to update existing tests to pass the `requestingUserId` when canceling, and add new test cases to verify that unauthorized cancellations and unauthorized slot bookings throw the correct exceptions.

`tests/ParkingBackend.Tests/Services/ReservationServiceTests.cs`

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOS;
using Parking.WebAPI.Models;
using Parking.WebAPI.Services;
using Xunit;

namespace ParkingBackend.Tests.Services
{
    public class ReservationServiceTests
    {
        private ApplicationDbContext GetInMemoryDbContext()
        {
            var options = new DbContextOptionsBuilder<ApplicationDbContext>()
                .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
                .Options;

            return new ApplicationDbContext(options);
        }

        [Fact]
        public async Task CreateReservation_ValidRequest_CreatesReservation()
        {
            // Arrange
            var context = GetInMemoryDbContext();
            var userId = Guid.NewGuid();
            var slotId = Guid.NewGuid();

            context.Users.Add(new User {
                Id = userId,
                Name = "Test User",
                AuthorizedSlotTypes = new List<SlotType> { SlotType.Regular }
            });
            context.ParkingSlots.Add(new ParkingSlot {
                Id = slotId,
                Designation = "A1",
                Type = SlotType.Regular
            });
            await context.SaveChangesAsync();

            var service = new ReservationService(context);
            var request = new ReservationRequestDto
            {
                UserId = userId,
                ParkingSlotId = slotId,
                StartTime = DateTime.UtcNow.AddHours(1),
                EndTime = DateTime.UtcNow.AddHours(2)
            };

            // Act
            var result = await service.CreateReservationAsync(request);

            // Assert
            Assert.NotNull(result);
            Assert.Equal(userId, result.UserId);
            Assert.Equal(slotId, result.ParkingSlotId);
        }

        [Fact]
        public async Task CreateReservation_UnauthorizedSlotType_ThrowsInvalidOperationException()
        {
            // Arrange
            var context = GetInMemoryDbContext();
            var userId = Guid.NewGuid();
            var slotId = Guid.NewGuid();

            // User only has Regular access
            context.Users.Add(new User { Id = userId, Name = "Test User", AuthorizedSlotTypes = new List<SlotType> {
```

```

// Slot is VIP
context.ParkingSlots.Add(new ParkingSlot { Id = slotId, Designation = "VIP1", Type = SlotType.VIP });
await context.SaveChangesAsync();

var service = new ReservationService(context);
var request = new ReservationRequestDto
{
    UserId = userId,
    ParkingSlotId = slotId,
    StartTime = DateTime.UtcNow.AddHours(1),
    EndTime = DateTime.UtcNow.AddHours(2)
};

// Act & Assert
var ex = await Assert.ThrowsAsync<InvalidOperationException>(() => service.CreateReservationAsync(request));
Assert.Contains("not authorized", ex.Message);
}

[Fact]
public async Task CancelReservation_ValidAndOwned_CancelsReservation()
{
    // Arrange
    var context = GetInMemoryDbContext();
    var reservationId = Guid.NewGuid();
    var userId = Guid.NewGuid();

    var reservation = new Reservation
    {
        Id = reservationId,
        UserId = userId,
        ParkingSlotId = Guid.NewGuid(),
        StartTime = DateTime.UtcNow.AddHours(2),
        EndTime = DateTime.UtcNow.AddHours(3)
    };

    context.Reservations.Add(reservation);
    await context.SaveChangesAsync();

    var service = new ReservationService(context);

    // Act - Passed matching userId
    await service.CancelReservationAsync(reservationId, userId);

    // Assert
    var deletedReservation = await context.Reservations.FindAsync(reservationId);
    Assert.Null(deletedReservation);
}

[Fact]
public async Task CancelReservation_WrongUser_ThrowsUnauthorizedAccessException()
{
    // Arrange
    var context = GetInMemoryDbContext();
    var reservationId = Guid.NewGuid();
    var ownerId = Guid.NewGuid();
    var hackerId = Guid.NewGuid();

    var reservation = new Reservation
    {
        Id = reservationId,
        UserId = ownerId, // Owned by Owner
        ParkingSlotId = Guid.NewGuid(),
        StartTime = DateTime.UtcNow.AddHours(2),
        EndTime = DateTime.UtcNow.AddHours(3)
    };

    context.Reservations.Add(reservation);
    await context.SaveChangesAsync();

    var service = new ReservationService(context);

    // Act & Assert - Attempting to cancel with Hacker's ID
    var ex = await Assert.ThrowsAsync<UnauthorizedAccessException>(() => service.CancelReservationAsync(reservationId, hackerId));
    Assert.Contains("not authorized", ex.Message);
}
}
}
}

```

## 2. Update the Reservation Controller Tests



We need to update the mock setup to account for the new `requestingUserId` parameter in `CancelReservation`, and write a test ensuring that an `UnauthorizedAccessException` properly returns a 403 Forbidden response.

#### tests/ParkingBackend.Tests/Controllers/ReservationControllerTests.cs

```
using System;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Moq;
using Parking.WebAPI.Controllers;
using Parking.WebAPI.DTOs;
using Parking.WebAPI.Services;
using Xunit;

namespace ParkingBackend.Tests.Controllers
{
    public class ReservationControllerTests
    {
        [Fact]
        public async Task CancelReservation_ValidRequest_ReturnsNoContent()
        {
            // Arrange
            var mockService = new Mock<IReservationService>();
            var reservationId = Guid.NewGuid();
            var requestingUserId = Guid.NewGuid();

            // The service now requires the user ID
            mockService.Setup(s => s.CancelReservationAsync(reservationId, requestingUserId))
                .Returns(Task.CompletedTask);

            var controller = new ReservationController(mockService.Object);

            // Act
            var result = await controller.CancelReservation(reservationId, requestingUserId);

            // Assert
            Assert.IsType<NoContentResult>(result);
        }

        [Fact]
        public async Task CancelReservation_UnauthorizedUser_ReturnsForbidden()
        {
            // Arrange
            var mockService = new Mock<IReservationService>();
            var reservationId = Guid.NewGuid();
            var requestingUserId = Guid.NewGuid();

            // Mock the service to throw UnauthorizedAccessException
            mockService.Setup(s => s.CancelReservationAsync(reservationId, requestingUserId))
                .ThrowsAsync(new UnauthorizedAccessException("You are not authorized to cancel this reservati

            var controller = new ReservationController(mockService.Object);

            // Act
            var result = await controller.CancelReservation(reservationId, requestingUserId);

            // Assert
            var objectResult = Assert.IsType<ObjectResult>(result);
            Assert.Equal(403, objectResult.StatusCode);

            // Optional: Verify the message property in the response object
            var value = objectResult.Value;
            var messageProperty = value.GetType().GetProperty("message");
            Assert.NotNull(messageProperty);
            Assert.Equal("You are not authorized to cancel this reservation.", messageProperty.GetValue(value, null)
        }

        [Fact]
        public async Task CancelReservation_MissingHeader_ReturnsBadRequest()
        {
            // Arrange
            var mockService = new Mock<IReservationService>();
            var controller = new ReservationController(mockService.Object);
            var reservationId = Guid.NewGuid();

            // Act - Send an empty Guid to simulate missing/invalid header
            var result = await controller.CancelReservation(reservationId, Guid.Empty);

            // Assert
            var badRequestResult = Assert.IsType<BadRequestObjectResult>(result);
        }
    }
}
```

```

        Assert.Equal(400, badRequestResult.StatusCode);
    }
}
}

```

User prompt: This was the original code. Rewrite this, with the technologies used originally. using System;using System.Linq;using System.Net;using System.Net.Http.Json;using System.Threading.Tasks;using FluentAssertions;using Microsoft.AspNetCore.Mvc.Testing;using Microsoft.Extensions.DependencyInjection;using Parking.WebAPI.Data;using Parking.WebAPI.DTOS;using Xunit;namespace Parking.WebAPI.IntegrationTests { public class ReservationControllerTests : IClassFixture<WebApplicationFactory<Program>> { private readonly WebApplicationFactory<Program> \_factory; public ReservationControllerTests(WebApplicationFactory<Program> factory) { \_factory = factory; } // Helper method to retrieve valid seeded IDs directly from the database context private (Guid UserId, Guid SlotId) GetValidIds() { using var scope = \_factory.Services.CreateScope(); var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>(); var userId = dbContext.Users.First().Id; var slotId = dbContext.ParkingSlots.First().Id; return (userId, slotId); } [Fact] public async Task PostReservation\_WithValidData\_ReturnsCreated() { // Arrange var client = \_factory.CreateClient(); var (userId, slotId) = GetValidIds(); var request = new ReservationRequestDTO { ParkingSlotId = slotId, UserId = userId, // Use a far future date to avoid overlapping with other parallel tests StartTime = DateTime.UtcNow.AddDays(10), EndTime = DateTime.UtcNow.AddDays(10).AddHours(2) }; // Act var response = await client.PostAsJsonAsync("/api/reservations", request); // Assert response.StatusCode.Should().Be(HttpStatusCode.Created); var createdReservation = await response.Content.ReadFromJsonAsync<ReservationResponseDTO>(); createdReservation.Should().NotBeNull(); createdReservation!.ParkingSlotId.Should().Be(slotId); createdReservation.UserId.Should().Be(userId); } [Fact] public async Task PostReservation\_WithOverlappingTime\_ReturnsBadRequest() { // Arrange var client = \_factory.CreateClient(); var (userId, slotId) = GetValidIds(); var request = new ReservationRequestDTO { ParkingSlotId = slotId, UserId = userId, StartTime = DateTime.UtcNow.AddDays(11), EndTime = DateTime.UtcNow.AddDays(11).AddHours(2) }; // Act // 1. Create the initial valid reservation await client.PostAsJsonAsync("/api/reservations", request); // 2. Try to create a second reservation for the exact same time and slot var overlappingResponse = await client.PostAsJsonAsync("/api/reservations", request); // Assert overlappingResponse.StatusCode.Should().Be(HttpStatusCode.BadRequest); var errorContent = await overlappingResponse.Content.ReadAsStringAsync(); errorContent.Should().Contain("booked"); } [Fact] public async Task DeleteReservation\_WithValidFutureId\_ReturnsNoContent() { // Arrange var client = \_factory.CreateClient(); var (userId, slotId) = GetValidIds(); var request = new ReservationRequestDTO { ParkingSlotId = slotId, UserId = userId, StartTime = DateTime.UtcNow.AddDays(12), EndTime = DateTime.UtcNow.AddDays(12).AddHours(2) }; // First, create a reservation to ensure we have a valid ID to delete var createResponse = await client.PostAsJsonAsync("/api/reservations", request); var createdReservation = await createResponse.Content.ReadFromJsonAsync<ReservationResponseDTO>(); // Act var deleteResponse = await client.DeleteAsync(\$"/api/reservations/{createdReservation!.Id}"); // Assert deleteResponse.StatusCode.Should().Be(HttpStatusCode.NoContent); } }

Response: Here is the rewritten integration test class.

It preserves your original technology stack (xUnit, FluentAssertions, WebApplicationFactory) while expanding the test coverage to enforce the new Milestone 5 business rules. Specifically, it now handles the custom X-User-Id HTTP header for cancellations and adds dedicated tests to verify SlotType authorizations and cancellation ownership security.

```

using System;
using System.Linq;
using System.Net;
using System.Net.Http;
using System.Net.Http.Json;
using System.Threading.Tasks;
using FluentAssertions;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.Extensions.DependencyInjection;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOS;
using Parking.WebAPI.Models;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class ReservationControllerTests : IClassFixture<WebApplicationFactory<Program>>
    {
        private readonly WebApplicationFactory<Program> _factory;

        public ReservationControllerTests(WebApplicationFactory<Program> factory)
        {
            _factory = factory;
        }

        // Helper method to retrieve valid seeded IDs directly from the database context
        // Updated to explicitly pull a 'Regular' slot so basic tests don't fail due to new SlotType restrictions
        private (Guid UserId, Guid SlotId) GetValidIds()
        {
            using var scope = _factory.Services.CreateScope();
            var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();

            var userId = dbContext.Users.First().Id;
            var slotId = dbContext.ParkingSlots.First(s => s.Type == SlotType.Regular).Id;

            return (userId, slotId);
        }

        [Fact]
    }
}

```

```

public async Task PostReservation_WithValidData_ReturnsCreated()
{
    // Arrange
    var client = _factory.CreateClient();
    var (userId, slotId) = GetValidIds();

    var request = new ReservationRequestDto
    {
        ParkingSlotId = slotId,
        UserId = userId,
        // Use a far future date to avoid overlapping with other parallel tests
        StartTime = DateTime.UtcNow.AddDays(10),
        EndTime = DateTime.UtcNow.AddDays(10).AddHours(2)
    };

    // Act
    var response = await client.PostAsJsonAsync("/api/reservations", request);

    // Assert
    // Note: Assuming your controller returns 201 Created on success
    response.StatusCode.Should().Be(HttpStatusCode.Created);

    var createdReservation = await response.Content.ReadFromJsonAsync<ReservationResponseDto>();
    createdReservation.Should().NotBeNull();
    createdReservation!.ParkingSlotId.Should().Be(slotId);
    createdReservation.UserId.Should().Be(userId);
}

[Fact]
public async Task PostReservation_WithUnauthorizedSlotType_ReturnsBadRequest()
{
    // Arrange
    var client = _factory.CreateClient();
    using var scope = _factory.Services.CreateScope();
    var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();

    // Find a user with only Regular access, and a VIP slot
    var regularUser = dbContext.Users.First(u => !u.AuthorizedSlotTypes.Contains(SlotType.VIP));
    var vipSlot = dbContext.ParkingSlots.First(s => s.Type == SlotType.VIP);

    var request = new ReservationRequestDto
    {
        ParkingSlotId = vipSlot.Id,
        UserId = regularUser.Id,
        StartTime = DateTime.UtcNow.AddDays(15),
        EndTime = DateTime.UtcNow.AddDays(15).AddHours(2)
    };

    // Act
    var response = await client.PostAsJsonAsync("/api/reservations", request);

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.BadRequest);

    var errorContent = await response.Content.ReadAsStringAsync();
    errorContent.Should().Contain("not authorized");
}

[Fact]
public async Task PostReservation_WithOverlappingTime_ReturnsBadRequest()
{
    // Arrange
    var client = _factory.CreateClient();
    var (userId, slotId) = GetValidIds();

    var request = new ReservationRequestDto
    {
        ParkingSlotId = slotId,
        UserId = userId,
        StartTime = DateTime.UtcNow.AddDays(11),
        EndTime = DateTime.UtcNow.AddDays(11).AddHours(2)
    };

    // Act
    // 1. Create the initial valid reservation
    await client.PostAsJsonAsync("/api/reservations", request);

    // 2. Try to create a second reservation for the exact same time and slot
    var overlappingResponse = await client.PostAsJsonAsync("/api/reservations", request);
}

```

```

        // Assert
        overlappingResponse.StatusCode.Should().Be(HttpStatusCode.BadRequest);

        var errorContent = await overlappingResponse.Content.ReadAsStringAsync();
        errorContent.Should().Contain("booked"); // Or whatever your overlap error message is
    }

    [Fact]
    public async Task DeleteReservation_WithValidFutureId_ReturnsNoContent()
    {
        // Arrange
        var client = _factory.CreateClient();
        var (userId, slotId) = GetValidIds();

        var request = new ReservationRequestDto
        {
            ParkingSlotId = slotId,
            UserId = userId,
            StartTime = DateTime.UtcNow.AddDays(12),
            EndTime = DateTime.UtcNow.AddDays(12).AddHours(2)
        };

        var createResponse = await client.PostAsJsonAsync("/api/reservations", request);
        var createdReservation = await createResponse.Content.ReadFromJsonAsync<ReservationResponseDto>();

        // Act
        // We must use HttpRequestMessage to inject the custom X-User-Id header
        var requestMessage = new HttpRequestMessage(HttpMethod.Delete, $"/api/reservations/{createdReservation!.Id}");
        requestMessage.Headers.Add("X-User-Id", userId.ToString());

        var deleteResponse = await client.SendAsync(requestMessage);

        // Assert
        deleteResponse.StatusCode.Should().Be(HttpStatusCode.NoContent);
    }

    [Fact]
    public async Task DeleteReservation_WithUnauthorizedUser_ReturnsForbidden()
    {
        // Arrange
        var client = _factory.CreateClient();
        var (ownerId, slotId) = GetValidIds();
        var hackerId = Guid.NewGuid(); // A different, unauthorized user ID

        var request = new ReservationRequestDto
        {
            ParkingSlotId = slotId,
            UserId = ownerId,
            StartTime = DateTime.UtcNow.AddDays(13),
            EndTime = DateTime.UtcNow.AddDays(13).AddHours(2)
        };

        var createResponse = await client.PostAsJsonAsync("/api/reservations", request);
        var createdReservation = await createResponse.Content.ReadFromJsonAsync<ReservationResponseDto>();

        // Act
        // Attempt deletion using the hacker's ID in the header
        var requestMessage = new HttpRequestMessage(HttpMethod.Delete, $"/api/reservations/{createdReservation!.Id}");
        requestMessage.Headers.Add("X-User-Id", hackerId.ToString());

        var deleteResponse = await client.SendAsync(requestMessage);

        // Assert
        deleteResponse.StatusCode.Should().Be(HttpStatusCode.Forbidden);

        var errorContent = await deleteResponse.Content.ReadAsStringAsync();
        errorContent.Should().Contain("not authorized");
    }
}
}
}

```

---

*User prompt: Now this needs refinement, as the CancelReservationAsync requires the requesting user ID. using System;using System.Threading.Tasks;using FluentAssertions;using Microsoft.Data.Sqlite;using Microsoft.EntityFrameworkCore;using Parking.WebAPI.Data;using Parking.WebAPI.DTOs;using Parking.WebAPI.Models;using Parking.WebAPI.Services;using Xunit;namespace Parking.WebAPI.IntegrationTests { public class ReservationServiceTests : IDisposable { private readonly SqliteConnection \_connection; private readonly DbContextOptions<ApplicationDbContext> \_options; private readonly ApplicationDbContext \_context; private readonly ReservationService \_service; public ReservationServiceTests() { \_connection = new SqliteConnection("DataSource=:memory:");*

```

_connection.Open(); _options = new DbContextOptionsBuilder<ApplicationDbContext>().UseSqlite(_connection).Options; _context = new
ApplicationDbContext(_options); _context.Database.Migrate(); // Ensure schema is created _service = new ReservationService(_context); }
[Fact] public async Task CreateReservation_WhenStartTimesInThePast_ShouldThrowArgumentException() { // Arrange var request = new
ReservationRequestDTO { ParkingSlotId = Guid.NewGuid(), UserId = Guid.NewGuid(), StartTime = DateTime.UtcNow.AddHours(-1), // Past
EndTime = DateTime.UtcNow.AddHours(1) }; // Act Func<Task> act = async () => await _service.CreateReservationAsync(request); // Assert
await act.Should().ThrowAsync<ArgumentException>().WithMessage("Reservation start time cannot be in the past."); } [Fact] public async Task
CreateReservation_WhenDurationIsLessThan30Minutes_ShouldThrowArgumentException() { // Arrange var startTime =
DateTime.UtcNow.AddHours(1); var request = new ReservationRequestDTO { ParkingSlotId = Guid.NewGuid(), UserId = Guid.NewGuid(),
StartTime = startTime, EndTime = startTime.AddMinutes(29) // Less than 30 mins }; // Act Func<Task> act = async () => await
_service.CreateReservationAsync(request); // Assert await act.Should().ThrowAsync<ArgumentException>().WithMessage("Reservation
duration must be at least 30 minutes."); } [Fact] public async Task CreateReservation_WithAdjacentReservations_ShouldNotTriggerOverlap() { //
Arrange var userId = Guid.NewGuid(); var slotId = Guid.NewGuid(); await SeedUserAndSlotAsync(userId, slotId); var baseTime =
DateTime.UtcNow.AddDays(1); // Existing reservation: 10:00 to 11:00 _context.Reservations.Add(new Reservation { Id = Guid.NewGuid(),
UserId = userId, ParkingSlotId = slotId, StartTime = baseTime, EndTime = baseTime.AddHours(1) }); await _context.SaveChangesAsync(); var
request = new ReservationRequestDTO { ParkingSlotId = slotId, UserId = userId, StartTime = baseTime.AddHours(1), // Starts exactly when the
other ends (11:00) EndTime = baseTime.AddHours(2) // Ends at 12:00 }; // Act var result = await _service.CreateReservationAsync(request); //
Assert result.Should().NotBeNull(); result.StartTime.Should().Be(request.StartTime); var count = await _context.Reservations.CountAsync();
count.Should().Be(2, "because the adjacent reservation should be saved alongside the original"); } [Fact] public async Task
CreateReservation_WithPartiallyOverlappingReservation_ShouldThrowInvalidOperationException() { // Arrange var userId = Guid.NewGuid();
var slotId = Guid.NewGuid(); await SeedUserAndSlotAsync(userId, slotId); var baseTime = DateTime.UtcNow.AddDays(1); // Existing
reservation: 10:00 to 11:00 _context.Reservations.Add(new Reservation { Id = Guid.NewGuid(), UserId = userId, ParkingSlotId = slotId,
StartTime = baseTime, EndTime = baseTime.AddHours(1) }); await _context.SaveChangesAsync(); var request = new ReservationRequestDTO
{ ParkingSlotId = slotId, UserId = userId, StartTime = baseTime.AddMinutes(30), // Starts at 10:30 (Overlap) EndTime =
baseTime.AddMinutes(90) // Ends at 11:30 }; // Act Func<Task> act = async () => await _service.CreateReservationAsync(request); // Assert
await act.Should().ThrowAsync<InvalidOperationException>().WithMessage("The parking slot is already booked for the requested time
period."); } [Fact] public async Task CreateReservation_WithValidData_ShouldSaveToDatabaseAndReturnDto() { // Arrange var userId =
Guid.NewGuid(); var slotId = Guid.NewGuid(); await SeedUserAndSlotAsync(userId, slotId); var request = new ReservationRequestDTO {
ParkingSlotId = slotId, UserId = userId, StartTime = DateTime.UtcNow.AddHours(2), EndTime = DateTime.UtcNow.AddHours(4) }; // Act var
response = await _service.CreateReservationAsync(request); // Assert response.Should().NotBeNull();
response.ParkingSlotId.Should().Be(slotId); response.UserId.Should().Be(userId); var dbEntity = await
_context.Reservations.FirstOrDefaultAsync(r => r.Id == response.Id); dbEntity.Should().NotBeNull();
dbEntity!.StartTime.Should().Be(request.StartTime); dbEntity.EndTime.Should().Be(request.EndTime); } private async Task
SeedUserAndSlotAsync(Guid userId, Guid slotId) { _context.Users.Add(new User { Id = userId, Name = "Test User" });
_context.ParkingSlots.Add(new ParkingSlot { Id = slotId, Designation = "T01" }); await _context.SaveChangesAsync(); } [Fact] public async Task
CancelReservation_WhenReservationDoesNotExist_ShouldThrowKeyNotFoundException() { // Arrange var nonExistentReservationId =
Guid.NewGuid(); // Act Func<Task> act = async () => await _service.CancelReservationAsync(nonExistentReservationId); // Assert await
act.Should().ThrowAsync<KeyNotFoundException>().WithMessage($"Reservation with ID {nonExistentReservationId} was not found."); } [Fact]
public async Task CancelReservation_WhenStartTimesInThePast_ShouldThrowInvalidOperationException() { // Arrange var userId =
Guid.NewGuid(); var slotId = Guid.NewGuid(); var reservationId = Guid.NewGuid(); await SeedUserAndSlotAsync(userId, slotId); // Add a
reservation that has already happened _context.Reservations.Add(new Reservation { Id = reservationId, UserId = userId, ParkingSlotId = slotId,
StartTime = DateTime.UtcNow.AddHours(-2), // Started 2 hours ago EndTime = DateTime.UtcNow.AddHours(-1) }); await
_context.SaveChangesAsync(); // Act Func<Task> act = async () => await _service.CancelReservationAsync(reservationId); // Assert await
act.Should().ThrowAsync<InvalidOperationException>().WithMessage("Cannot cancel a reservation that has already started or finished."); }
[Fact] public async Task CancelReservation_WhenStartTimesInTheFuture_ShouldDeleteFromDatabase() { // Arrange var userId =
Guid.NewGuid(); var slotId = Guid.NewGuid(); var reservationId = Guid.NewGuid(); await SeedUserAndSlotAsync(userId, slotId); // Add a
reservation in the future _context.Reservations.Add(new Reservation { Id = reservationId, UserId = userId, ParkingSlotId = slotId, StartTime =
DateTime.UtcNow.AddHours(2), // Starts in 2 hours EndTime = DateTime.UtcNow.AddHours(3) }); await _context.SaveChangesAsync(); // Act
await _service.CancelReservationAsync(reservationId); // Assert var deletedReservation = await
_context.Reservations.FindAsync(reservationId); deletedReservation.Should().BeNull("because the valid future reservation should have been
deleted from the database"); } public void Dispose() { _context.Dispose(); _connection.Close(); _connection.Dispose(); } }

```

Response: Here is the rewritten ReservationServiceTests class. It maintains your original testing framework (xUnit, FluentAssertions, SQLite in-memory) while integrating the new requestingUserId parameter for cancellations, the new SlotType logic, and the corresponding ownership authorization checks.

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using FluentAssertions;
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Parking.WebAPI.Data;
using Parking.WebAPI.DTOs;
using Parking.WebAPI.Models;
using Parking.WebAPI.Services;
using Xunit;

namespace Parking.WebAPI.IntegrationTests
{
    public class ReservationServiceTests : IDisposable
    {
        private readonly SqliteConnection _connection;
        private readonly DbContextOptions<ApplicationDbContext> _options;
        private readonly ApplicationDbContext _context;
        private readonly ReservationService _service;

        public ReservationServiceTests()
        {
            _connection = new SqliteConnection("DataSource=:memory:");
            _connection.Open();

            _options = new DbContextOptionsBuilder<ApplicationDbContext>()

```

```

        .UseSqlite(_connection)
        .Options;

        _context = new ApplicationDbContext(_options);
        _context.Database.Migrate(); // Ensure schema is created

        _service = new ReservationService(_context);
    }

// Helper updated to include SlotType and AuthorizedSlotTypes
private async Task SeedUserAndSlotAsync(Guid userId, Guid slotId, SlotType slotType = SlotType.Regular, List
{
    authorizedTypes ??= new List<SlotType> { SlotType.Regular };

    _context.Users.Add(new User
    {
        Id = userId,
        Name = "Test User",
        AuthorizedSlotTypes = authorizedTypes
    });

    _context.ParkingSlots.Add(new ParkingSlot
    {
        Id = slotId,
        Designation = "T01",
        Type = slotType
    });

    await _context.SaveChangesAsync();
}

[Fact]
public async Task CreateReservation_WhenStartTimeIsInThePast_ShouldThrowArgumentException()
{
    // Arrange
    var request = new ReservationRequestDTO
    {
        ParkingSlotId = Guid.NewGuid(),
        UserId = Guid.NewGuid(),
        StartTime = DateTime.UtcNow.AddHours(-1), // Past
        EndTime = DateTime.UtcNow.AddHours(1)
    };

    // Act
    Func<Task> act = async () => await _service.CreateReservationAsync(request);

    // Assert
    await act.Should().ThrowAsync<ArgumentException>()
        .WithMessage("Reservation start time cannot be in the past.");
}

[Fact]
public async Task CreateReservation_WhenDurationIsLessThan30Minutes_ShouldThrowArgumentException()
{
    // Arrange
    var startTime = DateTime.UtcNow.AddHours(1);
    var request = new ReservationRequestDTO
    {
        ParkingSlotId = Guid.NewGuid(),
        UserId = Guid.NewGuid(),
        StartTime = startTime,
        EndTime = startTime.AddMinutes(29) // Less than 30 mins
    };

    // Act
    Func<Task> act = async () => await _service.CreateReservationAsync(request);

    // Assert
    await act.Should().ThrowAsync<ArgumentException>()
        .WithMessage("Reservation duration must be at least 30 minutes.");
}

[Fact]
public async Task CreateReservation_WhenUserNotAuthorizedForSlotType_ShouldThrowInvalidOperationException()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();

```

```

// Seed a user with Regular access, but a VIP slot
await SeedUserAndSlotAsync(userId, slotId, SlotType.VIP, new List<SlotType> { SlotType.Regular });

var request = new ReservationRequestDTO
{
    ParkingSlotId = slotId,
    UserId = userId,
    StartTime = DateTime.UtcNow.AddHours(2),
    EndTime = DateTime.UtcNow.AddHours(4)
};

// Act
Func<Task> act = async () => await _service.CreateReservationAsync(request);

// Assert
await act.Should().ThrowAsync<InvalidOperationException>()
    .WithMessage("*authorized*"); // Matches "User is not authorized to book this VIP parking slot."
}

[Fact]
public async Task CreateReservation_WithAdjacentReservations_ShouldNotTriggerOverlap()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    await SeedUserAndSlotAsync(userId, slotId);

    var baseTime = DateTime.UtcNow.AddDays(1);

    // Existing reservation: 10:00 to 11:00
    _context.Reservations.Add(new Reservation
    {
        Id = Guid.NewGuid(),
        UserId = userId,
        ParkingSlotId = slotId,
        StartTime = baseTime,
        EndTime = baseTime.AddHours(1)
    });
    await _context.SaveChangesAsync();

    var request = new ReservationRequestDTO
    {
        ParkingSlotId = slotId,
        UserId = userId,
        StartTime = baseTime.AddHours(1), // Starts exactly when the other ends (11:00)
        EndTime = baseTime.AddHours(2)    // Ends at 12:00
    };

    // Act
    var result = await _service.CreateReservationAsync(request);

    // Assert
    result.Should().NotBeNull();
    result.StartTime.Should().Be(request.StartTime);

    var count = await _context.Reservations.CountAsync();
    count.Should().Be(2, "because the adjacent reservation should be saved alongside the original");
}

[Fact]
public async Task CreateReservation_WithPartiallyOverlappingReservation_ShouldThrowInvalidOperationException()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    await SeedUserAndSlotAsync(userId, slotId);

    var baseTime = DateTime.UtcNow.AddDays(1);

    // Existing reservation: 10:00 to 11:00
    _context.Reservations.Add(new Reservation
    {
        Id = Guid.NewGuid(),
        UserId = userId,
        ParkingSlotId = slotId,
        StartTime = baseTime,
        EndTime = baseTime.AddHours(1)
    });
    await _context.SaveChangesAsync();

```

```

var request = new ReservationRequestDTO
{
    ParkingSlotId = slotId,
    UserId = userId,
    StartTime = baseTime.AddMinutes(30), // Starts at 10:30 (Overlap)
    EndTime = baseTime.AddMinutes(90)    // Ends at 11:30
};

// Act
Func<Task> act = async () => await _service.CreateReservationAsync(request);

// Assert
await act.Should().ThrowAsync<InvalidOperationException>()
    .WithMessage("The parking slot is already booked for the requested time period.");
}

[Fact]
public async Task CreateReservation_WithValidData_ShouldSaveToDatabaseAndReturnDto()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    await SeedUserAndSlotAsync(userId, slotId);

    var request = new ReservationRequestDTO
    {
        ParkingSlotId = slotId,
        UserId = userId,
        StartTime = DateTime.UtcNow.AddHours(2),
        EndTime = DateTime.UtcNow.AddHours(4)
    };

    // Act
    var response = await _service.CreateReservationAsync(request);

    // Assert
    response.Should().NotBeNull();
    response.ParkingSlotId.Should().Be(slotId);
    response.UserId.Should().Be(userId);

    var dbEntity = await _context.Reservations.FirstOrDefaultAsync(r => r.Id == response.Id);
    dbEntity.Should().NotBeNull();
    dbEntity!.StartTime.Should().Be(request.StartTime);
    dbEntity.EndTime.Should().Be(request.EndTime);
}

[Fact]
public async Task CancelReservation_WhenReservationDoesNotExist_ShouldThrowKeyNotFoundException()
{
    // Arrange
    var nonExistentReservationId = Guid.NewGuid();
    var requestingUserId = Guid.NewGuid();

    // Act
    Func<Task> act = async () => await _service.CancelReservationAsync(nonExistentReservationId, requestingU

    // Assert
    await act.Should().ThrowAsync<KeyNotFoundException>()
        .WithMessage("*not found*");
}

[Fact]
public async Task CancelReservation_WhenUserDoesNotOwnReservation_ShouldThrowUnauthorizedAccessException()
{
    // Arrange
    var ownerId = Guid.NewGuid();
    var hackerId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    var reservationId = Guid.NewGuid();

    await SeedUserAndSlotAsync(ownerId, slotId);

    _context.Reservations.Add(new Reservation
    {
        Id = reservationId,
        UserId = ownerId,
        ParkingSlotId = slotId,
        StartTime = DateTime.UtcNow.AddHours(2),

```



```

        EndTime = DateTime.UtcNow.AddHours(3)
    });
    await _context.SaveChangesAsync();

    // Act - Trying to cancel with Hacker's ID
    Func<Task> act = async () => await _service.CancelReservationAsync(reservationId, hackerId);

    // Assert
    await act.Should().ThrowAsync<UnauthorizedAccessException>()
        .WithMessage("You are not authorized to cancel this reservation.");
}

[Fact]
public async Task CancelReservation_WhenStartTimeIsInThePast_ShouldThrowInvalidOperationException()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    var reservationId = Guid.NewGuid();

    await SeedUserAndSlotAsync(userId, slotId);

    // Add a reservation that has already happened
    _context.Reservations.Add(new Reservation
    {
        Id = reservationId,
        UserId = userId, // Owned by user
        ParkingSlotId = slotId,
        StartTime = DateTime.UtcNow.AddHours(-2), // Started 2 hours ago
        EndTime = DateTime.UtcNow.AddHours(-1)
    });
    await _context.SaveChangesAsync();

    // Act - Passed correct owner ID
    Func<Task> act = async () => await _service.CancelReservationAsync(reservationId, userId);

    // Assert
    await act.Should().ThrowAsync<InvalidOperationException>()
        .WithMessage("started or is in the past*");
}

[Fact]
public async Task CancelReservation_WhenStartTimeIsInTheFutureAndValidOwner_ShouldDeleteFromDatabase()
{
    // Arrange
    var userId = Guid.NewGuid();
    var slotId = Guid.NewGuid();
    var reservationId = Guid.NewGuid();

    await SeedUserAndSlotAsync(userId, slotId);

    // Add a reservation in the future
    _context.Reservations.Add(new Reservation
    {
        Id = reservationId,
        UserId = userId,
        ParkingSlotId = slotId,
        StartTime = DateTime.UtcNow.AddHours(2), // Starts in 2 hours
        EndTime = DateTime.UtcNow.AddHours(3)
    });
    await _context.SaveChangesAsync();

    // Act
    await _service.CancelReservationAsync(reservationId, userId);

    // Assert
    var deletedReservation = await _context.Reservations.FindAsync(reservationId);
    deletedReservation.Should().BeNull("because the valid future reservation should have been deleted by the");
}

public void Dispose()
{
    _context.Dispose();
    _connection.Close();
    _connection.Dispose();
}
}
}

```

---

User prompt: There are some errors in the tests that came up with the new development: 1) Message: System.Text.Json.JsonException : The input does not contain any JSON tokens. Expected the input to start with a valid JSON token, when isFinalBlock is true. Path: \$ | LineNumber: 0 | BytePositionInLine: 0. ---- System.Text.Json.JsonReaderException : The input does not contain any JSON tokens. Expected the input to start with a valid JSON token, when isFinalBlock is true. LineNumber: 0 | BytePositionInLine: 0. Stack Trace: ThrowHelper.ThrowWithPath(ReadStack& state, JsonReaderException ex) JsonSerializer`1.ReadCore(Utf8JsonReader& reader, T& value, JsonSerializerOptions options, ReadStack& state) JsonTypeInfo`1.Deserialize(Utf8JsonReader& reader, ReadStack& state) JsonSerializer.ReadFromSpan[TValue](ReadOnlySpan`1 utf8Json, JsonTypeInfo`1 jsonTypeInfo, Nullable`1 actualByteCount) JsonSerializer.ReadFromSpan[TValue](ReadOnlySpan`1 json, JsonTypeInfo`1 jsonTypeInfo) lambda\_method398(Closure, String) lambda\_method384(Closure, QueryContext, DbDataReader, ResultContext, SingleQueryResultCoordinator) Enumerator.MoveNext() Enumerable.TryGetSingle[TSource](IEnumerable`1 source, Boolean& found) lambda\_method382(Closure, QueryContext) QueryCompiler.ExecuteCore[TResult](Expression query, Boolean async, CancellationToken cancellationToken) QueryCompiler.Execute[TResult](Expression query) EntityQueryProvider.Execute[TResult](Expression expression) ReservationControllerTests.GetValidIds() line 31 ReservationControllerTests.DeleteReservation\_WithUnauthorizedUser\_ReturnsForbidden() line 158 --- End of stack trace from previous location --- ----- Inner Stack Trace ----- ThrowHelper.ThrowJsonReaderException(Utf8JsonReader& json, ExceptionResource resource, Byte nextByte, ReadOnlySpan`1 bytes) Utf8JsonReader.Read() JsonConverter`1.ReadCore(Utf8JsonReader& reader, T& value, JsonSerializerOptions options, ReadStack& state) 2) Message: Expected response.StatusCode to be HttpStatusCode.OK {value: 200}, but found HttpStatusCode.InternalServerError {value: 500}. Pinpoint the issues and suggest 3 solution paths to solve.

## Response: Pinpointing the Issues

Both of these errors trace back to the same root cause: **Entity Framework Core is failing to materialize the User entities from the database.**

When we added the AuthorizedSlotTypes list, we told EF Core to convert it to a JSON string. However, if the database has any rows where this column is null or an empty string (""), JsonSerializer.Deserialize throws a JsonException ("The input does not contain any JSON tokens").

- Error 1 (JsonException):** Happens directly in your test's GetValidIds() method when it calls .First() on the Users table. EF Core fetches the row, sees an empty string, tries to parse it as JSON, and crashes.
- Error 2 (500 Internal Server Error):** Happens because an integration test is calling an API endpoint (like POST to create a reservation), which queries the User internally. The same EF Core JSON crash occurs on the server side, resulting in a 500 crash response instead of a successful 200/201.

Here are 3 paths to solve this:

### Path 1: Make the EF Core Value Converter Null-Safe (Recommended)

The most robust solution is to update the value converter in your ApplicationDbContext to check for null or whitespace *before* attempting to deserialize the JSON.

**Update src/ParkingBackend/Parking.WebAPI/Data/ApplicationDbContext.cs:**

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<User>()
        .Property(u => u.AuthorizedSlotTypes)
        .HasConversion(
            v => JsonSerializer.Serialize(v, (JsonSerializerOptions)null),
            // Check if the string is empty/null before parsing
            v => string.IsNullOrEmpty(v)
                ? new List<SlotType>()
                : JsonSerializer.Deserialize<List<SlotType>>(v, (JsonSerializerOptions)null) ?? new List<SlotType>()
        );
}
```

### Path 2: Use Comma-Separated Strings Instead of JSON

Because you are only storing a simple list of Enums, using full JSON serialization is slightly overkill and prone to these exact parsing errors. You can store them as a simple comma-separated string (e.g., "0,1,2").

**Update the conversion logic in ApplicationDbContext.cs:**

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    base.OnModelCreating(modelBuilder);

    modelBuilder.Entity<User>()
        .Property(u => u.AuthorizedSlotTypes)
        .HasConversion(
            // Convert list to comma-separated string
            v => string.Join(",", v.Select(e => (int)e)),
            // Convert comma-separated string back to list
            v => string.IsNullOrEmpty(v)
                ? new List<SlotType>()
                : v.Split(',', StringSplitOptions.RemoveEmptyEntries)
                    .Select(s => (SlotType)int.Parse(s))
        );
}
```

```

        .ToList()
    );
}

```

### Path 3: Clear the Corrupted Test Database State

If you added this migration without clearing your integration test's database, the old test rows likely contain null for the new `AuthorizedSlotTypes` column. If you are using a physical SQLite file for testing (or a shared local database), EF Core is choking on the legacy data.

1. Locate and delete the physical `app.db` or `test.db` SQLite file used by your integration tests.
2. Ensure your `DatabaseSeeder.cs` never initializes a user with a null list (e.g., `AuthorizedSlotTypes = new List<SlotType>()`).
3. Run the tests again so EF Core can generate a fresh database where the default value defaults to a valid JSON array string (`[]`).

*User prompt: Last issue: System.InvalidOperationException : The LINQ expression 'DbSet<User>().Where(u => !(u.AuthorizedSlotTypes.Contains(VIP)))' could not be translated. Additional information: Translation of method 'System.Linq.Enumerable.Contains' failed.*

Response: This is a classic Entity Framework Core limitation.

### Why is this happening?

In our `ApplicationDbContext`, we used a **Value Converter** to store the `List<SlotType>` as a JSON string in the database. When you write a LINQ query like `.First(u => u.AuthorizedSlotTypes.Contains(VIP))`, EF Core tries to translate that into an SQL `SELECT` statement. However, because EF Core knows the column is actually a single text string in SQLite, it doesn't know how to translate a C# `List.Contains()` into an SQL query for that specific text format.

Fortunately, this crash is only happening in your **integration test setup** (`ReservationControllerTests.cs`), not in your actual application code (because the `ReservationService` pulls the user by ID into memory *before* checking the list).

### The Solution

We just need to tell EF Core to fetch the seeded users from the database into local memory first, and *then* run the `.Contains()` filter using standard C#. We do this by inserting `.AsEnumerable()` or `.ToList()` before the `.First()` or `.Where()` call.

### Update the Test File

Update the `PostReservation_WithUnauthorizedSlotType_ReturnsBadRequest` method in your `ReservationControllerTests.cs`:

```

[Fact]
public async Task PostReservation_WithUnauthorizedSlotType_ReturnsBadRequest()
{
    // Arrange
    var client = _factory.CreateClient();
    using var scope = _factory.Services.CreateScope();
    var dbContext = scope.ServiceProvider.GetRequiredService<ApplicationDbContext>();

    // FIX: Add .AsEnumerable() to pull the records into memory before applying the .Contains filter
    var regularUser = dbContext.Users
        .AsEnumerable()
        .First(u => !u.AuthorizedSlotTypes.Contains(SlotType.VIP));

    var vipSlot = dbContext.ParkingSlots
        .AsEnumerable()
        .First(s => s.Type == SlotType.VIP);

    var request = new ReservationRequestDto
    {
        ParkingSlotId = vipSlot.Id,
        UserId = regularUser.Id,
        StartTime = DateTime.UtcNow.AddDays(15),
        EndTime = DateTime.UtcNow.AddDays(15).AddHours(2)
    };

    // Act
    var response = await client.PostAsJsonAsync("/api/reservations", request);

    // Assert
    response.StatusCode.Should().Be(HttpStatusCode.BadRequest);

    var errorContent = await response.Content.ReadAsStringAsync();
    errorContent.Should().Contain("not authorized");
}

```

By adding `.AsEnumerable()`, EF Core will execute a simple `SELECT * FROM Users`, load the handful of seeded users into memory, parse the JSON back into the `List<SlotType>`, and then successfully run the C# `.Contains()` method!

